

Universidad Rafael Landívar

Facultad de Ingeniería

Ingeniería en Informática y Sistemas

Campus de Quetzaltenango

Ingeniería de Software I · Proyecto final 2026

Shapi

Manual técnico

Plataforma como servicio para publicar APIs, controlar su uso y cobrar por ellas

Integrantes del equipo 6

Héctor Jordin Adolfo García Coyoy	Carné 2427124
Ronaldo Emilio Méndez Mayorga	Carné 1563224
Dominique Guillermo Contreras Sierra	Carné 1508224
José Pablo Zúñiga de León	Carné 1507524

Quetzaltenango, 11 de noviembre de 2026

Índice

1. Introducción 5
1.1 Organización del documento 5
1.2 Documentos relacionados 5
1.3 Convenciones 5
2. Descripción del sistema 6
2.1 Categoría del enunciado 6
2.2 Problema y justificación 6
2.3 Actores 6
2.4 Funcionamiento 7
2.5 Modelo de negocio 7
3. Objetivos 9
3.1 Objetivo general 9
3.2 Objetivos específicos 9
4. Alcance 10
4.1 Incluido 10
4.2 Excluido 10
4.3 Restricciones 10
5. Requisitos funcionales y no funcionales 12
5.1 Requisitos funcionales 12
5.2 Requisitos no funcionales 14
5.3 Trazabilidad con los lineamientos del curso 15
6. Arquitectura 17
6.1 Estilo arquitectónico 17
6.2 Diagrama de contexto 17
6.3 Contenedores y procesos 19
6.4 Capas y proyectos 21
6.5 Tubería de filtros de la compuerta 22
6.6 Hosts y enrutamiento 22
6.7 Despliegue 22
6.8 Observabilidad y disponibilidad 25
6.9 Seguridad 25
6.10 Decisiones de arquitectura (ADR) 25
7. Definición del modelo 27
7.1 Modelo elegido 27
7.2 Justificación 27

7.3 Fases 27
7.4 Artefactos del proceso 28
7.5 Ciclo de vida de una tarea 28
7.6 Control de calidad 30
7.7 Roles del equipo 30
7.8 Resultado del proceso 30
8. Cronograma 31
8.1 Hitos del curso 31
8.2 Diagrama de Gantt 31
8.3 Cumplimiento por avance 32
8.4 Detalle por tarea 33
9. Diseño de interfaces 37
9.1 Sistema visual 37
9.2 Formato de los mockups 37
9.3 Pantallas principales 37
9.4 Catálogo completo de pantallas 50
10. Diagramas 54
10.1 Diagramas UML 54
10.2 Diagramas de flujo 72
10.3 Diagramas de bases de datos 76
11. Tecnologías utilizadas 86
11.1 Backend 86
11.2 Frontend 86
11.3 Datos, borde e infraestructura 87
11.4 Pruebas 87
11.5 Herramientas del equipo 88
12. Manual de instalación 89
12.1 Requisitos del equipo 89
12.2 Obtener el código 89
12.3 Configuración 90
12.4 Ambiente productivo simulado 90
12.5 Confiar en la autoridad certificadora local 92
12.6 Siembra y cuentas de demostración 92
12.7 Modo demostración y reloj 94
12.8 Entorno de desarrollo 94
12.9 Pruebas 95
12.10 Integración continua y publicación 96

12.11 Despliegue en Azure 96
13. Problemas que se pueden presentar 101
14. Respaldo y recuperación de datos 103
14.1 Qué se respalda 103
14.2 Respaldar la base de datos 103
14.3 Respaldar los volúmenes 104
14.4 Recuperar después de una pérdida total 104
14.5 Volver a un punto anterior 104
14.6 Recomendaciones 105
15. Anexos 106
Anexo A · Variables de entorno 106
Anexo B · Estructura del repositorio 108
Anexo C · Comandos frecuentes 108
Anexo D · Referencias 108

1. Introducción

Este manual técnico describe **Shapi**, la plataforma como servicio que el equipo 6 desarrolló como proyecto final de Ingeniería de Software I de la Universidad Rafael Landívar, campus de Quetzaltenango, en el segundo ciclo de 2026.

El manual está dirigido a quien tenga que **entender, instalar, operar o continuar** el sistema: la docente del curso, los integrantes del equipo y cualquier desarrollador que reciba el proyecto. Reúne en un solo documento qué hace el sistema, cómo está construido, cómo se planificó y cómo se instala, configura, respalda y recupera.

1.1 Organización del documento

Capítulos	Contenido
1 a 5	El sistema: descripción, objetivos, alcance y requisitos funcionales y no funcionales
6 y 7	Cómo se construyó: la arquitectura y el modelo de proceso de desarrollo
8	El cronograma pronosticado y el real
9 y 10	El diseño: las pantallas aprobadas (mockups) y los diagramas UML, de flujo y de la base de datos
11 a 14	La operación: tecnologías, instalación, problemas frecuentes y respaldo de los datos
15	Anexos: variables de entorno, estructura del repositorio, comandos frecuentes y referencias

1.2 Documentos relacionados

Este manual resume y enlaza las **especificaciones** del proyecto, que son la fuente de verdad: si el código las contradice, el código está mal. Están en `docs/specs/` del repositorio y se entregan también en PDF:

- **Documento de Requisitos** (`docs/pdf/Documento_de_Requisitos.pdf`): visión y alcance, glosario, requisitos, roles y permisos, y casos de uso.
- **Documento de Diseño** (`docs/pdf/Documento_de_Diseno.pdf`): arquitectura, modelo de datos, compuerta, cobros y suscripciones, identidad y seguridad, interfaz y decisiones de diseño (ADR).
- **Manual de Usuario** (tarea JZ-16, en `docs/manual-usuario.md`): el uso del sistema por cada actor.

Este manual se entrega en Markdown (`docs/manual-tecnico.md`, que GitHub muestra con sus diagramas) y en Word (`docs/Manual_Tecnico.docx`). El código fuente está en el repositorio github.com/jordin-garcia/shapi.

1.3 Convenciones

- Los comandos se escriben en bloques de código y se ejecutan **desde la raíz del repositorio**, salvo que se indique otra carpeta.
- Cuando un comando cambia entre sistemas operativos, se dan las dos versiones: **PowerShell** (Windows) y **bash** (Linux, macOS o Git Bash).
- Los requisitos se citan por su código (`RF-xx` funcionales y `RNF-xx` no funcionales); las pantallas, por su código de mockup (por ejemplo, A3.2); y las decisiones de diseño, por su número de ADR.
- Los montos están en quetzales (Q) y las fechas y horas, en la hora de Guatemala.

2. Descripción del sistema

Shapi es una plataforma como servicio que permite a una empresa **publicar su API, controlar quién la usa y cobrar por ella, sin modificar el servidor que ya tiene**. La empresa conecta su servidor, sube su especificación OpenAPI y define planes. Shapi le crea un portal con la marca de esa empresa, emite las claves de acceso, aplica las cuotas en cada petición y registra el consumo y los pagos.

2.1 Categoría del enunciado

De las tres categorías del enunciado del curso se eligió **Servidores Web como Servicio**, y la docente aprobó la propuesta. El recurso que se contrata es un **entorno web gestionado por API**, que Shapi crea, configura y monitorea desde una interfaz web. Cada entorno tiene un subdominio para el portal y otro para la API (ambos con certificado TLS), el enrutamiento hacia el servidor de origen, un portal web alojado por la plataforma y el monitoreo de peticiones, latencia, errores y datos transferidos.

Lo que exige el enunciado	Cómo lo cumple Shapi
Crear entornos web	Cada API registrada crea un entorno con subdominio, certificado, rutas y portal (RF-08 a RF-11)
Gestionar configuraciones	Origen, rutas, límites, caché, planes, claves, marca, dominio propio y secreto de origen
Gestión de dominios simulados	Subdominios bajo `shapi.localhost` y dominios propios verificados con un DNS simulado
Monitorear el consumo de recursos	Peticiones, llamadas, latencia p95, errores por código y bytes transferidos (RF-35)

2.2 Problema y justificación

Una empresa que ya tiene un servicio funcionando puede convertirlo en un ingreso. Pero entre un servidor que responde y un producto que alguien pueda contratar siempre hay que construir lo mismo: claves, límites de uso, medición, documentación, planes y cobro. Esa parte no diferencia a nadie y cuesta semanas de trabajo.

Hay plataformas internacionales que ya lo resuelven, pero cobran a través de pasarelas que no admiten empresas guatemaltecas, o imponen su marca y se quedan con una comisión del 20 % al 25 %. La propuesta de valor de Shapi es:

- una suscripción fija, sin comisión;
- el portal conserva la marca del proveedor;
- la interfaz está en español;
- el cobro queda detrás de una interfaz de pasarela de pagos (`IPasarelaPagos`). En esta versión académica la pasarela es un **simulador**, como permite el enunciado, y la arquitectura permite conectar después una pasarela local sin cambiar el resto del sistema.

2.3 Actores

Actor	Quién es	Dónde trabaja
Proveedor	La empresa que publica y cobra su API. Es el **cliente** de Shapi. Dentro de su organización, cada persona es propietario, editor o lector	Panel de Shapi (`shapi.localhost/panel`)

Actor	Quién es	Dónde trabaja
Consumidor	El desarrollador o la empresa que contrata un plan y usa la API. Su cuenta pertenece a la organización proveedora; no se registra en Shapi	Portal de marca blanca (`{sub}.shapi.localhost`) y la API (`{sub}.api.shapi.localhost`)
Administrador	Quien opera la plataforma	Panel de administración (`shapi.localhost/admin`)
Soporte	Quien atiende los casos de los proveedores, con acceso de solo lectura a los datos de las organizaciones	Panel de administración, con menos opciones
Sistema	Los procesos automáticos: la compuerta y el trabajador en segundo plano	Sin interfaz

2.4 Funcionamiento

El proveedor:

1. Se registra, confirma su correo y su organización queda en el plan **Prueba** (30 días, sin tarjeta).
2. Registra una API con un nombre, la URL de su servidor de origen y un subdominio que él elige. Shapi prueba la conexión con el origen antes de guardar.
3. Sube su especificación OpenAPI. Shapi extrae las rutas y el proveedor decide cuáles expone.
4. Configura cada ruta (límite por minuto, caché y peso en llamadas) y personaliza el portal (logotipo, color, nombre y texto de bienvenida).
5. Define los planes de su API y la publica.
6. Consulta el consumo, la facturación por consumidor y su historial de pagos. Puede invitar miembros y consumidores, y abrir casos de soporte.

El consumidor:

1. Entra al portal del proveedor, revisa la documentación y crea su cuenta, o acepta una invitación.
2. Confirma su correo, elige un plan y paga con tarjeta. Los planes gratuitos no piden tarjeta.
3. Recibe una clave de producción y una de pruebas, que se muestran completas una sola vez.
4. Llama a la API con la cabecera `X-Api-Key`. Desde el portal consulta su consumo por ruta y sus pagos, rota o revoca sus claves y cambia de plan.

Los procesos automáticos:

- **La compuerta** valida cada petición contra Redis, aplica los límites y las cuotas, reenvía la petición al origen y registra las métricas.
- **El trabajador** consolida las métricas en PostgreSQL cada 10 segundos; cierra los ciclos de las suscripciones y las renueva, las pone en gracia o las suspende; envía los correos pendientes, y verifica los dominios propios.

2.5 Modelo de negocio

Shapi cobra una **suscripción fija** a cada proveedor, sin comisión. Cada proveedor define además los planes que vende a sus consumidores (dos niveles de planes), y Shapi les cobra a los consumidores por cuenta del proveedor.

Plan de plataforma	Precio	Vigencia	APIs	Peticiones por ciclo	Miembros	Dominio propio
Prueba	Q 0.00	30 días, no se renueva	1	10,000	1	No
Lanzamiento	Q 199.00	30 días	3	250,000	3	No
Producto	Q 599.00	30 días	10	2,000,000	10	Sí

Plan de plataforma	Precio	Vigencia	APIs	Peticiones por ciclo	Miembros	Dominio propio
Escala mensual	Q 1,500.00	30 días	Sin límite	10,000,000	Sin límite	Sí
Escala anual	Q 15,000.00	365 días	Sin límite	120,000,000	Sin límite	Sí

La cuota de plataforma se cuenta en **peticiones** y no en llamadas, para que el proveedor no pueda saltársela poniéndoles peso 0 a sus rutas.

3. Objetivos

3.1 Objetivo general

Diseñar, desarrollar, documentar e implementar una plataforma como servicio que permita a una empresa publicar su API, administrar a sus consumidores mediante planes y claves de acceso, y cobrar por su uso desde un portal con su propia marca.

3.2 Objetivos específicos

1. Levantar los requisitos funcionales y no funcionales del sistema y modelar los casos de uso de sus actores: proveedor, consumidor, administrador, soporte y sistema.
2. Diseñar la arquitectura, el modelo entidad-relación y la base de datos, y justificar cada decisión.
3. Implementar la gestión de usuarios: registro, verificación de correo, recuperación de contraseña, autenticación con sesiones y control de roles, tanto para el personal como para los consumidores.
4. Desarrollar la compuerta de tráfico: validar claves, aplicar límites por minuto y cuotas, y medir el consumo de cada consumidor.
5. Implementar las suscripciones y los pagos: ver planes, contratar, historial de pagos, renovación automática, gracia y suspensión, con una pasarela simulada.
6. Generar automáticamente, a partir de la especificación OpenAPI de cada proveedor, un portal de marca blanca con documentación y consola de pruebas.
7. Desplegar el sistema en un ambiente productivo simulado y reproducible, y elaborar el manual técnico y el manual de usuario.

4. Alcance

4.1 Incluido

- Gestión de usuarios con registro, verificación de correo, recuperación de contraseña, sesiones, roles y organizaciones, **tanto para el personal como para los consumidores**.
- Publicación de APIs con especificación OpenAPI, rutas expuestas, subdominios, dominio propio verificado con DNS simulado y secreto de origen.
- Un portal de marca blanca por API, con inicio, documentación generada, consola de pruebas, planes y la cuenta del consumidor.
- La compuerta: validación de claves, límites por minuto, cuotas del consumidor y de la plataforma, caché opcional, reenvío al origen y medición.
- Planes en dos niveles, contratación con pasarela simulada, cambio de plan con prorrateo, renovación automática, periodo de gracia, suspensión y reversión de pagos.
- Paneles de consumo para el proveedor y el consumidor, el panel de administración, la bandeja de casos de soporte (del lado del proveedor y del lado de soporte), el estado de los componentes y la bitácora de acciones sensibles.
- Notificaciones por correo a través de un servidor de correo simulado (Mailpit).
- Un **modo demostración** con un reloj de negocio que se puede adelantar, para mostrar renovaciones, gracia y suspensiones sin esperar 30 días.
- Despliegue reproducible con Docker Compose en un **ambiente productivo simulado**, con los datos de demostración de los mockups.

4.2 Excluido

- Integración con un procesador de pagos real. Se usa la pasarela simulada que permiten los lineamientos.
- Liquidarle al proveedor lo que cobran sus consumidores.
- Emisión de documentos tributarios electrónicos.
- Dominios y certificados públicos reales. Todo corre en `*.localhost` con la autoridad certificadora interna del borde.
- Un mercado público de APIs: el portal siempre es de marca blanca.
- Ejecución de código subido por los clientes. La plataforma solo enruta tráfico.
- Verificación en dos pasos.
- Que un usuario pertenezca a más de una organización o que se transfiera la propiedad de una organización.
- Que el consumidor cancele la renovación automática o que el proveedor bloquee a un consumidor completo (el proveedor revoca claves y las suspensiones solo ocurren por falta de pago).
- Bibliotecas propias por lenguaje, alta disponibilidad en varias regiones y aplicación móvil.

4.3 Restricciones

- El presupuesto es **Q 0.00**: no se compran dominios, servidores ni servicios.
- El stack es el que permiten los lineamientos: .NET 10 (ASP.NET Core), React y PostgreSQL. También se usan Redis, YARP y Caddy.

- Las entregas son el Avance 1 (30 %) el 25 de septiembre, el Avance 2 (50 %) el 9 de octubre, el Avance 3 (80 %) el 31 de octubre, como video, y la entrega final en la primera semana de noviembre de 2026.
- El desarrollo es **dirigido por especificaciones** y lo ejecutan agentes de inteligencia artificial bajo la supervisión del equipo (capítulo 7).

5. Requisitos funcionales y no funcionales

Los requisitos completos, con su actor, su criterio de aceptación y sus pantallas, están en `docs/specs/03-requisitos.md` y en el Documento de Requisitos. Aquí se listan con su código y su enunciado. Los códigos RF-01 a RF-42 conservan el número del documento de requisitos del 11 de septiembre; RF-43 a RF-47 se agregaron después.

5.1 Requisitos funcionales

5.1.1 Gestión de usuarios y acceso

ID	Requisito
RF-01	El sistema permitirá registrar un proveedor con nombre, correo, nombre de la organización y contraseña. Creará la organización, dejará al usuario como **propietario** y pondrá a la organización en el plan **Prueba** sin pedirle tarjeta.
RF-02	El sistema enviará un enlace de verificación al correo del proveedor o del consumidor. El enlace vence a las 24 horas y puede pedirse de nuevo. Mientras no se confirme el correo, **el proveedor no puede publicar APIs y el consumidor no puede contratar planes**.
RF-03	El sistema permitirá recuperar la contraseña con un enlace de un solo uso que vence a los 60 minutos. **Al usarlo se cierran todas las demás sesiones** de esa cuenta.
RF-04	El sistema autenticará a los usuarios con correo y contraseña y mantendrá una sesión que vence tras un tiempo de inactividad configurable (8 h por defecto) o a los 7 días como máximo. El personal puede cerrar sesión, cambiar su nombre y cambiar su contraseña.
RF-05	El sistema permitirá registrar consumidores desde el portal de una API o **por invitación que el proveedor envía desde su panel**. La invitación vence a los 7 días.
RF-06	El sistema manejará estos roles: `administrador` y `soporte` en la organización de plataforma; `propietario`, `editor` y `lector` en las organizaciones proveedoras; y **consumidor** como identidad aparte. El propietario puede invitar miembros, cambiar su rol y quitarlos.
RF-07	El sistema restringirá cada operación según el rol y la organización del usuario, conforme a la matriz de permisos.

5.1.2 Publicación y configuración de APIs

ID	Requisito
RF-08	El sistema permitirá registrar una API con nombre, URL del servidor de origen y **subdominio**. Antes de guardar, validará la URL y **probará la conexión** con el origen.
RF-09	El sistema permitirá cargar una especificación OpenAPI 3.0 o 3.1, en JSON o YAML y de hasta 2 MB, y extraerá sus rutas. **Si se vuelve a cargar**, las rutas que ya existían conservan su configuración, las nuevas quedan **ocultas** y las que ya no aparecen se eliminan.
RF-10	El sistema permitirá marcar cada ruta como expuesta u oculta.
RF-11	El sistema servirá cada API en dos hosts: `{sub}.api.{dominio_base}` para la API y `{sub}.{dominio_base}` para el portal. Los dos tendrán TLS **emitido y renovado automáticamente por el borde**.
RF-12	El sistema permitirá conectar un **dominio propio a la API** si el plan de plataforma lo incluye. La propiedad del dominio se verifica con un registro CNAME que apunta a `{sub}.api.{dominio_base}`. En el entorno simulado, el registro se crea en el **DNS simulado** y el dominio debe terminar en `.localhost`.
RF-13	El sistema permitirá configurar en cada ruta el **límite por minuto** (en peticiones; si se deja vacío, la ruta no tiene límite propio), la **caché** (en segundos, solo para GET; 0 la desactiva) y el **peso en llamadas** (1 o más).
RF-14	El sistema permitirá publicar y despublicar una API sin eliminar sus datos.
RF-15	El sistema permitirá personalizar el portal **de cada API** con logotipo (PNG o SVG de hasta 512 KB), color principal, nombre del portal y texto de bienvenida.

ID	Requisito
RF-16	El sistema generará el portal automáticamente: inicio, documentación por ruta (descripción, parámetros y ejemplos sacados de la especificación, más el peso en llamadas) y consola de pruebas. **En la consola, el consumidor pega su clave de pruebas**, que solo se recuerda mientras la pestaña esté abierta.

5.1.3 Suscripciones y pagos

ID	Requisito
RF-17	El sistema permitirá al administrador crear y editar planes de plataforma con nombre, descripción, precio, vigencia en días, máximo de APIs, máximo de miembros, cuota de peticiones y si incluyen dominio propio.
RF-18	El sistema permitirá al proveedor crear y editar planes de API con nombre, descripción, precio, vigencia, cuota de llamadas por ciclo y límite de peticiones por minuto, y marcarlos como gratuitos.
RF-19	El sistema mostrará los planes disponibles al proveedor y al consumidor antes de contratar.
RF-20	El sistema permitirá contratar un plan con tarjeta y activará la suscripción **en el momento en que la pasarela autorice el cobro**. Los planes gratuitos se activan sin pedir tarjeta. Si el cobro se rechaza, no se crea la suscripción y la suscripción vigente no cambia.
RF-21	El sistema registrará el historial de pagos de cada organización y de cada consumidor.
RF-22	El sistema renovará automáticamente las suscripciones al cerrar cada ciclo, cobrando al medio de pago registrado.
RF-23	El sistema aplicará un **periodo de gracia de 7 días** cuando se rechace el cobro de una renovación y **suspenderá** la suscripción si no se paga en ese plazo. Mientras dure la gracia, el servicio sigue funcionando. Si se suspende la plataforma de una organización, **todas sus APIs responden 403**; si se suspende la suscripción de un consumidor, **sus claves responden 403**. Un cobro autorizado reactiva la suscripción con un ciclo nuevo desde ese día.
RF-24	El sistema permitirá al administrador **revisar los pagos de plataforma y revertir** un cobro autorizado.
RF-25	El sistema permitirá cambiar de plan. **Subir** de plan se cobra prorrateado y se aplica de inmediato. **Bajar** de plan se aplica en la siguiente renovación, sin reembolso. **No se puede bajar** a un plan si la organización excede sus límites (por ejemplo, si tiene más APIs publicadas de las que el plan permite). Si el plan de destino no incluye dominio propio, los dominios se desconectan al aplicar el cambio; hasta entonces se conservan.

5.1.4 Compuerta de tráfico

ID	Requisito
RF-26	El sistema emitirá una clave de producción y una de pruebas al activarse una suscripción de API y **las mostrará completas una sola vez**.
RF-27	El sistema permitirá **al consumidor** rotar una de sus claves. La clave nueva se muestra una sola vez y la anterior sigue funcionando 24 horas.
RF-28	El sistema permitirá revocar una clave, al **consumidor** si es suya y al **proveedor** si es de uno de sus consumidores. La revocación se hace efectiva en **menos de 10 segundos**. Después de revocar una clave, el consumidor puede emitir una nueva del mismo tipo.
RF-29	El sistema validará en cada petición que la API esté publicada, la clave, el estado de la organización, el estado de la suscripción y que la ruta esté expuesta. Rechazará con **404, 401 o 403** según corresponda.
RF-30	El sistema rechazará con **429** las peticiones que superen el límite por minuto (del plan o de la ruta), las que agoten la cuota del consumidor y las que agoten la cuota de plataforma del proveedor.
RF-31	El sistema reenviará al origen solo las peticiones que pasen todos los filtros. Antes de reenviarlas quitará `X-Api-Key` y agregará `X-Shapi-Consumidor`, `X-Shapi-Entorno` y, si existe, `X-Shapi-Secreto`.

ID	Requisito
RF-32	El sistema devolverá al consumidor cabeceras con el plan vigente, el límite por minuto y la cuota restante.
RF-33	El sistema registrará de cada petición la API, la ruta, la suscripción, el entorno, el código de respuesta, la latencia total, la latencia de la compuerta y los bytes de entrada y de salida.
RF-34	El sistema consolidará cada 10 segundos las métricas acumuladas en Redis en la tabla `consumo_diario`, sin perder datos y sin contarlos dos veces.

5.1.5 Consulta, administración y soporte

ID	Requisito
RF-35	El sistema mostrará al proveedor, por cada API y periodo: peticiones, llamadas descontadas, **latencia total p95 y latencia de la compuerta p95** (sacadas de histogramas), errores por código (401/403/429 de la compuerta y 4xx/5xx/fallas de conexión del origen), y **bytes transferidos**.
RF-36	El sistema mostrará al proveedor el consumo del ciclo y la facturación de cada consumidor.
RF-37	El sistema mostrará al consumidor su consumo del ciclo, desglosado por ruta: peticiones, peso y llamadas descontadas.
RF-38	El sistema permitirá al administrador listar organizaciones y suspenderlas o reactivarlas por motivos administrativos.
RF-39	El sistema mostrará al administrador y al soporte el estado de cada componente y la latencia p95 que añade la compuerta.
RF-40	El sistema permitirá **al proveedor abrir casos de soporte y conversar en ellos**, y **al soporte y al administrador** registrar casos a nombre de una organización, responder, asignarse casos y cerrarlos, con acceso de **solo lectura** a los datos de la organización.
RF-41	El sistema registrará en una bitácora las acciones sensibles, indicando actor, organización, acción, objetivo, fecha e IP.
RF-42	El sistema permitirá al administrador crear, desactivar y reactivar cuentas de administración y de soporte. Estas cuentas no tienen registro público.
RF-43	El sistema aplicará los **límites del plan de plataforma**: máximo de APIs registradas, máximo de miembros (contando las invitaciones pendientes), dominio propio y cuota de peticiones. Avisará en el panel cuando se llegue al 80 % y al 100 % de la cuota.
RF-44	El plan **Prueba** vence a los 30 días y no se renueva. Si en ese plazo la organización no contrata un plan de pago, su suscripción pasa a gracia y después a suspensión, como en RF-23.
RF-45	La **clave de pruebas** llega al mismo origen con `X-Shapi-Entorno: pruebas`, **no descuenta cuota ni se factura**, y tiene un límite fijo de 10 peticiones por minuto y 1,000 por día.
RF-46	El sistema enviará correos de verificación, recuperación, invitación (a miembros, a consumidores y a cuentas de plataforma), pago rechazado, entrada en gracia, suspensión, aviso de vencimiento de la Prueba y respuesta a un caso.
RF-47	El sistema generará para cada API un **secreto de origen**, que se muestra una sola vez y puede regenerarse, y lo enviará en `X-Shapi-Secreto` en cada petición reenviada.

5.2 Requisitos no funcionales

Están ordenados según la norma ISO/IEC 25010 y cada uno tiene un criterio que se puede medir, salvo RNF-06 y RNF-11, que son objetivos de diseño y dicen qué los respalda.

ID	Requisito
RNF-01	**Eficiencia.** La compuerta no añadirá más de **15 ms en el percentil 95** al tiempo de respuesta del origen. Se mide con una prueba de carga contra un origen de demostración.
RNF-02	**Eficiencia.** Validar una petición **no consultará PostgreSQL**; todo se resuelve contra Redis. Se verifica con una prueba que apaga PostgreSQL.

ID	Requisito
RNF-03	**Eficiencia.** El sistema soportará al menos **200 peticiones por segundo** sostenidas durante 5 minutos, en el equipo de despliegue, con menos del 1 % de errores de la compuerta.
RNF-04	**Fiabilidad.** La compuerta seguirá enrutando aunque la API de control, el trabajador o PostgreSQL estén fuera de servicio, siempre que Redis esté disponible.
RNF-05	**Fiabilidad.** Una caída del trabajador no hará perder datos de consumo ni de facturación. Redis se configura con persistencia AOF (`appendfsync everysec`) y la consolidación es idempotente. Si Redis pierde sus datos, la caché se reconstruye desde PostgreSQL.
RNF-06	**Fiabilidad.** La disponibilidad mensual objetivo del enrutamiento es del **99.5 %** en un despliegue real. Es un **objetivo de diseño**: el ambiente productivo simulado no corre de forma continua, así que no se mide. Lo respaldan tres mecanismos: el reinicio automático y las verificaciones de salud de los contenedores (06 §7); que la compuerta siga enrutando aunque fallen la API de control, el trabajador o PostgreSQL (RNF-04); y el estado de los componentes de B3.1 (06 §8).
RNF-07	**Seguridad.** Las contraseñas se guardan con **PBKDF2** (el hasher de ASP.NET Core Identity, v3). Las claves de API se guardan con **hash SHA-256** y se muestran una sola vez. Los tokens de correo y los identificadores de sesión se guardan con hash. El secreto de origen se guarda **cifrado** con Data Protection. Nunca se guardan datos completos de tarjetas.
RNF-08	**Seguridad.** Ninguna consulta devolverá datos de una organización distinta a la del solicitante. Se aplica con un filtro global de EF Core por organización y se verifica con pruebas automatizadas de aislamiento.
RNF-09	**Seguridad.** Todo el tráfico se servirá por **HTTPS**, con certificados que el borde emite y renueva automáticamente antes de su vencimiento (en el entorno simulado, con la autoridad certificadora interna de Caddy). HTTP redirige a HTTPS.
RNF-10	**Seguridad.** La plataforma **no ejecutará código de los clientes**; solo enruta tráfico. Las URL de origen no pueden resolver a direcciones de loopback, privadas, link-local ni de la red interna de Docker. Esto se valida **al guardar la URL y al abrir cada conexión**, como protección contra SSRF.
RNF-11	**Usabilidad.** Publicar una API, desde el registro hasta la primera respuesta válida, tomará **menos de 5 minutos**. Es un **objetivo de diseño** y no se mide. Lo respalda un flujo sin intervención del administrador ni del soporte de Shapi y sin esperas de aprobación (05, CU-01, CU-05, CU-09, CU-11, CU-12 y CU-14): el proveedor y el consumidor verifican su correo con un enlace, la conexión con el origen se prueba al registrar la API, publicar no necesita aprobación, los hosts responden en menos de 10 segundos (RF-11), un plan gratuito se activa sin pedir tarjeta (RF-20) y las claves se entregan al activarse la suscripción (RF-26).
RNF-12	**Usabilidad.** Las interfaces estarán en español y funcionarán en la versión actual de Chrome, Edge y Firefox de escritorio, desde 1280 px de ancho.
RNF-13	**Mantenibilidad.** Agregar una regla de validación a la compuerta consistirá en **agregar un filtro** a la tubería sin modificar los demás. La cuota de plataforma (RF-30) sirve de ejemplo.
RNF-14	**Portabilidad.** Todo el sistema se levantará en cualquier equipo con **un solo comando** (`docker compose up`), con los datos de siembra de los mockups.
RNF-15	**Mantenibilidad.** Las reglas de negocio del dominio, cada filtro de la compuerta y el aislamiento entre organizaciones tendrán pruebas automatizadas que se ejecutan en la integración continua. La integración ordinaria a `main` exige todos los checks en verde. La protección conserva `enforce_admins: false` para que únicamente el coordinador, a mano y ante un falso positivo documentado de `revision-claude`, pueda usar la excepción de protocolo B11; los agentes nunca la usan ni omiten pruebas. Los jobs que construyen código de PR no reciben `packages: write`; solo la publicación tras un push a `main` lo recibe (JG-27).

5.3 Trazabilidad con los lineamientos del curso

Lineamiento	Requisitos
Registro, validación de correo, recuperación de contraseña	RF-01, RF-02, RF-03, RF-05
Autenticación segura y manejo de sesiones	RF-04, RNF-07
Roles (Administrador y Cliente como mínimo; Soporte es opcional)	RF-06, RF-07, RF-42
Servicios con nombre, descripción, precio y vigencia	RF-17, RF-18

Lineamiento	Requisitos
Visualización de planes, contratación, historial de pagos y renovación	RF-19, RF-20, RF-21, RF-22, RF-23, RF-25
Crear entornos web	RF-08 a RF-11, RF-14
Gestionar configuraciones	RF-12, RF-13, RF-15, RF-47
Monitorear consumo de recursos	RF-35, RF-36, RF-37, RF-39
Despliegue en ambientes productivos	RNF-09, RNF-14 y el capítulo 6.7

6. Arquitectura

6.1 Estilo arquitectónico

La solución separa el **plano de control** del **plano de datos**. Se despliegan como procesos independientes y solo comparten PostgreSQL y Redis. Cada plano usa una arquitectura distinta porque sus atributos de calidad son distintos:

Plano	Qué prioriza	Estilo	Por qué
Datos (la compuerta)	Latencia de 15 ms o menos en el p95, disponibilidad y extensibilidad de las reglas	**Tuberías y filtros**	Cada regla es un filtro independiente que deja pasar la petición o la detiene. Una regla nueva es un filtro más (RNF-13). No consulta PostgreSQL, solo Redis (RNF-02)
Control (API de control y trabajador)	Correctitud, trazabilidad y reglas de negocio	**Tres capas**: presentación, lógica de negocio y acceso a datos	Separa la API HTTP de las reglas del dominio y de la persistencia, para poder probar las reglas sin infraestructura

Se descartaron tres alternativas (ADR-27):

- **Un monolito**: si la compuerta viviera en el mismo proceso que el panel, una caída o un despliegue del panel detendría el tráfico de los clientes (incumple RNF-04).
- **Microservicios por dominio**: para 4 personas y 6 semanas de desarrollo cuestan más en operación de lo que aportan.
- **Una compuerta existente** (Kong, Tyk): el curso pide desarrollo *full stack* y la compuerta es el núcleo del producto. Con YARP se programa en C# y se reutiliza el modelo de dominio.

6.2 Diagrama de contexto

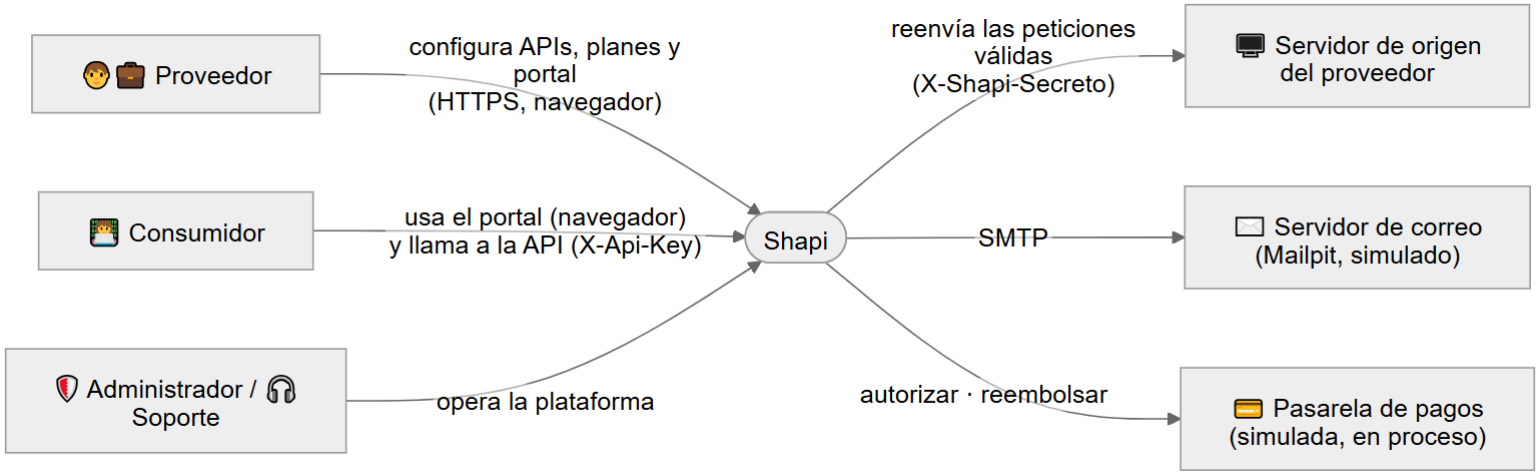


Diagrama 1. Diagrama de contexto.

6.3 Contenedores y procesos

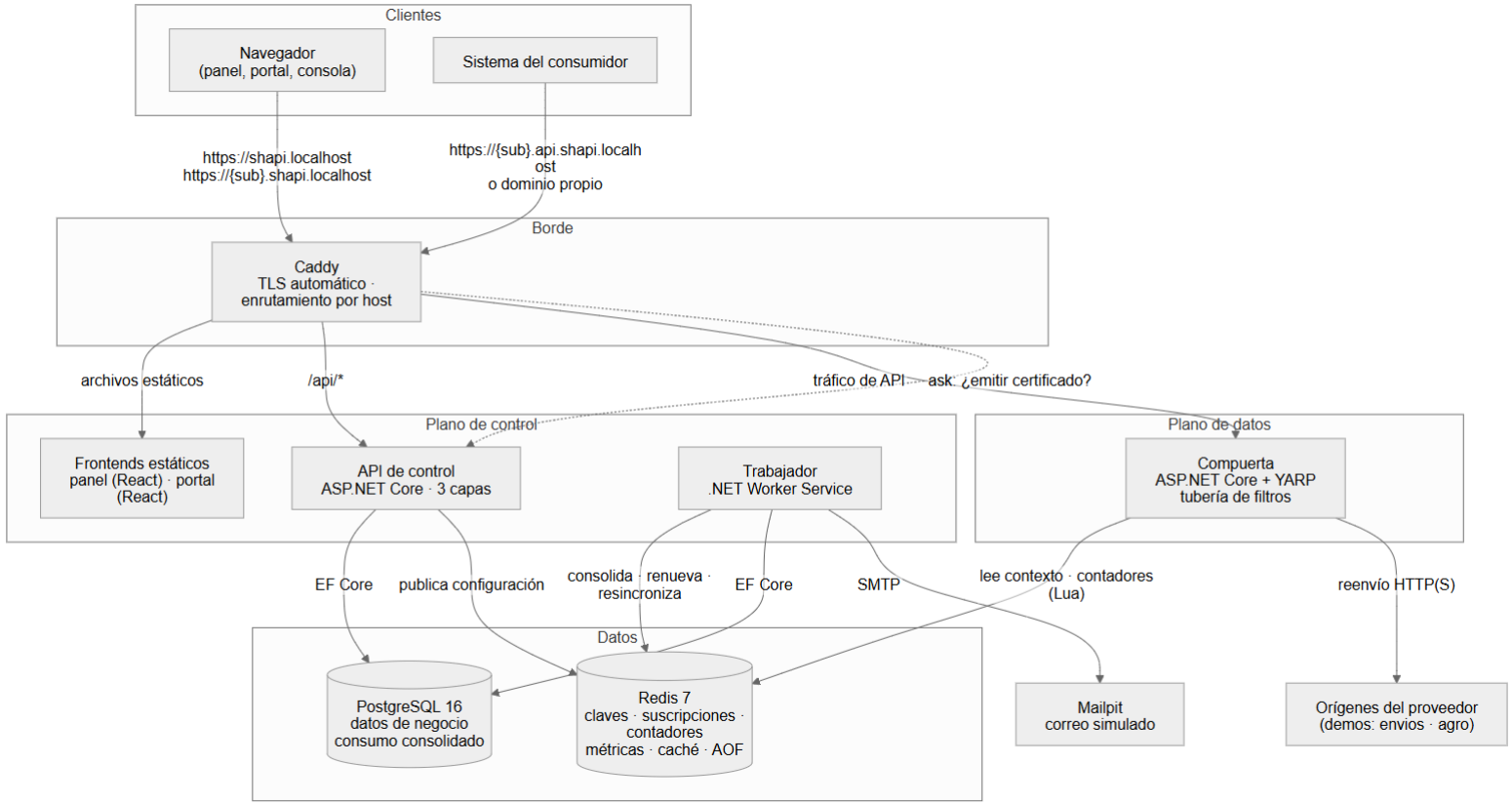


Diagrama 2. Contenedores y procesos.

Proceso	Tecnología	Responsabilidad	Si se cae
Borde	Caddy 2	Termina TLS, emite certificados automáticamente, redirige HTTP a HTTPS, enruta por host y sirve los frontends	Se cae todo el sistema; es el único punto de entrada
Compuerta	.NET 10 + YARP	Tubería de filtros, reenvío al origen, cabeceras y métricas	Se detiene el tráfico de las APIs; el panel sigue funcionando
API de control	.NET 10, ASP.NET Core	API REST del panel, del portal y de la administración; reglas de negocio; publica la configuración en Redis	No se puede administrar nada, pero **el tráfico sigue** (RNF-04)
Trabajador	.NET 10 Worker	Consolida las métricas cada 10 s, cierra ciclos cada minuto, envía correos cada 5 s, verifica dominios cada 5 min y resincroniza Redis cada 5 min	Se pausan la consolidación y las renovaciones, **sin perder datos** (RNF-05)
Frontends	React 19 + TypeScript + Vite	`panel`: sitio público, panel del proveedor y administración. `portal`: portal de marca blanca, que identifica la API por el host	—
Redis	Redis 7.4, AOF `everysec`	Vista de lectura de la compuerta, contadores atómicos, métricas pendientes y caché de respuestas	Se detiene la compuerta. Al volver, el trabajador reconstruye la caché desde PostgreSQL
PostgreSQL	PostgreSQL 16	Fuente de verdad de los datos de negocio	Se detiene la administración; **la compuerta sigue funcionando**
Mailpit	Mailpit	Servidor SMTP y bandeja web de correo simulado	Los correos esperan en `correo_saliente` y el estado de los componentes marca el correo como degradado

6.4 Capas y proyectos

El backend es una solución .NET (`Shapi.slnx`) con siete proyectos. Las dependencias apuntan siempre hacia el dominio:

Proyecto	Capa	Contenido
`Shapi.Dominio`	Lógica de negocio	Entidades, reglas y máquinas de estado, sin dependencias externas
`Shapi.Aplicacion`	Lógica de negocio	Casos de uso e interfaces de los servicios externos: `IPasarelaPagos`, `IResolutorDns`, `IReloj`, `IColaCorreo` e `IPublicadorCache`
`Shapi.Infraestructura`	Acceso a datos	EF Core con Npgsql, migraciones, Redis, pasarela simulada, SMTP y DNS
`Shapi.Api`	Presentación	API REST (panel, portal, administración e interno)
`Shapi.Contratos`	Compartido	Formato de las llaves de Redis, códigos de error y objetos de la caché
`Shapi.Compuerta`	Plano de datos	YARP y los filtros; depende solo de `Shapi.Contratos` y de StackExchange.Redis
`Shapi.Trabajador`	Procesos en segundo plano	Consolidación, cierre de ciclos, correo, dominios, resincronización y siembra de demostración

Los dos frontends viven en un *workspace* de pnpm (`frontend/apps/panel`, `frontend/apps/portal` y el paquete compartido `frontend/packages/ui`). Los tipos de TypeScript de la API se generan a partir de los contratos OpenAPI de `contratos/openapi/`.

6.5 Tubería de filtros de la compuerta

Cada petición a una API publicada recorre estos filtros en orden (ADR-20). Cada filtro deja pasar la petición o la rechaza con un error JSON; si la rechaza, la petición nunca llega al origen:

Orden	Filtro	Rechazo
0	CORS: responde el *preflight* de la consola de pruebas sin pedir clave	—
1	Resolver la API por el host	404 `api_no_encontrada`
2	Clave: `X-Api-Key` → hash SHA-256 → Redis	401 `clave_ausente`, `clave_invalida` o `clave_en_url`
3	Estado de la organización	403 `api_no_disponible`
4	Suscripción activa o en gracia	403 `suscripcion_inactiva`
5	Ruta expuesta (método y patrón)	403 `ruta_no_permitida`
6	Límites por minuto y cuotas, en un solo script Lua atómico	429 `limite_por_minuto`, `cuota_agotada` o `cuota_plataforma_agotada`
7	Caché de respuestas GET	— (responde desde la caché)
8	Reenvío al origen con YARP: quita `X-Api-Key` y agrega `X-Shapi-Consumidor`, `X-Shapi-Entorno` y `X-Shapi-Secreto`	502 `origen_inaccesible` (no conecta en 10 s) o 504 `origen_sin_respuesta` (más de 30 s)
9	Medición, después de responder: métricas del día e histogramas de latencia en Redis	—

Si Redis no está disponible, cualquier filtro responde 503 `servicio_no_disponible`. El diagrama de flujo completo está en el capítulo 10.

6.6 Hosts y enrutamiento

El dominio base se configura con `SHAPI_DOMINIO_BASE` y vale `shapi.localhost` en todos los ambientes, porque se usan dominios simulados (ADR-09).

Host	Destino	Contenido
`shapi.localhost`	Frontend `panel` + `/api/*` → API de control	Sitio público, panel del proveedor (`/panel/*`) y administración (`/admin/*`)
`{sub}.shapi.localhost`	Frontend `portal` + `/api/portal/*` → API de control	Portal de marca blanca de la API con ese subdominio
`{sub}.api.shapi.localhost`	Compuerta	Tráfico de la API
Dominio propio verificado (por ejemplo, `api.enviosxelaju.localhost`)	Compuerta	Tráfico de la API, con certificado emitido bajo demanda
`correo.shapi.localhost`	Mailpit	Bandeja de correo simulado (solo en desarrollo y demostración)

Los navegadores actuales resuelven cualquier `*.localhost` a `127.0.0.1` sin configurar nada (RFC 6761). Caddy usa su autoridad certificadora interna para emitir los comodines `*.shapi.localhost` y `*.api.shapi.localhost` y los certificados de los dominios propios; cada equipo instala una vez la raíz de esa autoridad (capítulo 12).

6.7 Despliegue

Ambiente	Cómo se levanta	Diferencias

Ambiente	Cómo se levanta	Diferencias
Desarrollo	Docker Compose con la infraestructura; los proyectos .NET y Vite corren en el equipo con recarga en caliente	Modo demostración, registros detallados y siembra de demostración
Productivo simulado (el que se presenta)	Docker Compose con `compose.yml` y `compose.prod.yml`, con las imágenes que publica la integración continua en GHCR	Compilación *Release*, frontends minificados, variables desde `.env`, volúmenes persistentes, reinicio automático, verificaciones de salud y sin puertos internos expuestos
Desplegado en Azure (el de la presentación final; el productivo simulado queda de respaldo)	Los mismos archivos más `infra/compose.servidor.yml`, en una máquina virtual de Azure con el dominio `shapi.tech`	Lo mismo que el productivo simulado, más certificados de Let's Encrypt, Mailpit con contraseña y reenvío a Resend de los correos con destinatario real
Pruebas (CI)	GitHub Actions, con Testcontainers para PostgreSQL y Redis	Se ejecuta en cada *pull request* y en cada integración a `main`

El "ambiente productivo" que piden los lineamientos se cumple con el mismo stack, contenedores versionados, configuración externa, TLS, persistencia y verificaciones de salud. Como el presupuesto es cero, corre en un equipo del equipo de trabajo. Para la presentación final corre además en una máquina virtual de Azure for Students, en `https://shapi.tech`, pagada con el crédito de estudiante (12.11).

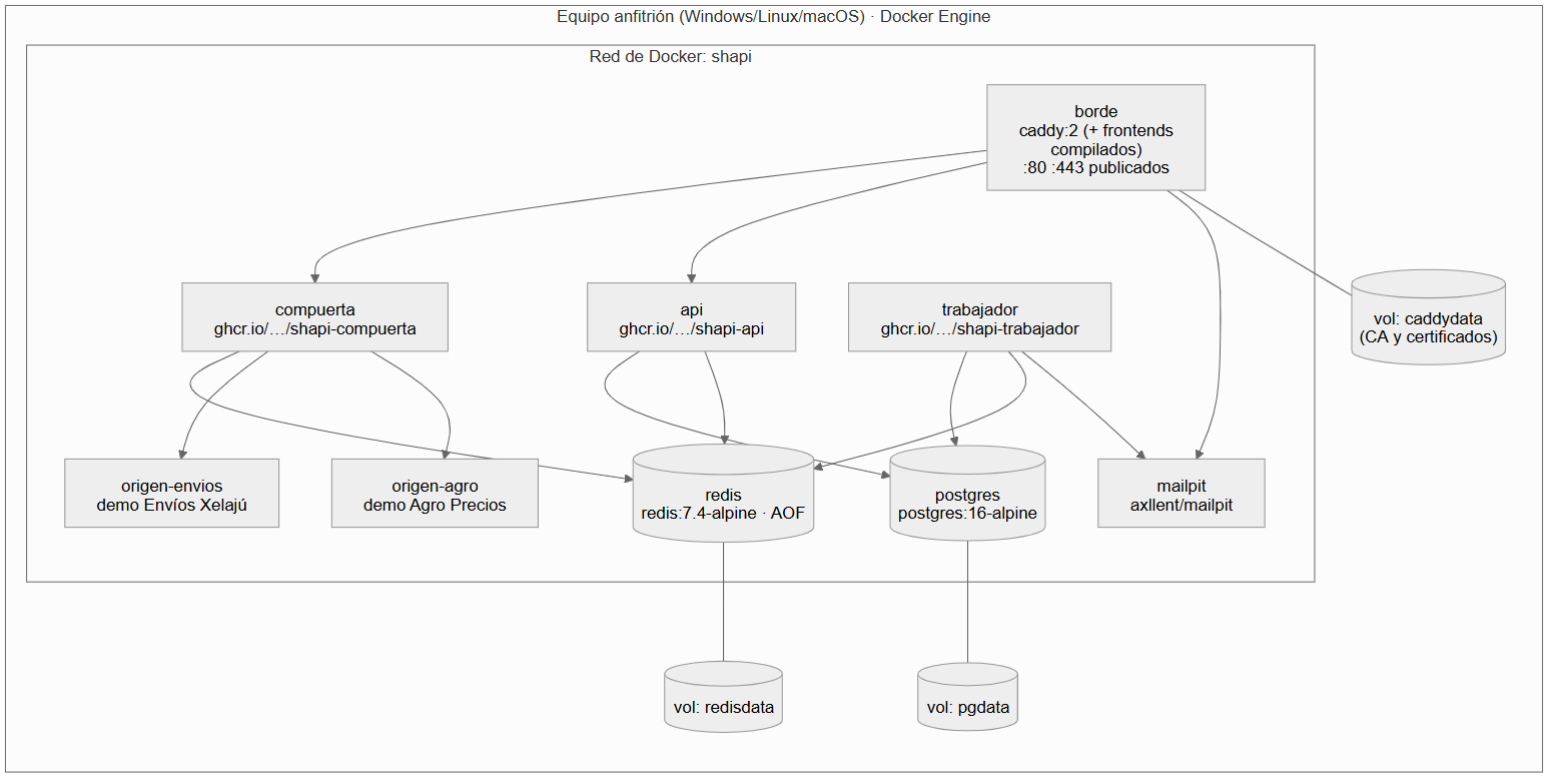


Diagrama 3. Despliegue.

Los orígenes de demostración (`origen-envios` y `origen-agro`) son dos APIs mínimas con su especificación OpenAPI y los datos de los mockups. Sirven para demostrar el sistema de punta a punta (ADR-33).

6.8 Observabilidad y disponibilidad

- La API de control y la compuerta exponen `/salud`. La compuerta y el trabajador escriben cada 10 segundos un **latido** en Redis, con un vencimiento de 30 segundos.
- La API de control calcula cada 30 segundos el **estado de los componentes** (pantalla B3.1): compuerta, API de control, trabajador, Redis, PostgreSQL, pasarela simulada y envío de correo, cada uno `en servicio`, `degradado` o `fuera de servicio`.
- Los tres procesos .NET escriben registros estructurados en JSON en la salida estándar, que se consultan con `docker compose logs` (capítulo 12).
- **Disponibilidad (RNF-06).** La disponibilidad mensual objetivo del enrutamiento es del 99.5 % en un despliegue real. Es un objetivo de diseño: el ambiente productivo simulado no corre de forma continua, así que no se mide. Lo respaldan tres mecanismos: el reinicio automático y las verificaciones de salud de los contenedores; que la compuerta siga enrutando aunque fallen la API de control, el trabajador o PostgreSQL (RNF-04), y el estado de los componentes de B3.1.

6.9 Seguridad

- Las contraseñas se guardan con PBKDF2 (el hasher de ASP.NET Core Identity). Las claves de API, los tokens de correo y los identificadores de sesión se guardan con hash SHA-256; las claves se muestran completas una sola vez (RNF-07).
- Las sesiones son del lado del servidor, con cookie `HttpOnly`, `Secure` y `SameSite=Lax`; la cuenta se bloquea 15 minutos tras 5 intentos fallidos (RF-04).
- El secreto de origen se guarda cifrado con ASP.NET Data Protection. De las tarjetas solo se guarda el token de la pasarela, la marca, los últimos 4 dígitos y el vencimiento.
- Un filtro global de EF Core impide que una consulta devuelva datos de otra organización (RNF-08).
- Las URL de origen no pueden apuntar a direcciones internas; se valida al guardarlas y al abrir cada conexión, como protección contra SSRF (RNF-10).
- Todo el tráfico va por HTTPS y HTTP redirige a HTTPS (RNF-09). Las acciones sensibles quedan en una bitácora que no admite cambios ni borrados (RF-41).

6.10 Decisiones de arquitectura (ADR)

Cada decisión importante quedó registrada como un ADR en `docs/specs/12-decisiones.md`, con su contexto, las alternativas descartadas y sus consecuencias. Esta es la lista completa:

ADR	Decisión
ADR-01	Las claves de API se guardan con hash y se muestran una sola vez
ADR-02	El consumidor rota; el consumidor y el proveedor revocan; el soporte no toca claves
ADR-03	El consumidor es una identidad separada, aislada por organización
ADR-04	El consumidor también verifica su correo y recupera su contraseña
ADR-05	Matriz de permisos explícita y una sola organización por usuario
ADR-06	Sesiones del lado del servidor con cookie, PBKDF2 y sin verificación en dos pasos
ADR-07	El subdominio, el portal y la marca son de cada API
ADR-08	Hosts distintos para el portal y la API
ADR-09	Caddy en el borde y dominios simulados bajo `*.localhost` (plan B, costo cero)

ADR	Decisión
ADR-10	El dominio propio es solo para la API y se verifica con DNS simulado
ADR-11	Protección contra SSRF y secreto de origen
ADR-12	Pasarela de pagos simulada detrás de una interfaz
ADR-13	Se guarda el token de la tarjeta, no la tarjeta
ADR-14	Historia de negocio y cobro por cuenta del proveedor
ADR-15	Una sola máquina de estados para las suscripciones, con 7 días de gracia
ADR-16	El plan Prueba vence
ADR-17	Reglas para cambiar de plan
ADR-18	Petición y llamada son unidades distintas
ADR-19	Datos concretos de los planes y una suscripción por API
ADR-20	Orden definitivo de la tubería de filtros
ADR-21	Reserva atómica de la cuota con Lua en Redis
ADR-22	La compuerta sin datos en memoria, Redis persistente y resincronización
ADR-23	Histogramas de latencia, errores por código y bytes
ADR-24	Qué hace la clave de pruebas
ADR-25	Cabeceras de la compuerta
ADR-26	OpenAPI y caché de respuestas
ADR-27	Procesos y estilo arquitectónico
ADR-28	Modelo de datos completo
ADR-29	Mockups corregidos y pantallas nuevas
ADR-30	Ciclos exactos de 30 días y cómo se muestran las fechas
ADR-31	Las especificaciones viven en el repositorio y se verifican automáticamente
ADR-32	La variante 4 es el sistema visual oficial
ADR-33	Orígenes de demostración y siembra igual a los mockups
ADR-34	La categoría ya está aprobada
ADR-35	Versiones del stack

7. Definición del modelo

Este capítulo define el **modelo de proceso de desarrollo de software** que siguió el equipo. Los modelos del sistema (casos de uso, clases, estados, entidad-relación y relacional) están en el capítulo 10.

7.1 Modelo elegido

Shapi se construyó con un **modelo incremental** combinado con **desarrollo dirigido por especificaciones** (*spec-driven development*), ejecutado por agentes de inteligencia artificial bajo la supervisión del equipo y con prácticas ágiles de integración continua.

- **Incremental:** el sistema se entregó en incrementos funcionales que coinciden con los avances del curso (30 %, 50 %, 80 % y 100 %). Cada incremento se integra, se prueba y se demuestra completo, y el siguiente se construye sobre él.
- **Dirigido por especificaciones:** antes de escribir código se escribió **qué** construir. Las especificaciones de `docs/specs/` (requisitos, casos de uso, arquitectura, modelo de datos, interfaz y decisiones) y los mockups aprobados son la fuente de verdad: si el código las contradice, el código está mal.
- **Ejecutado por agentes:** cada integrante le pide a su agente de IA (Claude Code, Codex u otro) que implemente sus tareas. El agente sigue un protocolo escrito (`docs/plan/protocolo.md`) y la persona responde solo cuando hace falta una decisión, una credencial o un acceso.

7.2 Justificación

Característica del proyecto	Por qué encaja el modelo
Fechas de entrega fijas por el curso	Los incrementos coinciden con los avances y cada uno se puede demostrar
Equipo de 4 personas con unas 3 horas por semana cada una	Los agentes hacen el trabajo mecánico; las personas revisan y deciden
Requisitos estables después de la fase de análisis	Las especificaciones se escribieron y auditaron una vez, y luego solo se precisaron
Varias personas y agentes trabajando en paralelo	Las tareas son pequeñas, tienen dueño, dependencias explícitas y archivos asignados, lo que evita que se pisen
Necesidad de calidad verificable	Cada tarea lleva pruebas de sus criterios de aceptación, y la integración continua bloquea lo que no pasa

7.3 Fases

Fase	Fechas	Qué se hizo
0 · Análisis y diseño	24 ago – 23 sep	Propuesta, requisitos y diseño preliminar, mockups, especificaciones, auditoría de las especificaciones y plan de tareas
Avance 1 (30 %)	23 – 25 sep	Andamiaje, CI, infraestructura, base de datos, registro y acceso, compuerta básica
Avance 2 (50 %)	26 sep – 9 oct	Publicación de APIs, planes, contratación, claves, portal, compuerta completa y siembra
Avance 3 (80 %)	10 – 23 oct	Consumo y métricas, ciclos y renovación, administración y soporte, dominios propios y pruebas E2E. El curso movió su entrega al 31 de octubre, como video
Semana final	23 – 29 oct	Correcciones de la convergencia, documentos de requisitos y de diseño en PDF, manuales y congelamiento del código (JG-17)
Video y entrega final	30 oct – 6 nov	Grabación del video del Avance 3 (31 oct) y entrega final con exposición (semana del 2 al 6 de noviembre)

7.4 Artefactos del proceso

Artefacto	Dónde está	Para qué sirve
Especificaciones	`docs/specs/`	Fuente de verdad del sistema
Mockups	`mockups/`	Pantallas aprobadas; la interfaz debe coincidir en textos, datos, orden y estados
Contratos HTTP	`contratos/openapi/`	Un contrato OpenAPI por módulo; de ellos se generan los tipos del frontend
Tareas	`docs/plan/tareas/`	Un archivo por tarea con responsable, objetivo, contexto, archivos, criterios de aceptación, pruebas, verificación, dependencias y fecha programada
Calendario	`docs/plan/calendario.md`	Hitos, ventanas de cada avance y la tarea de cada persona por día
Bitácoras	`docs/plan/bitacora/`	Un diario por persona con lo hecho, las decisiones y los avisos para los demás
Protocolo y convenciones	`docs/plan/protocolo.md` y `convenciones.md`	El procedimiento obligatorio y las reglas técnicas y de propiedad de archivos

7.5 Ciclo de vida de una tarea

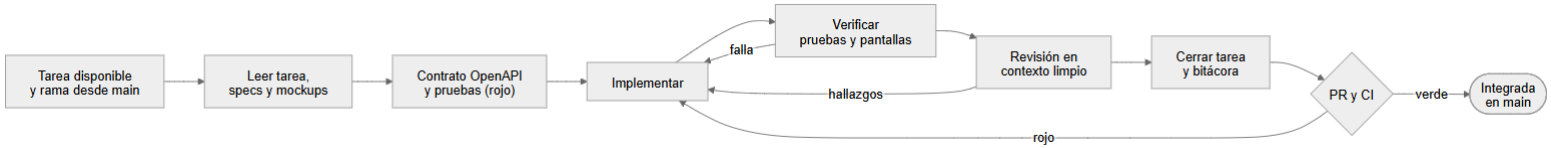


Diagrama 4. Ciclo de vida de una tarea.

7.6 Control de calidad

- **Pruebas primero.** Cada criterio de aceptación tiene al menos una prueba automatizada, nombrada con el código del requisito que cubre.
- **Integración continua.** Ningún cambio llega a `main` sin pasar los checks obligatorios: `plan` (las tareas y el calendario son válidos), `titulo` (el PR pertenece a una tarea y la cierra bien), `backend` (compilación, formato y pruebas .NET con Testcontainers), `frontend` (lint, tipos, pruebas y compilación), `revision-claude` (revisión automática del PR contra la tarea y las especificaciones) y `ambiente-productivo` (pruebas E2E con Playwright contra el ambiente productivo simulado).
- **Revisión en contexto limpio.** Antes de abrir el PR, un agente que no escribió el código lo revisa contra la tarea y las especificaciones.
- **Auditorías del coordinador.** Se auditaron las tareas integradas el 25 de septiembre y el 3 de octubre, y cada hallazgo se corrigió con una prueba que lo habría detectado.
- **Convergencias.** Antes de cada avance (8 y 22 de octubre) y al terminar (29 de octubre), el coordinador compara lo construido con las especificaciones, ensaya la demostración y crea tareas nuevas para las brechas.
- **Definición de terminado.** Una tarea está terminada cuando sus criterios tienen pruebas que pasan, la CI está en verde, sus pantallas coinciden con el mockup, su contrato está actualizado, la revisión no deja hallazgos de corrección y el PR quedó integrado en `main`.

7.7 Roles del equipo

Integrante	Rol	Área principal
Héctor Jordin Adolfo García Coyoy	Coordinador	Andamiaje y CI, compuerta de tráfico, claves, medición del consumo, convergencias, auditorías y documentos en PDF
Ronaldo Emilio Méndez Mayorga	Desarrollador	Base de datos, identidad y sesiones, organizaciones y miembros, planes, suscripciones, pasarela simulada, pagos y renovaciones
Dominique Guillermo Contreras Sierra	Desarrollador	Sistema de diseño, estructura del panel y del portal, publicación de APIs, portal de marca blanca y cuenta del consumidor
José Pablo Zúñiga de León	Desarrollador	Infraestructura, orígenes de demostración, siembra, correo, bitácora, administración y soporte, pruebas E2E y de carga, y manuales

7.8 Resultado del proceso

Al 1 de noviembre de 2026, el plan tiene 92 tareas: 90 integradas en `main`, contando este manual (JZ-15), y 2 pendientes de la entrega final: el manual de usuario (JZ-16) y la convergencia final con el congelamiento del código (JG-17). Se integraron más de 130 *pull requests*, todos con la CI en verde. El detalle por tarea, con su fecha pronosticada y la real, está en el capítulo 8.

8. Cronograma

Este capítulo compara lo que se planificó con lo que ocurrió. Las fechas salen del historial del repositorio, así que se pueden comprobar:

- **Pronosticado:** la primera fecha programada que tuvo cada tarea (el campo `programada` de su archivo, que existe desde el 28 de septiembre). Las 14 tareas que se terminaron antes de esa fecha toman el cierre de su avance en el calendario original (24 de septiembre para el Avance 1 y 8 de octubre para el Avance 2), o el día en que se creó la tarea, si fue posterior.
- **Real:** el día en que la tarea quedó marcada como hecha en `main`, es decir, cuando se integró su *pull request*, en la hora de Guatemala.
- **Diferencia:** días entre las dos fechas. Un número negativo indica que la tarea se adelantó; uno positivo, que se atrasó.

8.1 Hitos del curso

En esta tabla, el pronosticado es la fecha que fijaba el calendario del curso para cada hito; la fecha programada de cada tarea está en 8.4.

Hito	Pronosticado	Real	Observación
Propuesta, alcance y cronograma	vie 28 ago	vie 28 ago	Entregada
Requisitos y diseño preliminar	vie 11 sep	vie 11 sep	Entregado
Avance funcional 1 (30 %)	vie 25 sep	vie 25 sep	Demostración en vivo
Convergencia del Avance 2 (JG-08)	jue 8 oct	jue 8 oct	Informe de brechas y tareas nuevas
Avance funcional 2 (50 %)	vie 9 oct	vie 9 oct	Demostración en vivo, solo por la interfaz
Convergencia del Avance 3 (JG-14)	jue 22 oct	vie 23 oct	Informe de brechas y tareas nuevas
Avance funcional 3 (80 %)	vie 23 oct	sáb 31 oct	El curso lo movió al 31 de octubre y lo convirtió en un video de 25 minutos
Documentos de requisitos y de diseño en PDF (JG-16)	lun 2 nov	dom 25 oct	Se adelantó. La tarea estaba programada para el 24 de octubre (8.4.4)
Manual técnico (JZ-15)	lun 2 nov	dom 1 nov	Este documento. La tarea JZ-15 estaba programada para el 26 de octubre (8.4.4)
Convergencia final y congelamiento del código (JG-17)	jue 29 oct	pendiente	Se cierra con los dos manuales
Entrega final y exposición	semana del 2 al 6 nov	pendiente	La fecha exacta la confirma el curso

8.2 Diagrama de Gantt

El diagrama resume el proyecto completo. La fase 0 (análisis y diseño) se muestra por semanas, con sus fechas reales: sus dos entregas, la propuesta y los requisitos, se cumplieron en la fecha del curso. Desde el 23 de septiembre, cada barra va de la primera a la última tarea de una persona en ese avance. Las barras oscuras son lo pronosticado y las claras, lo real; los rombos son las entregas del curso.

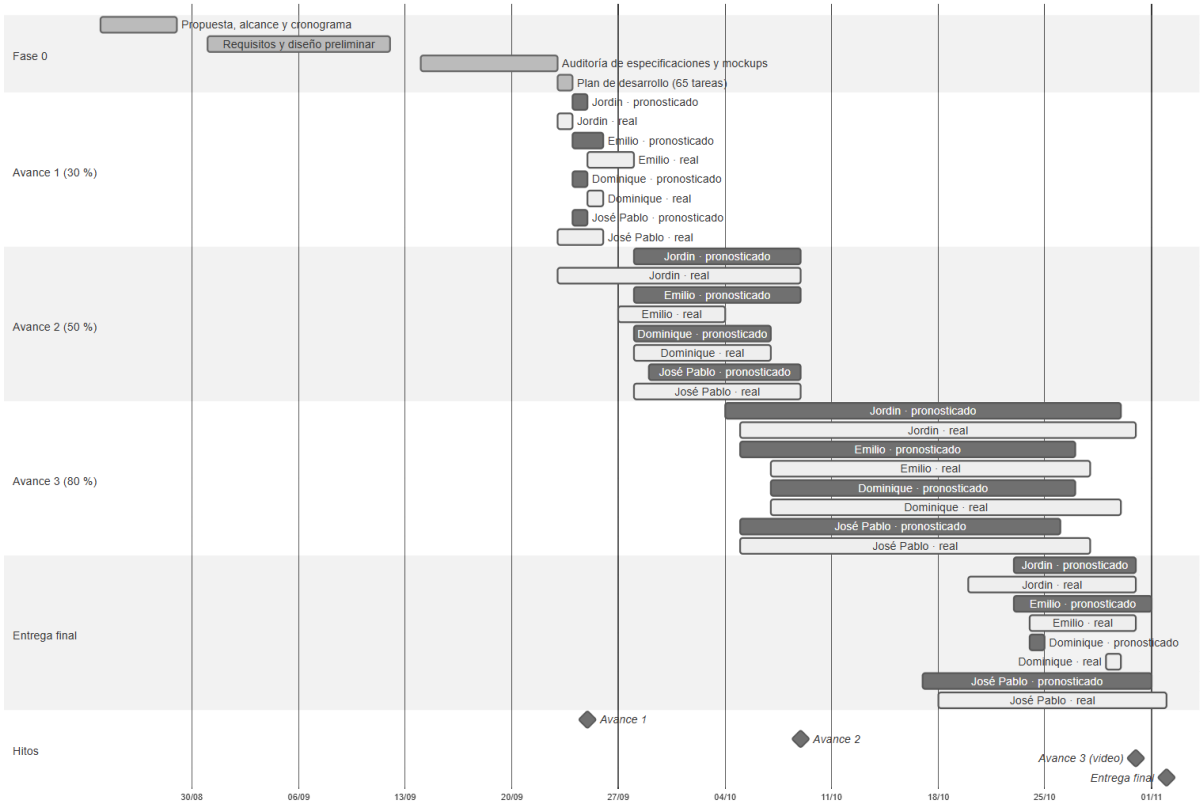


Diagrama 5. Diagrama de Gantt.

8.3 Cumplimiento por avance

Avance	Tareas	Integradas	A tiempo o antes	Con atraso	Diferencia promedio	Atraso máximo
Avance 1 (30 %)	11	11	4	7	+0.5 días	2 días
Avance 2 (50 %)	24	24	18	6	−1.2 días	1 día
Avance 3 (80 %)	36	36	19	17	+0.6 días	3 días
Entrega final	21	19	5	14	+1.2 días	6 días
Total	**92**	**90**	**46**	**44**		

Por qué cambió el plan:

- **El plan creció de 65 a 92 tareas.** Las auditorías del coordinador (25 de septiembre y 3 de octubre) y las convergencias (8 y 22 de octubre) encontraron brechas entre lo construido y las especificaciones, y cada brecha se convirtió en una tarea nueva con su fecha.
- **El Avance 3 se movió del 23 al 31 de octubre** y pasó de demostración en vivo a video. El equipo decidió grabar el proyecto terminado, así que todas las tareas, incluidos los manuales, se reprogramaron para terminar el 29 de octubre.
- **Los atrasos fueron cortos.** El mayor fue el de este manual (JZ-15), de 6 días; entre las tareas integradas antes, fue de 5 días (DC-16, la revisión visual contra los mockups, que generó correcciones para tres personas). Varias tareas se adelantaron a un avance anterior para repartir la carga: JG-09, EM-09, EM-12, DC-09, JZ-10 y JZ-11 al Avance 2, y JZ-14 al Avance 3.
- **Pendientes al cierre de este manual:** el manual de usuario (JZ-16) y la convergencia final con el congelamiento del código (JG-17).

8.4 Detalle por tarea

Las fechas son de 2026 y la diferencia está en días.

8.4.1 Avance 1 (30 %)

ID	Tarea	Responsable	Pronosticado	Real	Diferencia
JG-01	Andamiaje del backend, CI y reglas del repositorio	Jordin	jue 24 sep	mié 23 sep	−1
JG-02	Compuerta mínima: host → API, clave y reenvío	Jordin	jue 24 sep	mié 23 sep	−1
JZ-01	Infraestructura local: Docker Compose, Caddy, TLS y Mailpit	José Pablo	jue 24 sep	mié 23 sep	−1
JZ-02	Orígenes de demostración (Envíos Xelajú y Agro Precios)	José Pablo	jue 24 sep	mié 23 sep	−1
DC-01	Workspace del frontend y sistema de diseño	Dominique	jue 24 sep	vie 25 sep	+1
DC-02	Estructura del panel y del sitio público	Dominique	jue 24 sep	vie 25 sep	+1
EM-01	Esquema completo de la base de datos y datos base	Emilio	jue 24 sep	vie 25 sep	+1
EM-02	Registro, verificación de correo e inicio de sesión del personal (backend)	Emilio	jue 24 sep	vie 25 sep	+1
JZ-03	Envío de correos desde la bandeja de salida	José Pablo	jue 24 sep	vie 25 sep	+1
EM-03	Pantallas de registro, verificación y acceso (A1.1 a A1.3)	Emilio	jue 24 sep	sáb 26 sep	+2
EM-17	Publicar el contrato OpenAPI de identidad	Emilio	vie 25 sep	dom 27 sep	+2

8.4.2 Avance 2 (50 %)

ID	Tarea	Responsable	Pronosticado	Real	Diferencia
DC-03	Estructura del portal de marca blanca	Dominique	lun 28 sep	lun 28 sep	a tiempo
JG-04	Publicador de configuración en Redis y resincronización	Jordin	lun 28 sep	lun 28 sep	a tiempo
EM-04	Recuperación de contraseña y Mi perfil (A1.4a, A1.4b y A8.1)	Emilio	lun 28 sep	mar 29 sep	+1
DC-04	Registrar una API y lista de APIs (A3.1 y A3.2)	Dominique	mar 29 sep	mar 29 sep	a tiempo
JG-07	Servicio de claves: emisión, rotación y revocación (backend)	Jordin	mar 29 sep	mar 29 sep	a tiempo
JZ-06	Imágenes Docker, ambiente productivo simulado y publicación en GHCR	José Pablo	mar 29 sep	mar 29 sep	a tiempo
EM-05	Identidad del consumidor (backend del portal)	Emilio	mié 30 sep	mié 30 sep	a tiempo
JG-05	Compuerta: organización, suscripción, ruta, secreto, SSRF y CORS	Jordin	mié 30 sep	jue 1 oct	+1
DC-05	Especificación OpenAPI y rutas expuestas (A3.3 y A3.4)	Dominique	jue 1 oct	jue 1 oct	a tiempo
JZ-07	Pruebas de extremo a extremo y herramienta de capturas	José Pablo	jue 1 oct	jue 1 oct	a tiempo
EM-07	Planes de API (A4.1)	Emilio	jue 1 oct	vie 2 oct	+1
JG-06	Compuerta: límites por minuto, cuotas y cabeceras (Lua)	Jordin	vie 2 oct	vie 2 oct	a tiempo

ID	Tarea	Responsable	Pronosticado	Real	Diferencia
DC-08	Pantallas de acceso del consumidor (A5.3, A5.3b y A5.7 a A5.10)	Dominique	vie 2 oct	sáb 3 oct	+1
EM-18	Publicar el destino de la sesión del consumidor	Emilio	vie 2 oct	sáb 3 oct	+1
EM-08	Contratación de un plan de API (backend)	Emilio	sáb 3 oct	sáb 3 oct	a tiempo
JG-18	Optimización de las pruebas y la CI	Jordin	sáb 3 oct	sáb 3 oct	a tiempo
JZ-05	Siembra de demostración	José Pablo	sáb 3 oct	sáb 3 oct	a tiempo
DC-06	Configuración por ruta y publicación (A3.5)	Dominique	dom 4 oct	lun 5 oct	+1
DC-07	Portal público: inicio y documentación (A5.0, A5.1 y A5.5)	Dominique	mar 6 oct	mar 6 oct	a tiempo
JG-03	Revisión automática con Claude y tablero del plan	Jordin	jue 8 oct	mié 23 sep	−15
EM-06	Pasarela de pagos simulada	Emilio	jue 8 oct	dom 27 sep	−11
JZ-04	Bitácora de acciones sensibles (B3.2)	José Pablo	jue 8 oct	lun 28 sep	−10
JG-08	Convergencia del Avance 2	Jordin	jue 8 oct	jue 8 oct	a tiempo
JZ-17	Siembra de demostración en el ambiente productivo simulado	José Pablo	jue 8 oct	jue 8 oct	a tiempo

8.4.3 Avance 3 (80 %)

ID	Tarea	Responsable	Pronosticado	Real	Diferencia
JG-09	Medición en la compuerta y consolidación del consumo	Jordin	dom 4 oct	lun 5 oct	+1
JZ-11	Plantillas de correo completas	José Pablo	lun 5 oct	lun 5 oct	a tiempo
EM-09	Suscripción de plataforma: contratar y cambiar de plan (A2 y B1.4)	Emilio	lun 5 oct	mié 7 oct	+2
DC-09	Planes y contratación en el portal (A5.4, A5.6 y A5.4b)	Dominique	mié 7 oct	mié 7 oct	a tiempo
JZ-10	Casos de soporte (A6.4, A6.4b, A7.1 y A7.2)	José Pablo	mié 7 oct	mié 7 oct	a tiempo
EM-12	Miembros e invitaciones (A4.2 y A8.2)	Emilio	mié 7 oct	jue 8 oct	+1
JZ-08	Administración de organizaciones (A6.2 y A6.2b)	José Pablo	sáb 10 oct	vie 9 oct	−1
JG-11	Consumo, latencia y errores por API (B1.1)	Jordin	sáb 10 oct	dom 11 oct	+1
DC-11	Suscripción y claves del consumidor (B2.3 a B2.6)	Dominique	sáb 10 oct	lun 12 oct	+2
EM-10	Cierre de ciclo: renovación, gracia, suspensión y fin de la Prueba	Emilio	sáb 10 oct	lun 12 oct	+2
JZ-18	Levantar el ambiente con la siembra en un solo comando (RNF-14)	José Pablo	dom 11 oct	vie 9 oct	−2
JG-10	Pantalla de claves del proveedor (A4.3 y A4.3b)	Jordin	lun 12 oct	lun 12 oct	a tiempo
JZ-09	Cuentas de administración y soporte (A6.5)	José Pablo	lun 12 oct	lun 12 oct	a tiempo
DC-10	Consola de pruebas del portal (A5.2)	Dominique	lun 12 oct	mar 13 oct	+1
EM-11	Historial de pagos (B1.3 y la API de B2.2)	Emilio	lun 12 oct	mar 13 oct	+1
DC-17	Barra lateral del portal en planes y pago, y texto de A5.3	Dominique	mar 13 oct	mar 13 oct	a tiempo
EM-19	Pruebas de backend del acceso del consumidor	Emilio	mar 13 oct	mar 13 oct	a tiempo
JZ-12	Estado de los componentes (B3.1)	José Pablo	mié 14 oct	mar 13 oct	−1

ID	Tarea	Responsable	Pronosticado	Real	Diferencia
DC-14	Dominio propio, DNS simulado y secreto de origen (A3.6)	Dominique	mié 14 oct	mié 14 oct	a tiempo
JG-12	Consumo del ciclo para el consumidor (B2.1)	Jordin	mié 14 oct	mié 14 oct	a tiempo
EM-13	Límites del plan de plataforma y cambio de plan del consumidor	Emilio	mié 14 oct	vie 16 oct	+2
JG-20	Ajustes de interfaz del ensayo del Avance 2 (tarjeta, barra lateral, cursor y contraseña)	Jordin	jue 15 oct	jue 15 oct	a tiempo
DC-13	Pagos y cambio de plan del consumidor (B2.2 y B2.7)	Dominique	vie 16 oct	vie 16 oct	a tiempo
EM-14	Administración: planes de plataforma y pagos (A6.1 y A6.3)	Emilio	vie 16 oct	vie 16 oct	a tiempo
JG-13	Consumo y facturación por consumidor (B1.2)	Jordin	vie 16 oct	lun 19 oct	+3
EM-20	Medio de pago en la confirmación de la contratación (A5.4b)	Emilio	sáb 17 oct	dom 18 oct	+1
DC-15	Sitio público: inicio de Shapi (A0.1)	Dominique	dom 18 oct	jue 15 oct	−3
EM-15	Invitar consumidores (B1.5)	Emilio	dom 18 oct	dom 18 oct	a tiempo
JG-19	Prueba de la compuerta sin PostgreSQL, API de control ni trabajador (RNF-02 y RNF-04)	Jordin	dom 18 oct	mar 20 oct	+2
EM-21	Disponibilidad del proveedor en la cuenta del consumidor	Emilio	lun 19 oct	lun 19 oct	a tiempo
DC-12	Personalización del portal (A3.7)	Dominique	mar 20 oct	mar 20 oct	a tiempo
JG-14	Convergencia del Avance 3	Jordin	jue 22 oct	vie 23 oct	+1
JZ-19	Elemento de código para el video del Avance 3 (Clean Code y patrón de diseño)	José Pablo	dom 25 oct	mar 27 oct	+2
EM-22	Elemento de código para el video del Avance 3 (Clean Code y patrón de diseño)	Emilio	lun 26 oct	mar 27 oct	+1
DC-18	Elemento de código para el video del Avance 3 (Clean Code y patrón de diseño)	Dominique	lun 26 oct	jue 29 oct	+3
JG-29	Guion del video del Avance 3 y elemento de código de Jordin	Jordin	jue 29 oct	vie 30 oct	+1

8.4.4 Entrega final

ID	Tarea	Responsable	Pronosticado	Real	Diferencia
JZ-14	Pruebas de carga (RNF-01 y RNF-03)	José Pablo	sáb 17 oct	dom 18 oct	+1
JG-21	Timeout de Redis en el reloj y en los trabajos del trabajador	Jordin	vie 23 oct	dom 25 oct	+2
EM-23	Publicar la organización al registrar un proveedor	Emilio	vie 23 oct	lun 26 oct	+3
EM-16	Pruebas de aislamiento entre organizaciones y de permisos	Emilio	sáb 24 oct	sáb 24 oct	a tiempo
JG-16	Generador de PDF y documentos de requisitos y de diseño	Jordin	sáb 24 oct	dom 25 oct	+1
JG-22	Modo demostración en la interfaz (reloj, fechas y bitácora)	Jordin	sáb 24 oct	lun 26 oct	+2
JZ-13	Pruebas E2E de los flujos principales	José Pablo	sáb 24 oct	lun 26 oct	+2
DC-16	Revisión visual contra los mockups	Dominique	sáb 24 oct	jue 29 oct	+5

ID	Tarea	Responsable	Pronosticado	Real	Diferencia
JG-23	Correcciones de cobros y suscripciones (gracia por reversión, aviso de la Prueba, cambio programado y cierre de ciclo)	Jordin	dom 25 oct	lun 26 oct	+1
JG-24	Correcciones del portal y del dominio propio (A3.6, consola y marca)	Jordin	dom 25 oct	mar 27 oct	+2
JG-15	Caché de respuestas en la compuerta	Jordin	lun 26 oct	mar 20 oct	−6
JG-25	Correcciones de soporte, bitácora y miembros	Jordin	lun 26 oct	mié 28 oct	+2
JZ-15	Manual técnico	José Pablo	lun 26 oct	dom 1 nov	+6
JG-26	Correcciones de pantallas del panel y del portal (fechas, tablas, permisos y estilos)	Jordin	mar 27 oct	mié 28 oct	+1
JG-27	Correcciones de la siembra, el ambiente y la CI	Jordin	mar 27 oct	jue 29 oct	+2
JZ-16	Manual de usuario	José Pablo	mar 27 oct	—	pendiente
JG-28	Pendientes de la auditoría del 3 de octubre (identidad, planes, contratación y compuerta)	Jordin	mié 28 oct	jue 29 oct	+1
JG-17	Convergencia final y congelamiento del código	Jordin	jue 29 oct	—	pendiente
JG-30	Encabezado y pie del portal con la sesión abierta (A5.0 y A5.2)	Jordin	vie 30 oct	vie 30 oct	a tiempo
JZ-20	Diferencias visuales de la revisión DC-16 (A6.2, A6.5 y B3.1)	José Pablo	sáb 31 oct	jue 29 oct	−2
EM-24	Diferencias visuales de la revisión DC-16 (A2.1, A6.3 y B1.4)	Emilio	sáb 31 oct	vie 30 oct	−1

9. Diseño de interfaces

Las pantallas se diseñaron antes de programarlas y quedaron aprobadas como **mockups**. La interfaz construida debe coincidir con ellos en textos, datos, orden y estados; la revisión visual de DC-16 lo comprobó pantalla por pantalla. El detalle completo está en `docs/specs/11-interfaz.md`.

9.1 Sistema visual

El sistema visual oficial es la **variante 4, «Plano azul»** (ADR-32), elegida entre seis propuestas. Sus tokens se implementan como variables CSS y como tema de Tailwind en `frontend/packages/ui`, y los comparten el panel y el portal.

Elemento	Definición
Tipografía	**Sora** para títulos, cifras y precios; **IBM Plex Sans** para la interfaz y los datos, con cifras tabulares en las tablas
Color principal	Azul `#3B6FF0` en las superficies claras y `#7FA6FF` en las oscuras. En el portal se reemplaza por el color de la marca de cada API
Superficies	Fondo `#F4F6FA`, paneles blancos, tinta `#0B1220`. El sitio público y las barras del panel usan una superficie oscura (`#060910`)
Estados	Solo tres colores: correcto (activa, pagado, en servicio), alerta (suspendida, rechazado, en gracia, degradado) y neutro (despublicada, pendiente, cerrado)
Componentes	Radio único de 8 px; botones de 46 px; campos de 48 px; tarjetas con borde y sin sombra; tablas con filas de 42 px
Estructura del panel	Barra superior oscura de 76 px y barra lateral oscura de 272 px con los grupos Publicación, API y Organización. Se diseña para 1440 × 900 y funciona desde 1280 px (RNF-12)

Toda pantalla cubre además cinco estados que no tienen mockup propio: **cargando** (esqueleto con la forma del contenido), **error de red o del servidor** (aviso con el botón «Reintentar»), **error de validación** (mensaje debajo del campo), **sin permiso** («No tiene permiso para ver esta página») y **acción confirmada** (aviso breve de 4 segundos).

9.2 Formato de los mockups

Cada mockup es un archivo HTML estático de 1440 × 900 en `mockups/<tanda>/*.dc.html`. El archivo `canvas.json` de cada tanda tiene el orden y el título de sus pantallas, y los archivos `.html` de la raíz de `mockups/` son paquetes autocontenidos que se abren con doble clic en el navegador. Las pantallas usan los **mismos datos que la siembra de demostración** (capítulo 12), así que el sistema instalado se ve igual que los mockups.

Las tandas A0 a A8 son los flujos de cada área (sitio público, acceso, contratación, publicación, portal, administración, soporte y cuenta), y las tandas B1 a B3 reúnen las salidas: consumo, pagos, suscripciones, estado de los componentes y bitácora.

9.3 Pantallas principales

Esta sección muestra una o dos pantallas por tanda. Las imágenes se capturaron directamente de los mockups aprobados.

A0 · Sitio público

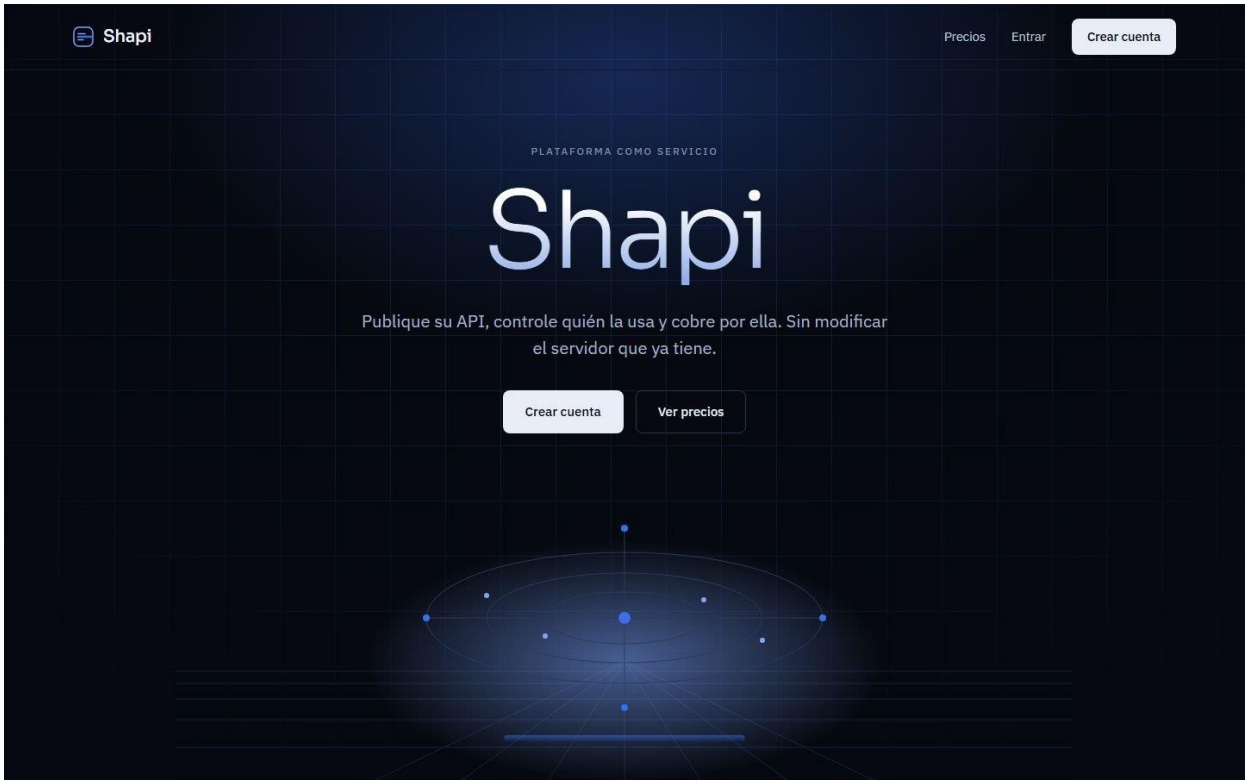


Figura 1. A0.1 · Inicio de Shapi con planes. El sitio público presenta Shapi, explica los tres pasos para publicar una API y muestra los planes de plataforma. Se muestra la primera pantalla de la página. Fuente: `mockups/A0/Main.dc.html`.

A1 · Acceso del proveedor

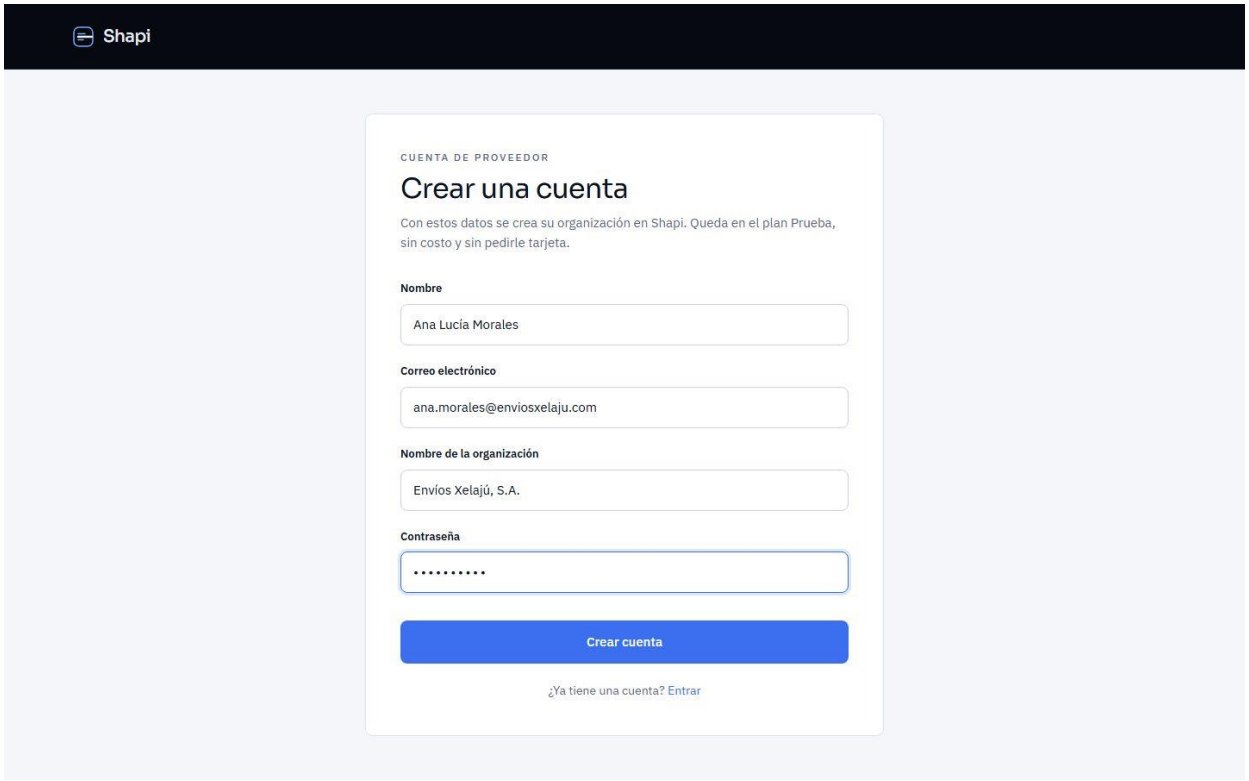


Figura 2. A1.1 · Registro del proveedor. El proveedor crea su cuenta y su organización; queda en el plan Prueba sin pedirle tarjeta. Fuente: `mockups/A1/Main.dc.html`.

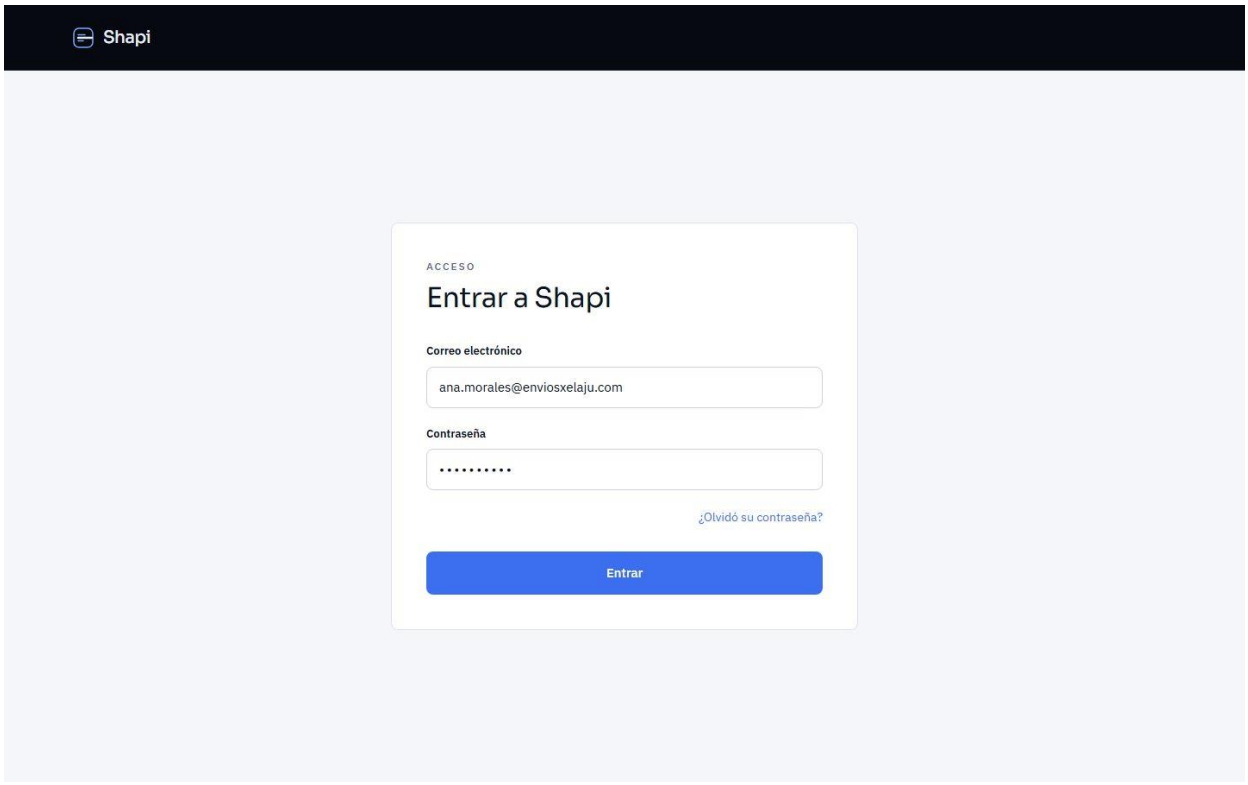


Figura 3. A1.3 · Inicio de sesión. El mismo formulario sirve al proveedor, al administrador y al soporte; el sistema los enruta según su rol. Fuente: `mockups/A1/Sesion.dc.html`.

A2 · Plan de plataforma

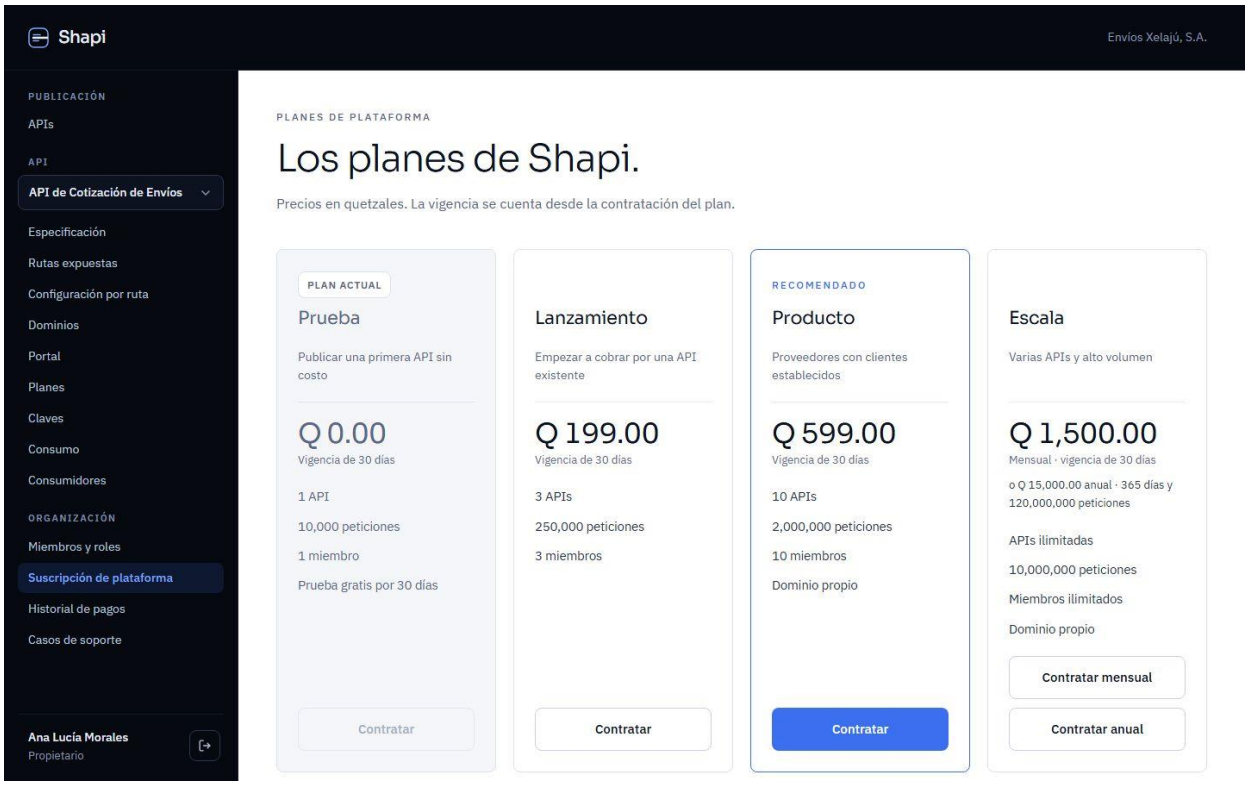


Figura 4. A2.1 · Planes de plataforma. Los planes de plataforma con sus límites de APIs, peticiones y miembros, y el plan vigente de la organización. Fuente: `mockups/A2/Main.dc.html`.

A3 · Publicación de una API

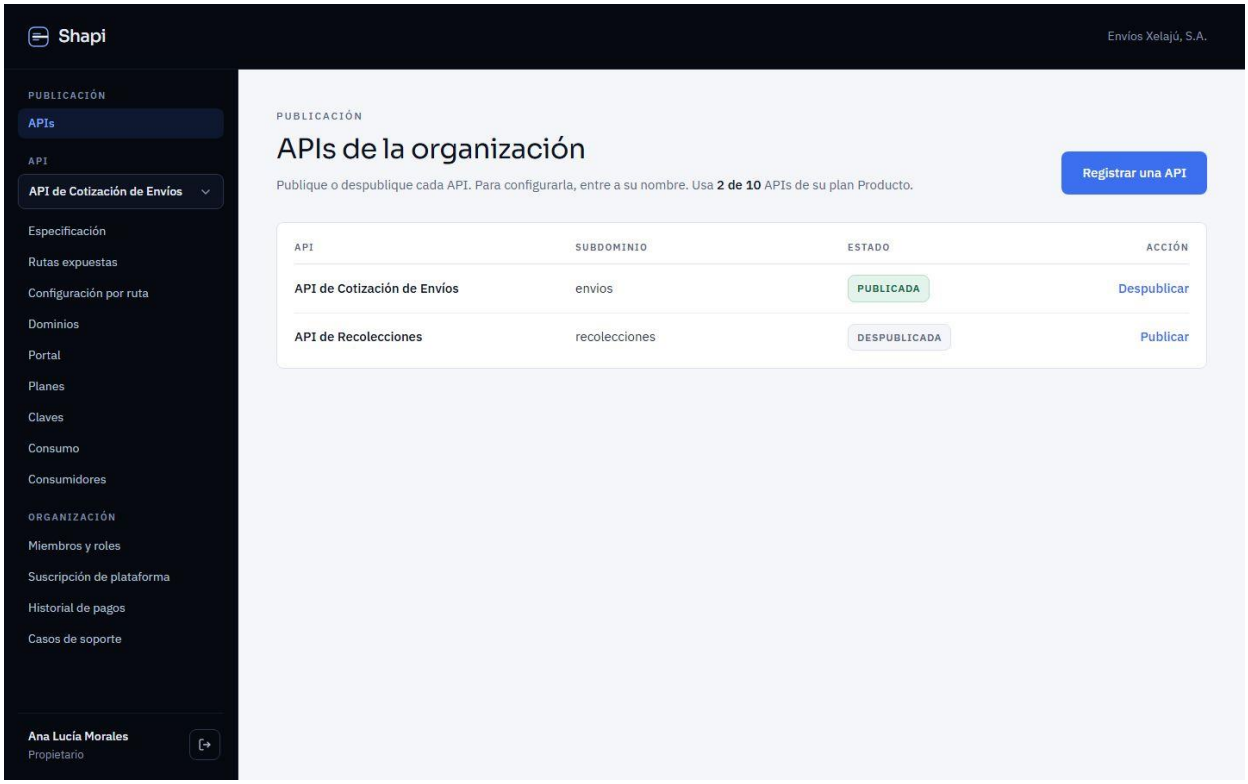


Figura 5. A3.1 · APIs de la organización. La lista de APIs de la organización, con su estado y el uso del límite del plan. Fuente: `mockups/A3/Main.dc.html`.



Figura 6. A3.5 · Configuración por ruta. El límite por minuto, la caché y el peso en llamadas de cada ruta expuesta. Fuente: `mockups/A3/ConfigRutas.dc.html`.

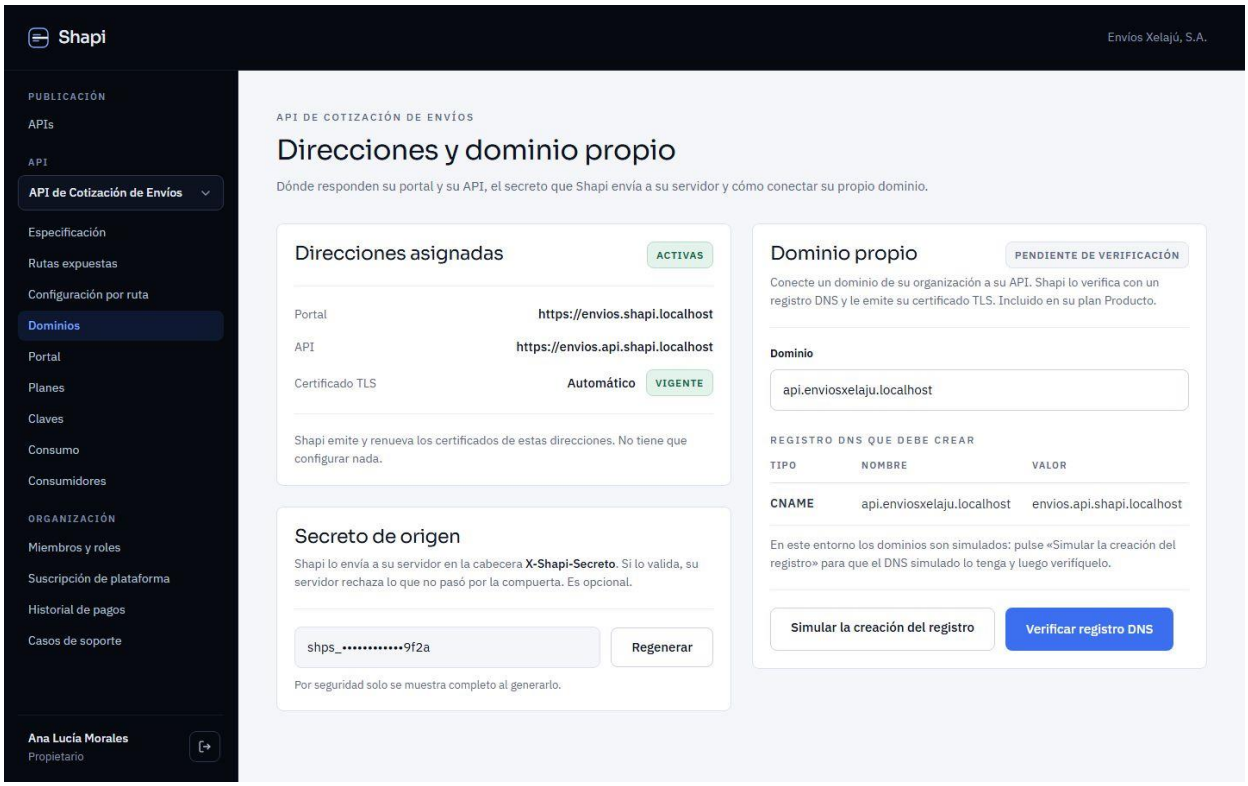


Figura 7. A3.6 · Direcciones, dominio propio y secreto de origen. Los hosts del portal y de la API, el dominio propio verificado con el DNS simulado y el secreto de origen. Fuente: `mockups/A3/Dominio.dc.html`.

A4 · Planes, claves y equipo

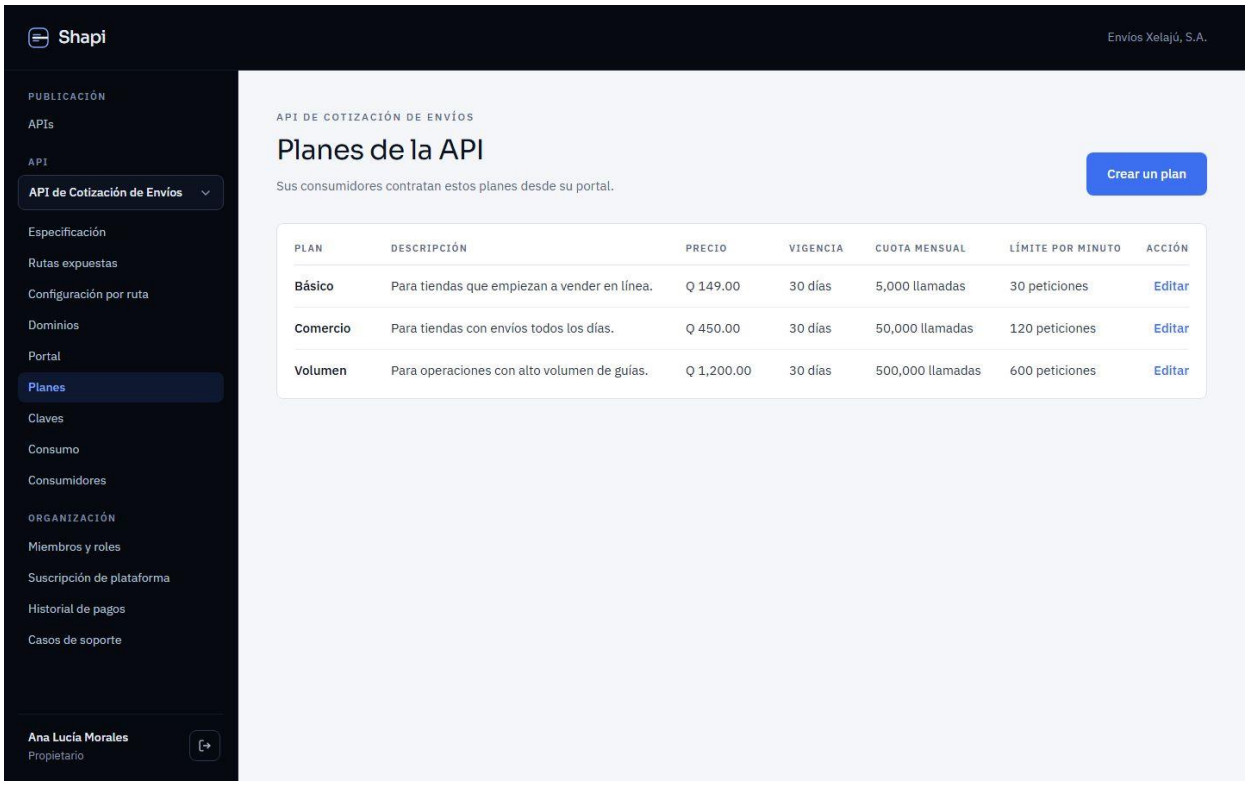


Figura 8. A4.1 · Planes de la API. Los planes que el proveedor vende a sus consumidores: precio, vigencia, cuota de llamadas y límite por minuto. Fuente: `mockups/A4/Main.dc.html`.

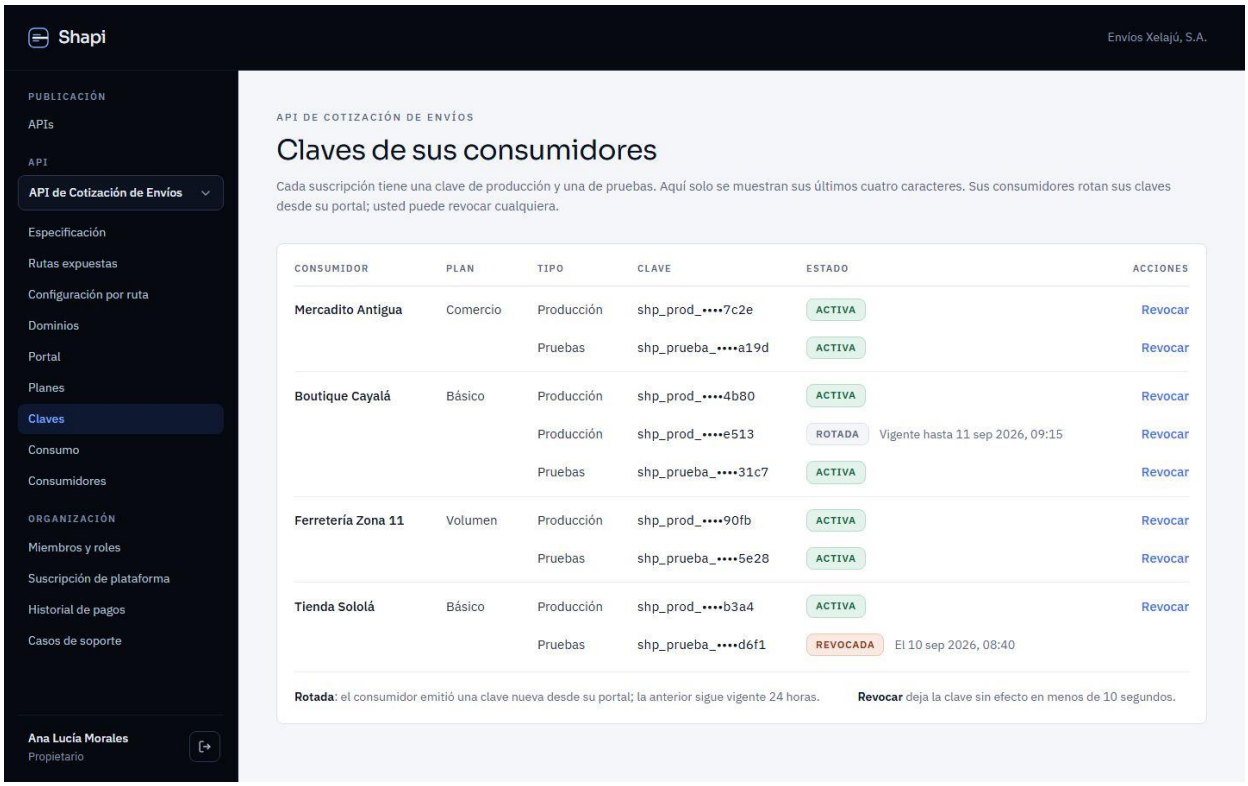


Figura 9. A4.3 · Claves de los consumidores. Las claves de los consumidores, siempre enmascaradas; el proveedor solo puede revocarlas. Fuente: `mockups/A4/Claves.dc.html`.

A5 · Portal de marca blanca

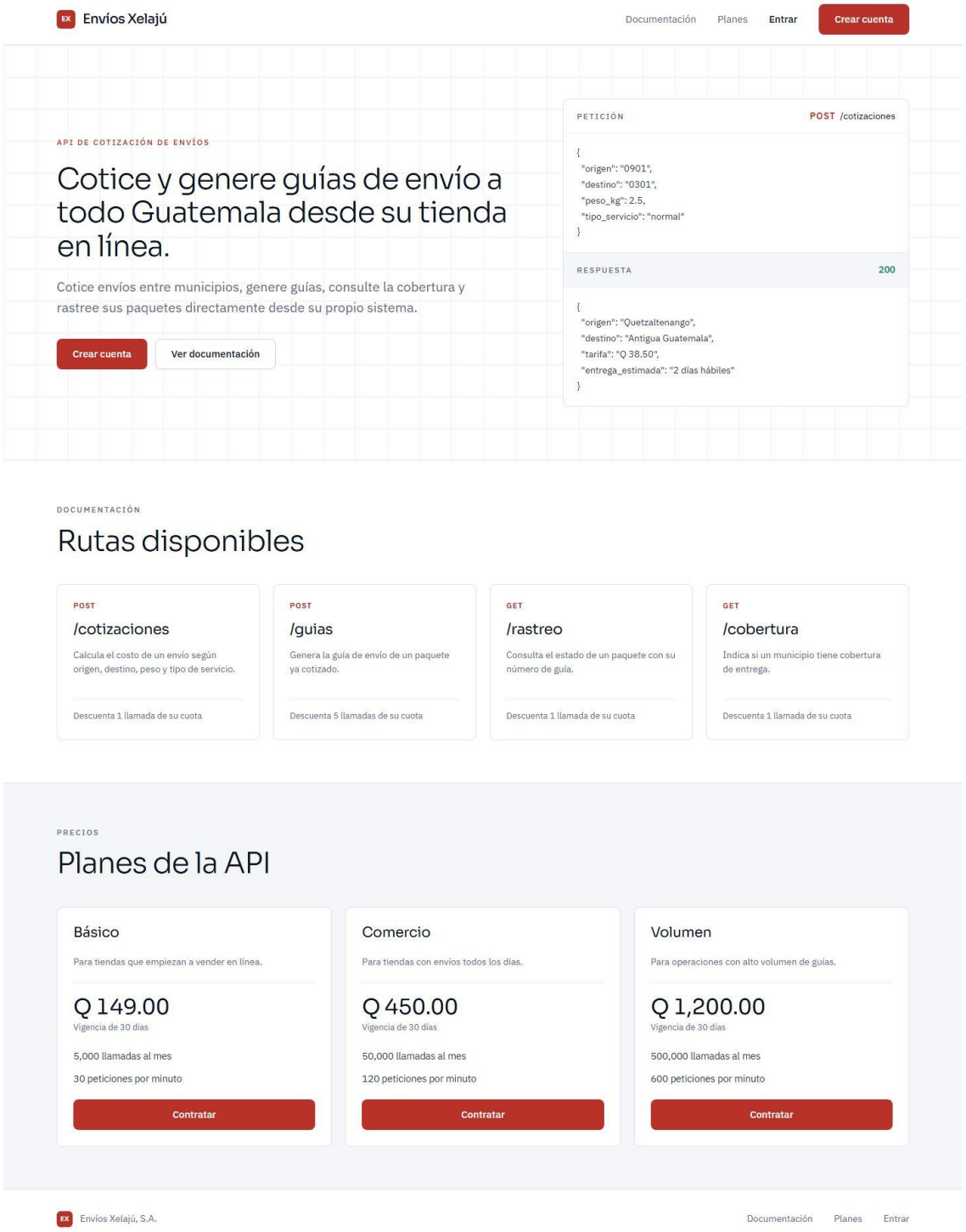


Figura 10. A5.0 · Inicio del portal. El portal de marca blanca de Envíos Xelajú, con su logotipo y su color y sin la marca de Shapi. Fuente: `mockups/A5/Main.dc.html`.

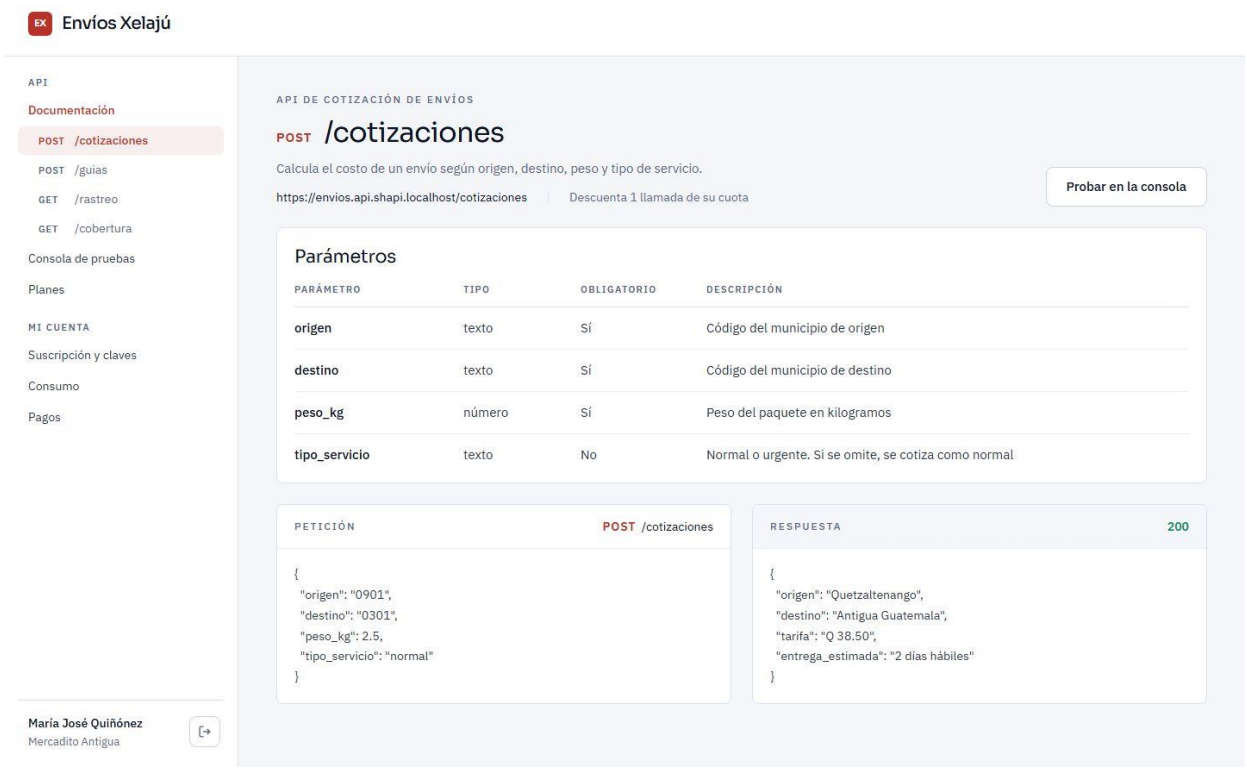


Figura 11. A5.1 · Documentación. La documentación generada desde la especificación OpenAPI: descripción, parámetros, ejemplos y peso de cada ruta. Fuente: `mockups/A5/Documentacion.dc.html`.

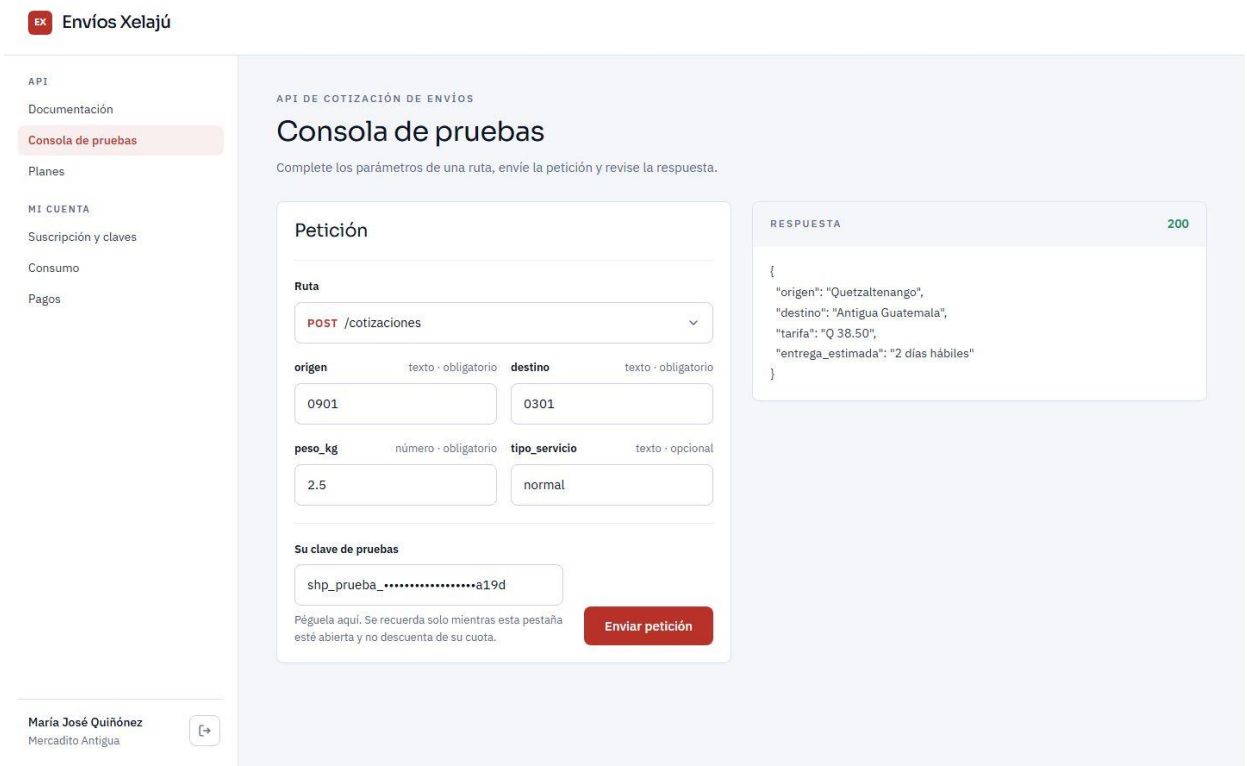


Figura 12. A5.2 · Consola de pruebas. La consola de pruebas envía peticiones reales a la API con la clave de pruebas del consumidor. Fuente: `mockups/A5/Consola.dc.html`.

A6 · Administración y soporte

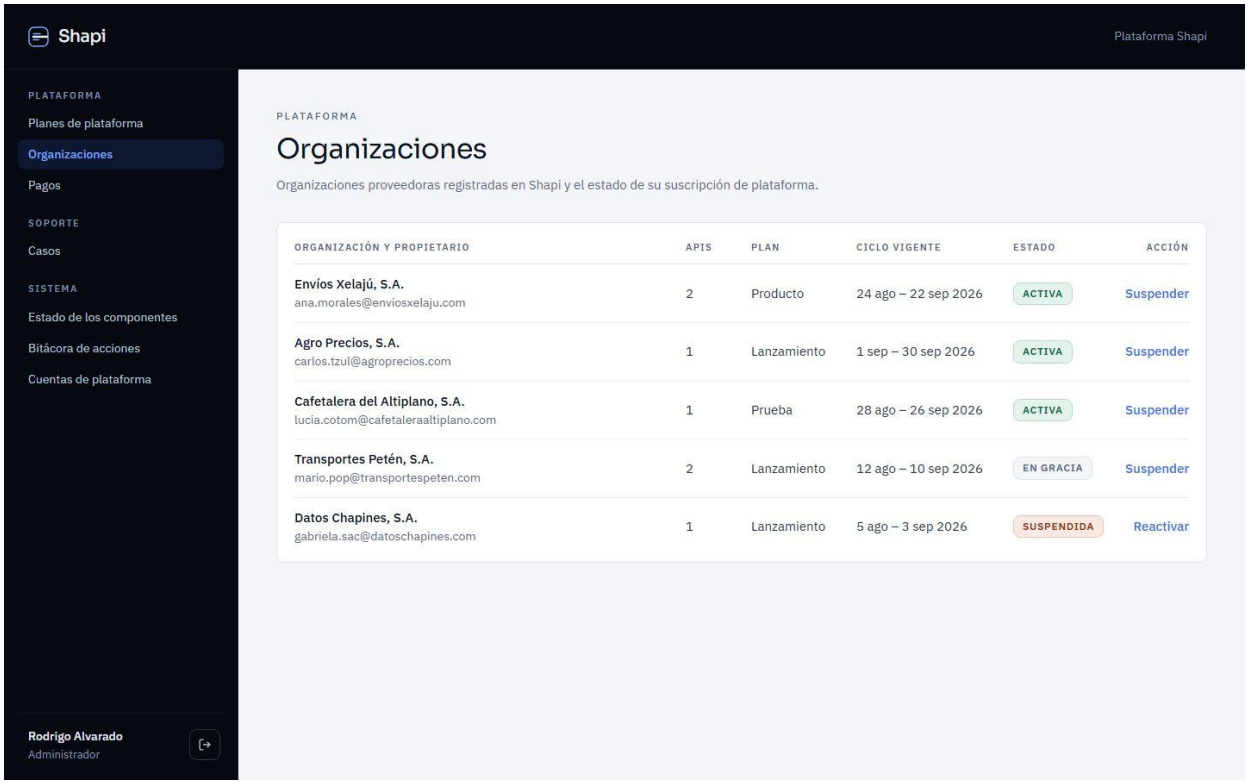


Figura 13. A6.2 · Organizaciones. El administrador lista las organizaciones y las suspende o reactiva por motivos administrativos. Fuente: `mockups/A6/Organizaciones.dc.html`.

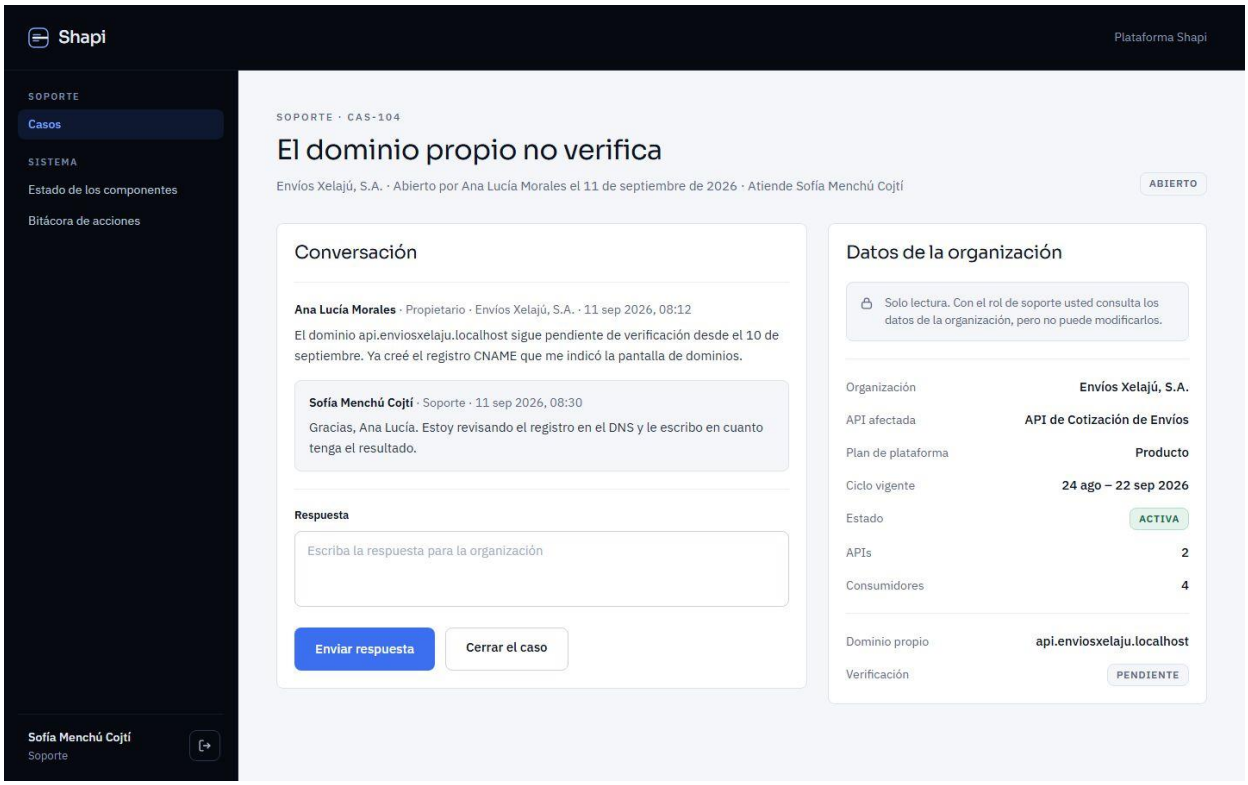


Figura 14. A6.4b · Atención de un caso. El soporte atiende un caso: conversación con el proveedor, API afectada, asignación y cierre. Fuente: `mockups/A6/Caso.dc.html`.

A7 · Soporte del proveedor

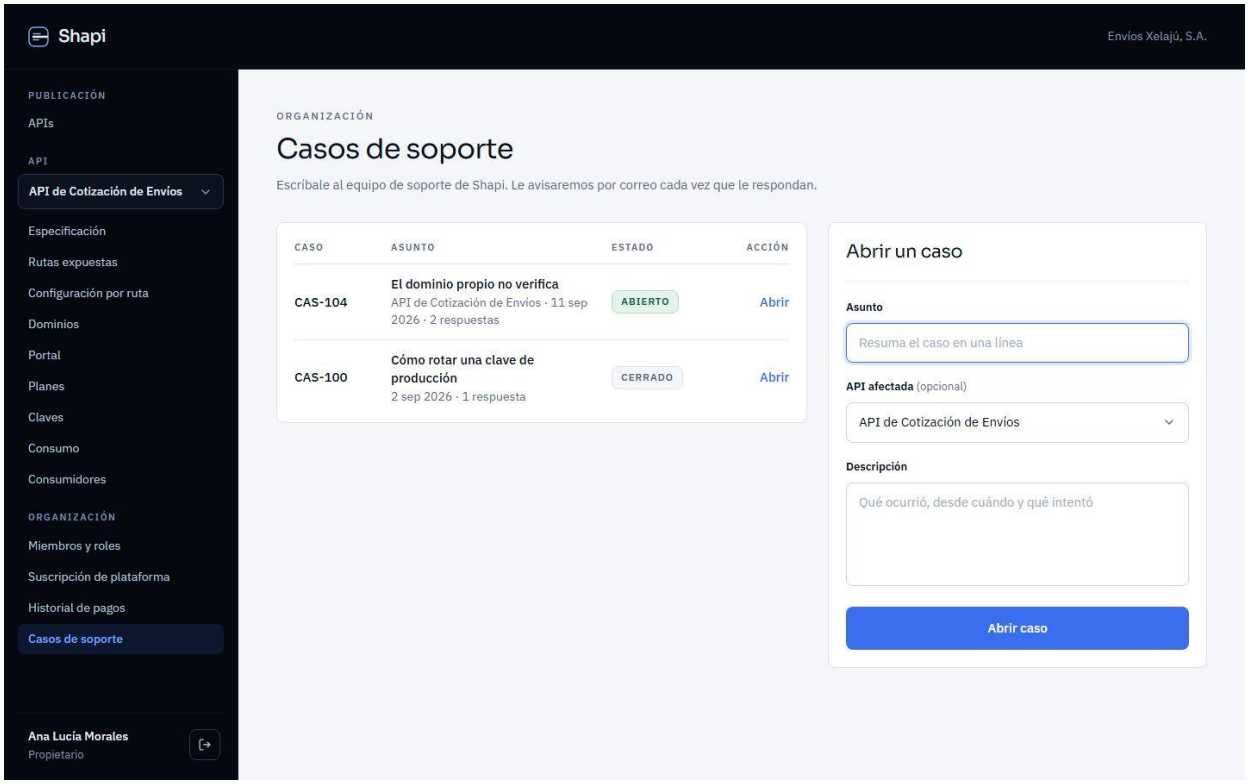


Figura 15. A7.1 · Casos de soporte de la organización y formulario para abrir uno. El proveedor abre casos de soporte y consulta los de su organización. Fuente: `mockups/A7/Casos.dc.html`.

A8 · Cuenta del personal

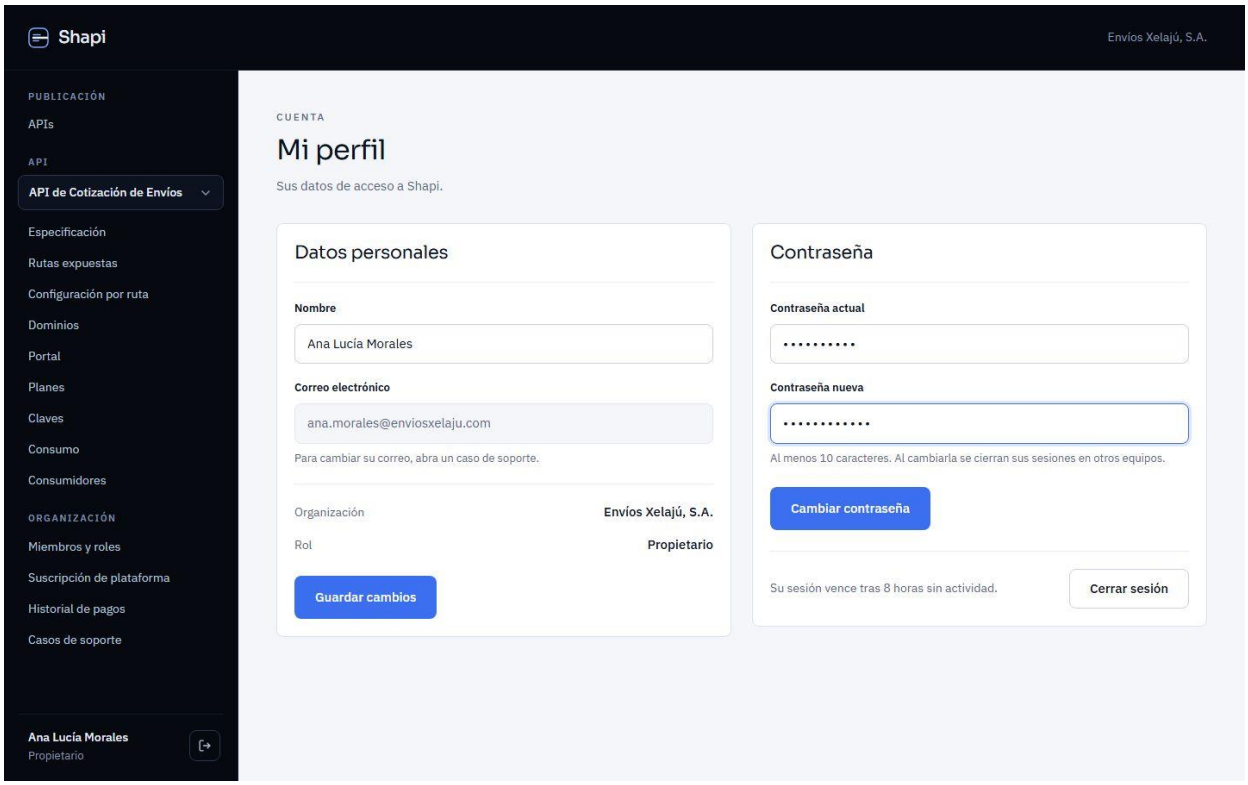


Figura 16. A8.1 · Mi perfil. Cualquier persona del personal cambia su nombre y su contraseña, o cierra la sesión. Fuente: `mockups/A8/Perfil.dc.html`.

B1 · Salidas del proveedor

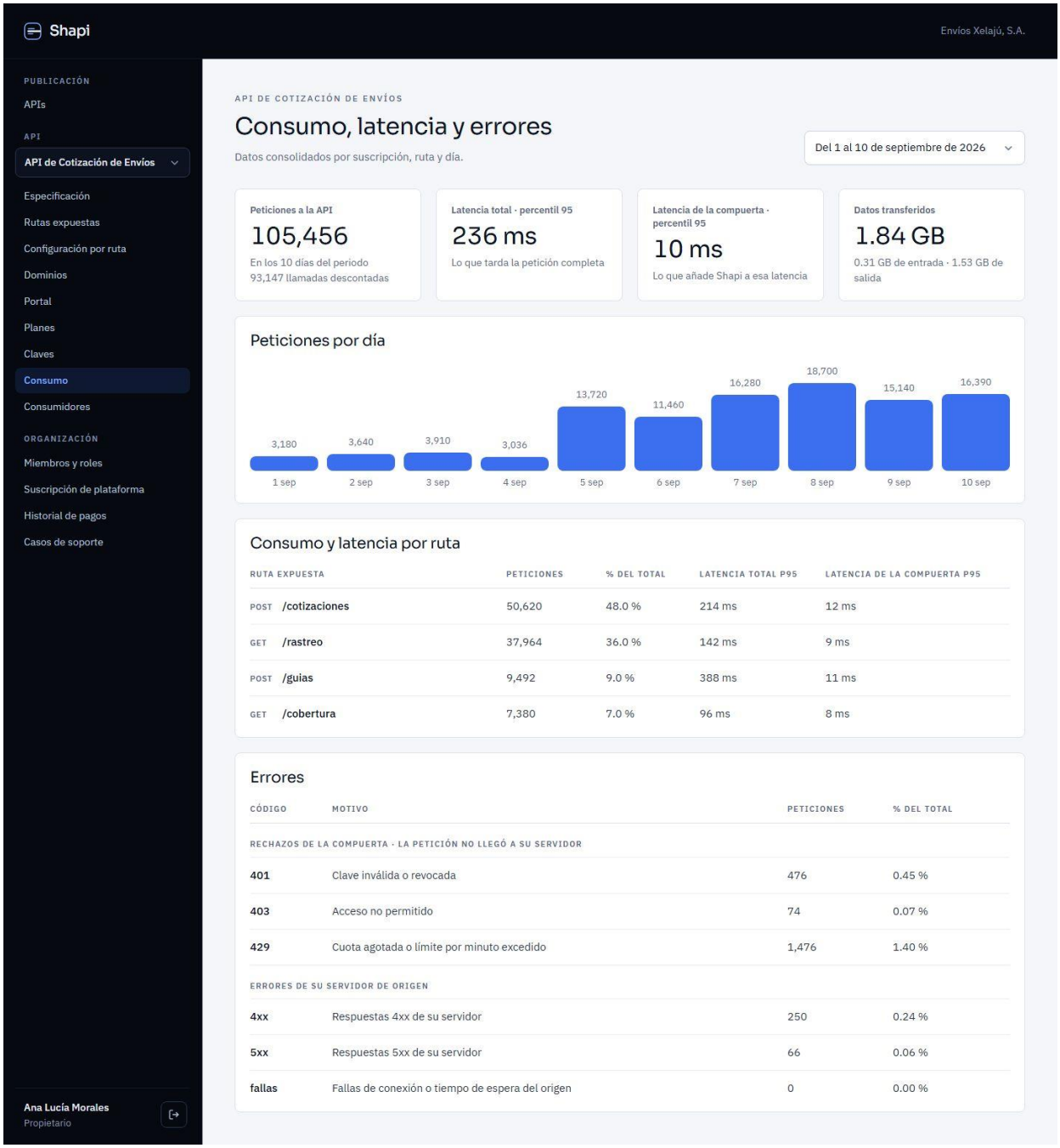


Figura 17. B1.1 · Consumo, latencia y errores. Peticiones, llamadas, latencia p95, errores por código y bytes transferidos de una API. Fuente: `mockups/B1/Main.dc.html`.

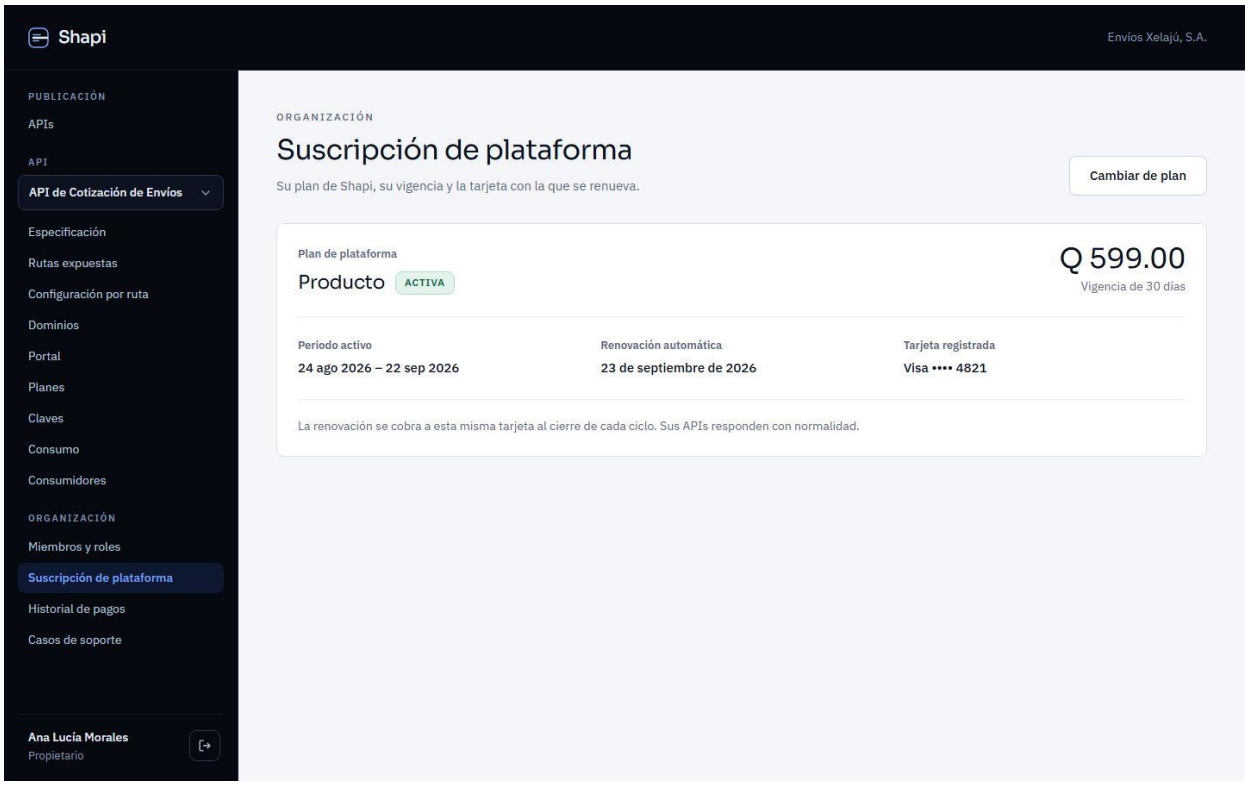


Figura 18. B1.4 · Suscripción de plataforma: vigente, en gracia y suspendida. La suscripción de plataforma vigente, con su ciclo, su consumo y el medio de pago. Fuente: `mockups/B1/Suscripcion.dc.html`.

B2 · Salidas del consumidor



Figura 19. B2.1 · Consumo del ciclo por ruta. El consumidor ve el consumo de su ciclo desglosado por ruta. Fuente: `mockups/B2/Main.dc.html`.

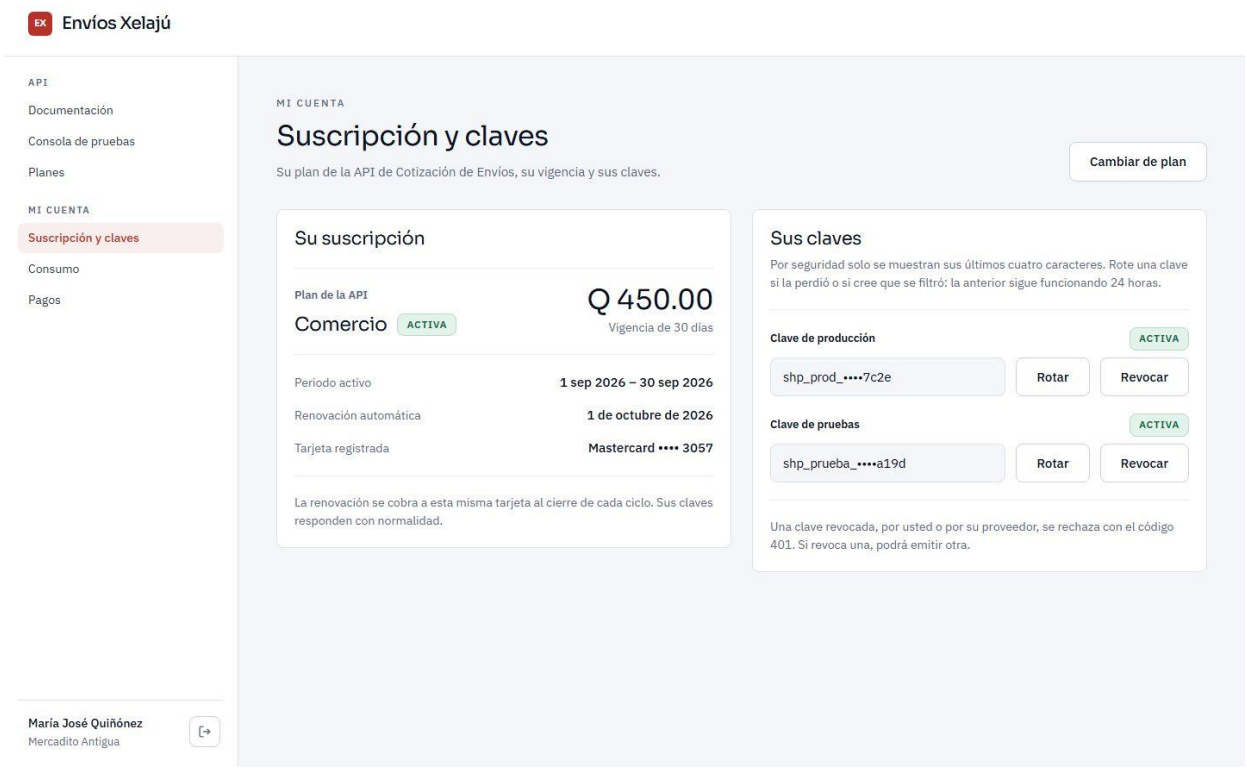


Figura 20. B2.3 · Suscripción y claves: vigente, en gracia, no disponible y vacía. La suscripción del consumidor y sus claves enmascaradas, que puede rotar o revocar. Fuente: `mockups/B2/Suscripcion.dc.html`.

B3 · Salidas del administrador y del soporte



Figura 21. B3.1 · Estado de los componentes: normal, degradado y vista de soporte. El estado de los siete componentes y la latencia p95 que añade la compuerta, actualizado cada 30 segundos. Fuente: `mockups/B3/Main.dc.html`.

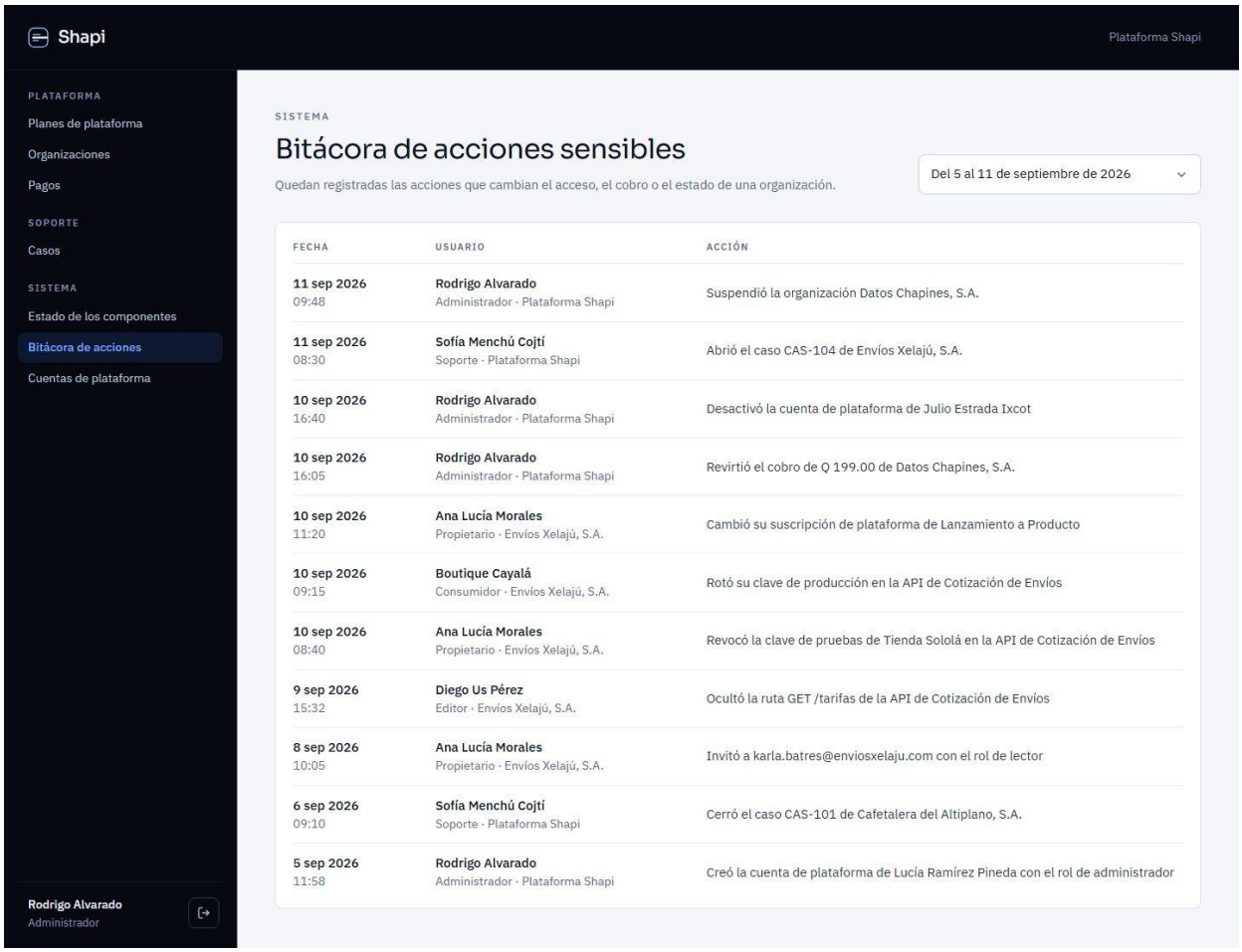


Figura 22. B3.2 · Bitácora. La bitácora de acciones sensibles: actor, organización, acción, objetivo, fecha e IP. Fuente: `mockups/B3/Bitacora.dc.html`.

9.4 Catálogo completo de pantallas

El catálogo lista todas las pantallas aprobadas, con su ruta y los requisitos que cubren. Varias pantallas tienen además variantes (vacía, en gracia, suspendida, con error) en el mismo mockup.

A0 · Sitio público

ID	Pantalla	Ruta	Requisitos
A0.1	Inicio de Shapi con planes	`shapi.localhost/`	RF-19
A0.2	Lámina de estilo oficial	`shapi.localhost/_ui` (solo en desarrollo)	—

A1 · Acceso del proveedor

ID	Pantalla	Ruta	Requisitos
A1.1	Registro del proveedor	`/registro`	RF-01
A1.2	Aviso de verificación de correo	`/verificar-correo`	RF-02
A1.3	Inicio de sesión	`/entrar`	RF-04
A1.4a	Recuperación: pedir el enlace	`/recuperar`	RF-03
A1.4b	Recuperación: contraseña nueva	`/restablecer?token=`	RF-03

A2 · Plan de plataforma

ID	Pantalla	Ruta	Requisitos
A2.1	Planes de plataforma	`/panel/suscripcion/planes`	RF-19
A2.2	Contratar un plan superior	`/panel/suscripcion/contratar/:plan`	RF-20
A2.3	Confirmación	—	RF-20
A2.4	Tarjeta rechazada	—	RF-20
A2.5	Cambio de plan con prorrateo	`/panel/suscripcion/cambiar/:plan`	RF-25

A3 · Publicación de una API

ID	Pantalla	Ruta	Requisitos
A3.1	APIs de la organización (y su variante vacía)	`/panel/apis`	RF-14, RF-43
A3.2	Registrar una API	`/panel/apis/nueva`	RF-08
A3.3	Cargar especificación	`/panel/apis/:id/especificacion`	RF-09
A3.4	Rutas expuestas	`/panel/apis/:id/rutas`	RF-10
A3.5	Configuración por ruta	`/panel/apis/:id/configuracion-rutas`	RF-13
A3.6	Direcciones, dominio propio y secreto de origen	`/panel/apis/:id/dominios`	RF-11, RF-12, RF-47
A3.7	Personalización del portal	`/panel/apis/:id/portal`	RF-15
A3.7r	Referencia: inicio del portal a tamaño real	—	RF-16

A4 · Planes, claves y equipo

ID	Pantalla	Ruta	Requisitos
A4.1	Planes de la API (vacía, editar, nuevo)	`/panel/apis/:id/planes`	RF-18
A4.2	Miembros y roles (vacía, quitar)	`/panel/miembros`	RF-06, RF-43
A4.3	Claves de los consumidores (vacía)	`/panel/apis/:id/claves`	RF-26, RF-28
A4.3b	Revocar una clave	—	RF-28

A5 · Portal de marca blanca

ID	Pantalla	Ruta	Requisitos
A5.0	Inicio del portal	`/`	RF-15, RF-16
A5.1	Documentación	`/documentacion` y `/documentacion/:ruta`	RF-16
A5.2	Consola de pruebas	`/consola`	RF-16, RF-45
A5.3	Registro del consumidor	`/registro`	RF-05
A5.3b	Registro por invitación	`/invitacion?token=`	RF-05
A5.4	Planes de la API (vacía)	`/planes`	RF-19
A5.4b	Plan contratado y claves (única vez)	—	RF-20, RF-26
A5.5	El mismo portal con otra marca (inicio y documentación)	`agro.shapi.localhost`	RF-15
A5.6	Pago en el portal	`/contratar/:plan`	RF-20

ID	Pantalla	Ruta	Requisitos
A5.7	Inicio de sesión del consumidor	`/entrar`	RF-04
A5.8	Verificación de correo del consumidor	`/verificar-correo`	RF-02
A5.9	Recuperación: pedir el enlace	`/recuperar`	RF-03
A5.10	Recuperación: contraseña nueva	`/restablecer?token=`	RF-03

A6 · Administración y soporte

ID	Pantalla	Ruta	Requisitos
A6.1	Planes de plataforma (nuevo y editar)	`/admin/planes`	RF-17
A6.2	Organizaciones (vacía)	`/admin/organizaciones`	RF-38
A6.2b	Suspender una organización	—	RF-38
A6.3	Pagos de plataforma (vacía)	`/admin/pagos`	RF-24
A6.3b	Revertir un pago	—	RF-24
A6.4	Casos de soporte (vacía)	`/admin/casos`	RF-40
A6.4b	Atención de un caso	`/admin/casos/:numero`	RF-40
A6.5	Cuentas de administración y soporte (vacía)	`/admin/cuentas`	RF-42

A7 · Soporte del proveedor

ID	Pantalla	Ruta	Requisitos
A7.1	Casos de soporte de la organización y formulario para abrir uno	`/panel/soporte`	RF-40
A7.2	Conversación de un caso	`/panel/soporte/:numero`	RF-40

A8 · Cuenta del personal

ID	Pantalla	Ruta	Requisitos
A8.1	Mi perfil (nombre, contraseña, cerrar sesión)	`/panel/perfil` y `/admin/perfil`	RF-04
A8.2	Aceptar la invitación de un miembro	`/invitacion?token=`	RF-06

B1 · Salidas del proveedor

ID	Pantalla	Ruta	Requisitos
B1.1	Consumo, latencia y errores (vacía)	`/panel/apis/:id/consumo`	RF-35
B1.2	Consumo y facturación por consumidor (vacía)	`/panel/apis/:id/consumidores`	RF-36
B1.3	Historial de pagos (vacía)	`/panel/pagos`	RF-21
B1.4	Suscripción de plataforma: vigente, en gracia y suspendida	`/panel/suscripcion`	RF-22, RF-23
B1.5	Invitar consumidores	`/panel/apis/:id/consumidores/invitar`	RF-05

B2 · Salidas del consumidor

ID	Pantalla	Ruta	Requisitos

ID	Pantalla	Ruta	Requisitos
B2.1	Consumo del ciclo por ruta (vacía)	`/cuenta/consumo`	RF-37
B2.2	Historial de pagos (vacía)	`/cuenta/pagos`	RF-21
B2.3	Suscripción y claves: vigente, en gracia, no disponible y vacía	`/cuenta/suscripcion`	RF-22, RF-23, RF-26 a RF-28
B2.4	Rotar una clave	—	RF-27
B2.5	Clave nueva (única vez)	—	RF-27
B2.6	Revocar una clave propia	—	RF-28
B2.7	Cambio de plan del consumidor	`/cuenta/cambiar-plan/:plan`	RF-25

B3 · Salidas del administrador y del soporte

ID	Pantalla	Ruta	Requisitos
B3.1	Estado de los componentes: normal, degradado y vista de soporte	`/admin/estado`	RF-39
B3.2	Bitácora (vacía)	`/admin/bitacora`	RF-41

N · Navegación

ID	Pantalla	Ruta	Requisitos
N.1	Barra lateral del proveedor, aplicada a A4.1	—	—

10. Diagramas

Los diagramas están escritos en Mermaid dentro de las especificaciones (`docs/specs/`), así que se versionan junto al código y GitHub los dibuja. Los de este capítulo son copia fiel de esas fuentes, con dos excepciones: los flujos de 10.2.2 a 10.2.4, que se escribieron para este manual a partir de las especificaciones (01 §5, 05 y 09), y el modelo relacional, que se generó a partir del modelo real de la base de datos (`ShapiDbContextModelSnapshot.cs`, la instantánea que Entity Framework Core mantiene de las migraciones).

10.1 Diagramas UML

10.1.1 Casos de uso

Hay un diagrama por grupo de actores. El caso compartido de autenticación (CU-02 y CU-03) aparece en los tres. La descripción de cada caso de uso, con su flujo principal y sus alternativas, está en `docs/specs/05-casos-de-uso.md`.

Proveedor

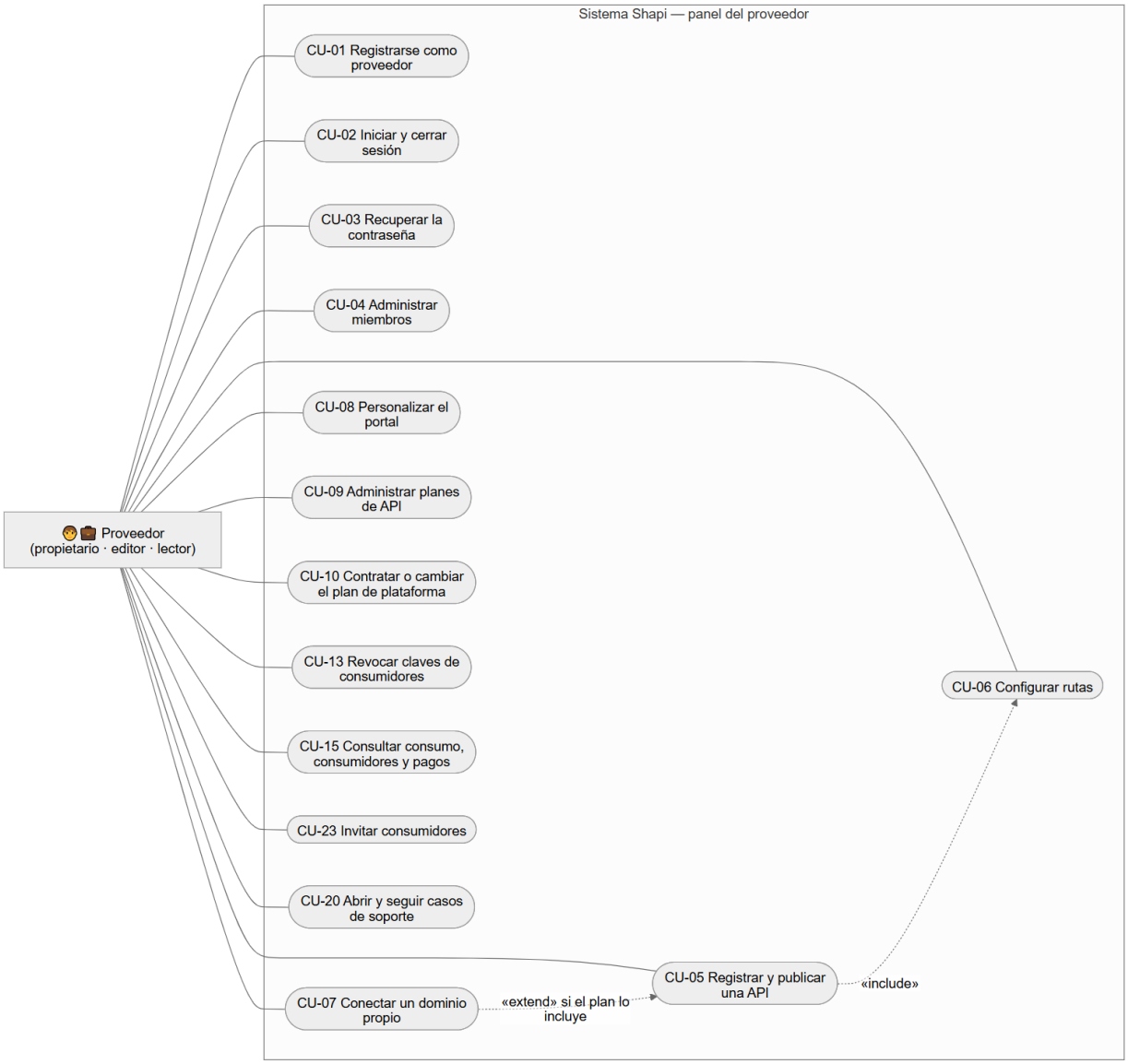


Diagrama 6. Proveedor.

Consumidor

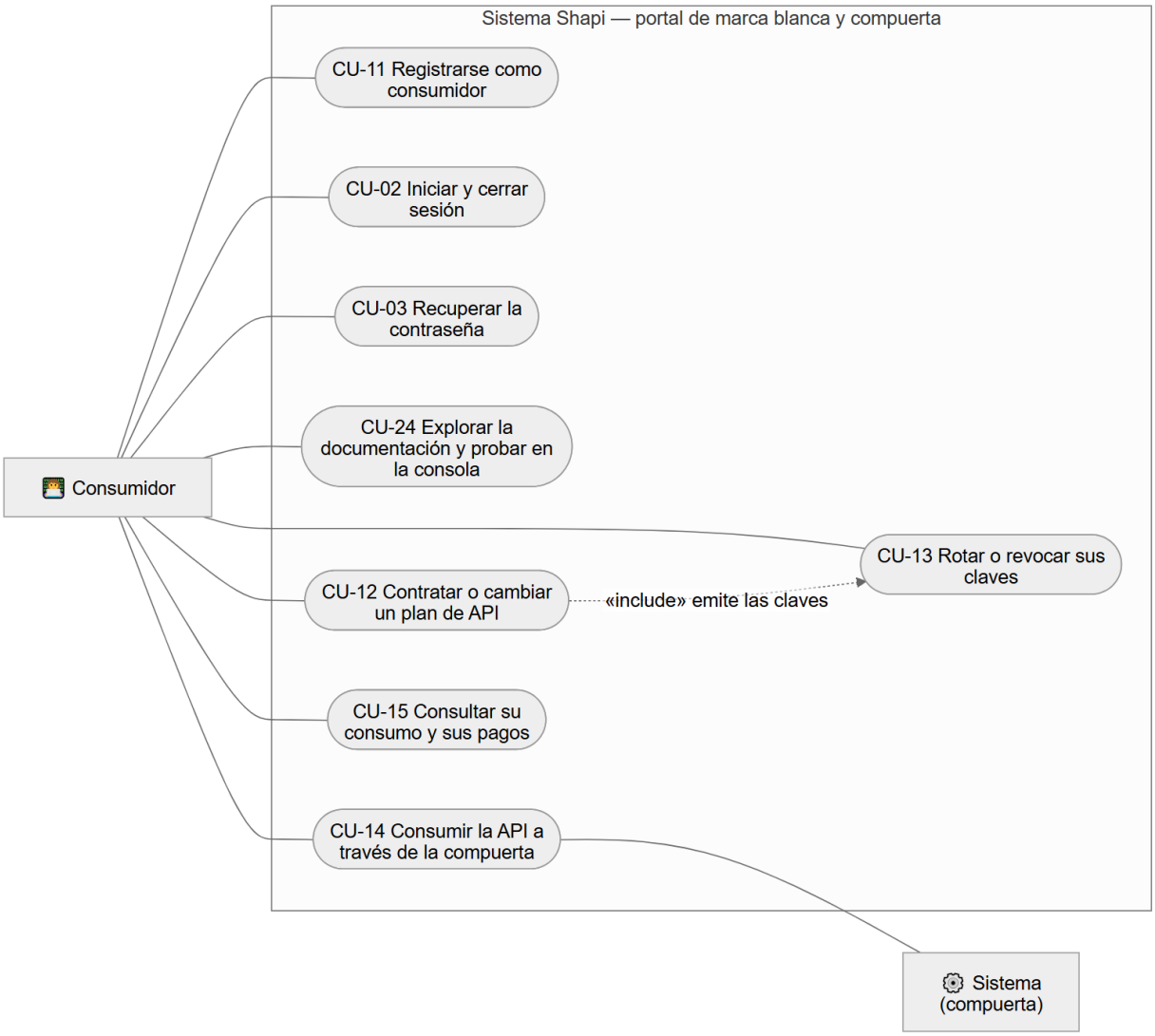


Diagrama 7. Consumidor.

Administrador, soporte y procesos automáticos

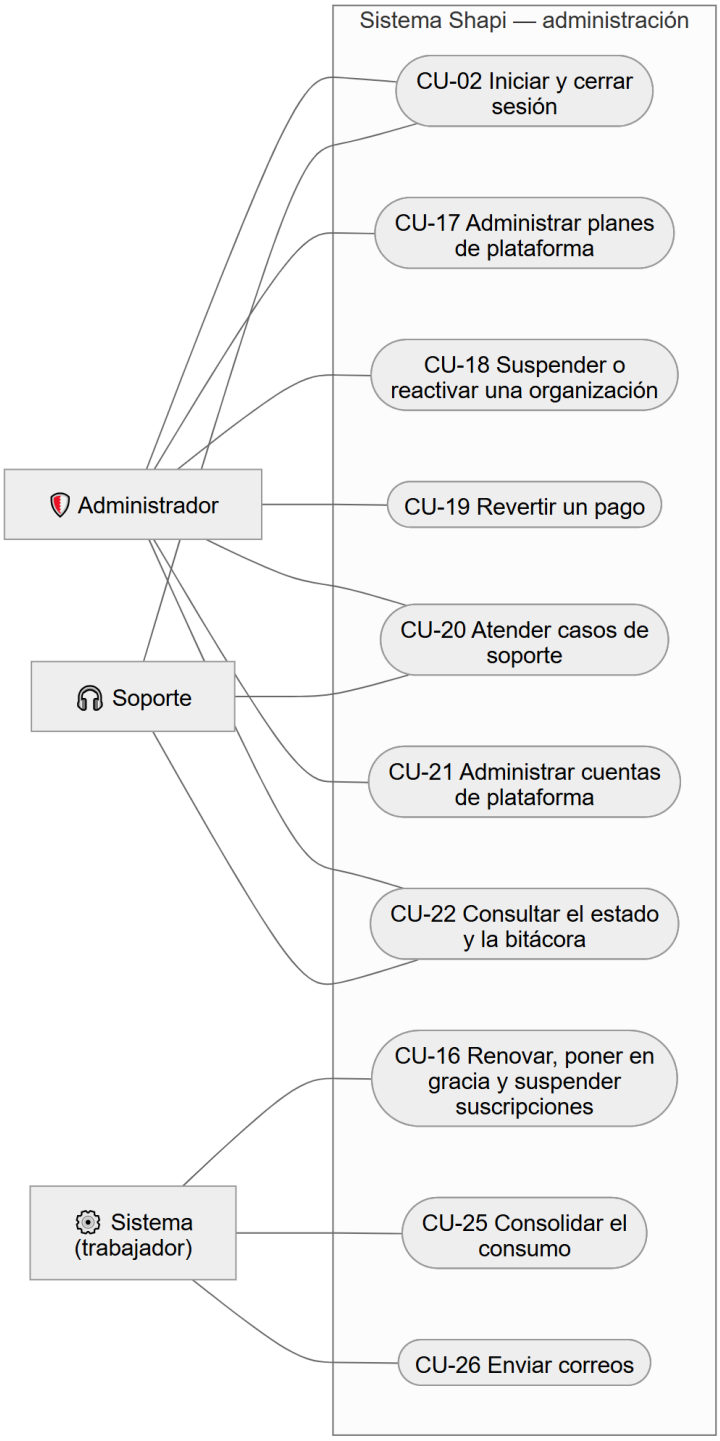


Diagrama 8. Administrador, soporte y procesos automáticos.

10.1.2 Diagrama de clases del dominio

Las clases del dominio (`src/Shapi.Dominio`) con sus atributos principales, sus métodos y sus relaciones. Hay dos ámbitos de identidad (`Usuario` para el personal y `Consumidor` para los clientes de cada proveedor) y dos jerarquías de suscripción que comparten la misma máquina de estados.

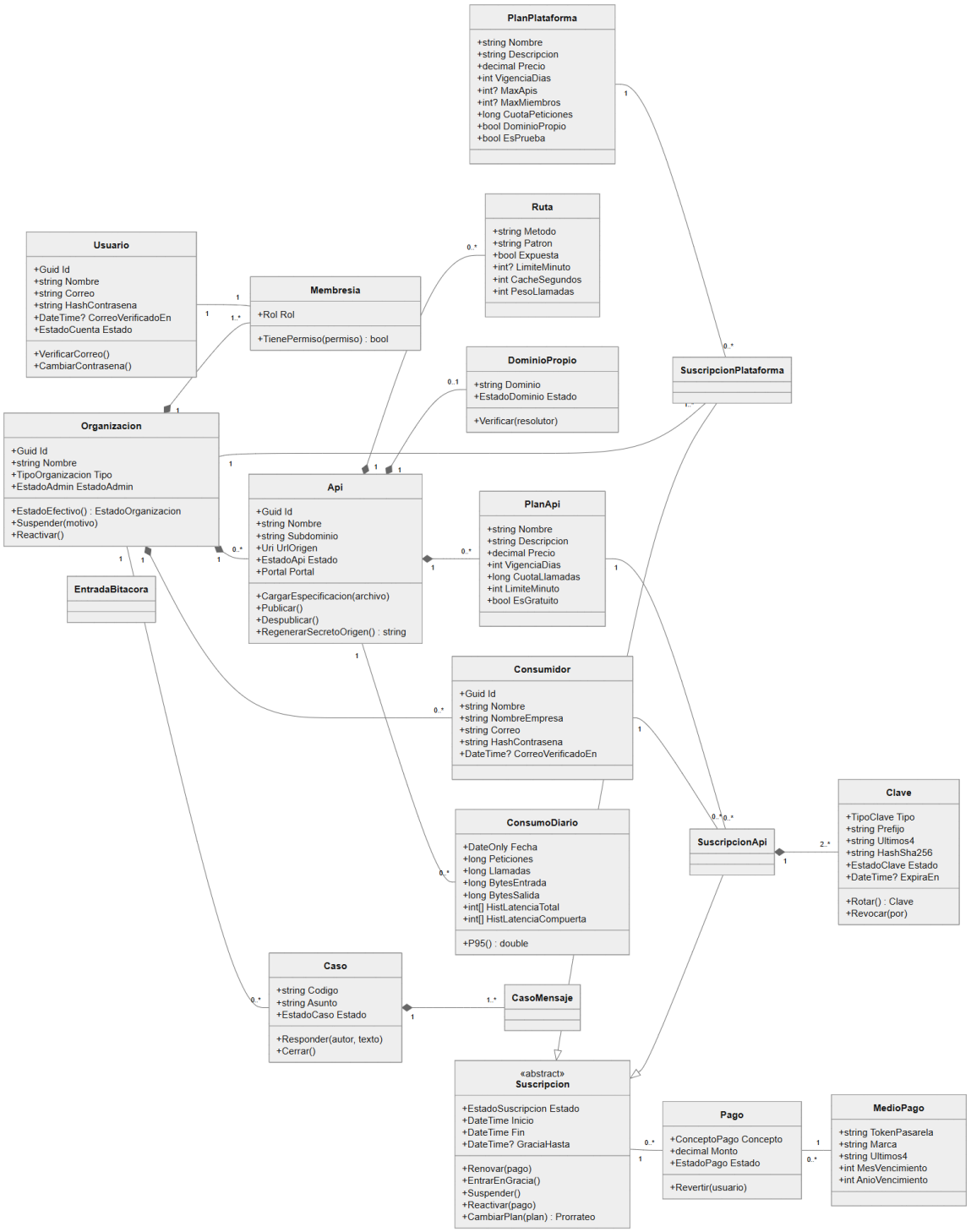


Diagrama 9. Diagrama de clases del dominio.

10.1.3 Diagramas de secuencia

Procesamiento de una petición en la compuerta

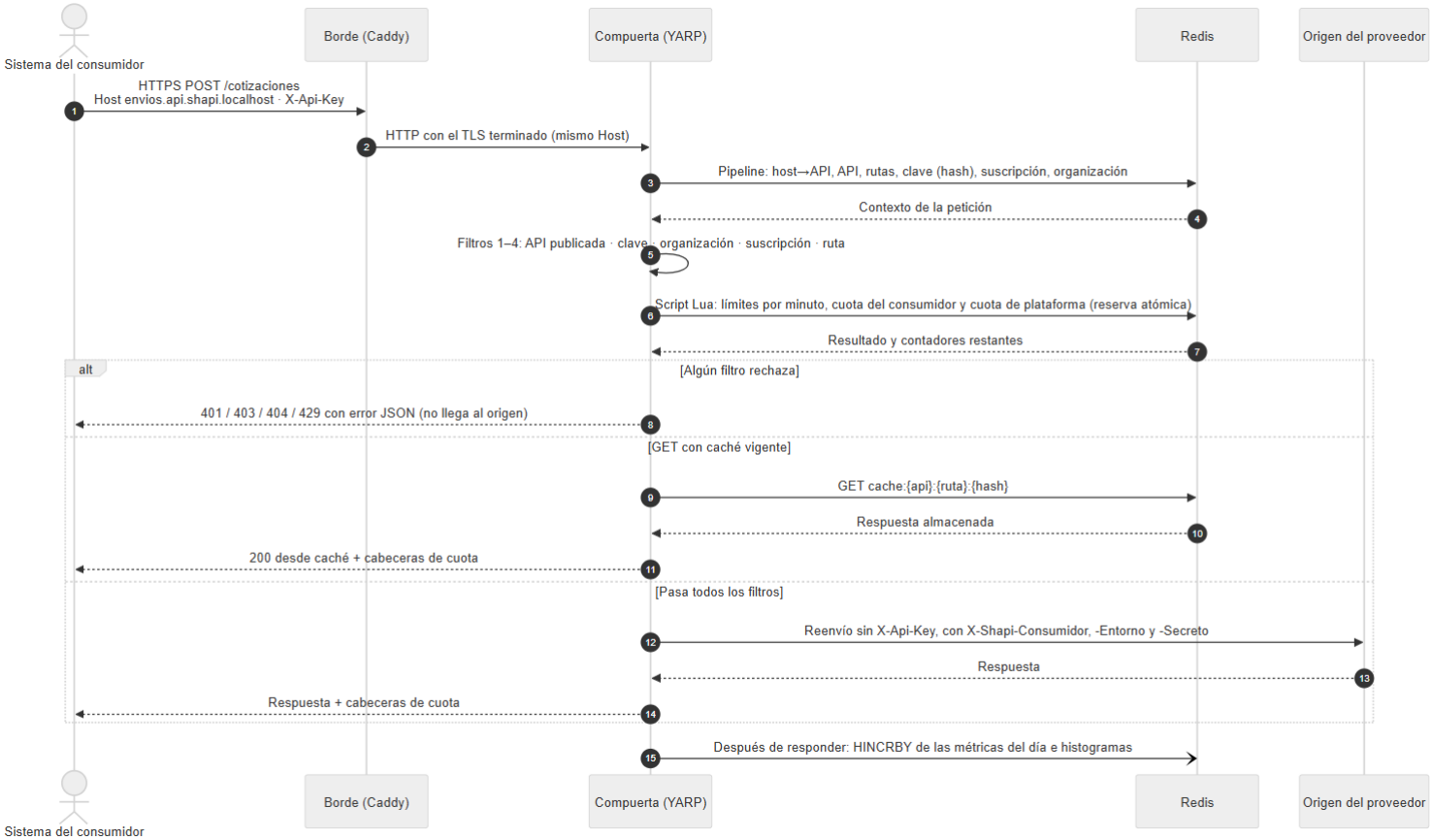


Diagrama 10. Procesamiento de una petición en la compuerta.

Contratación de un plan de API

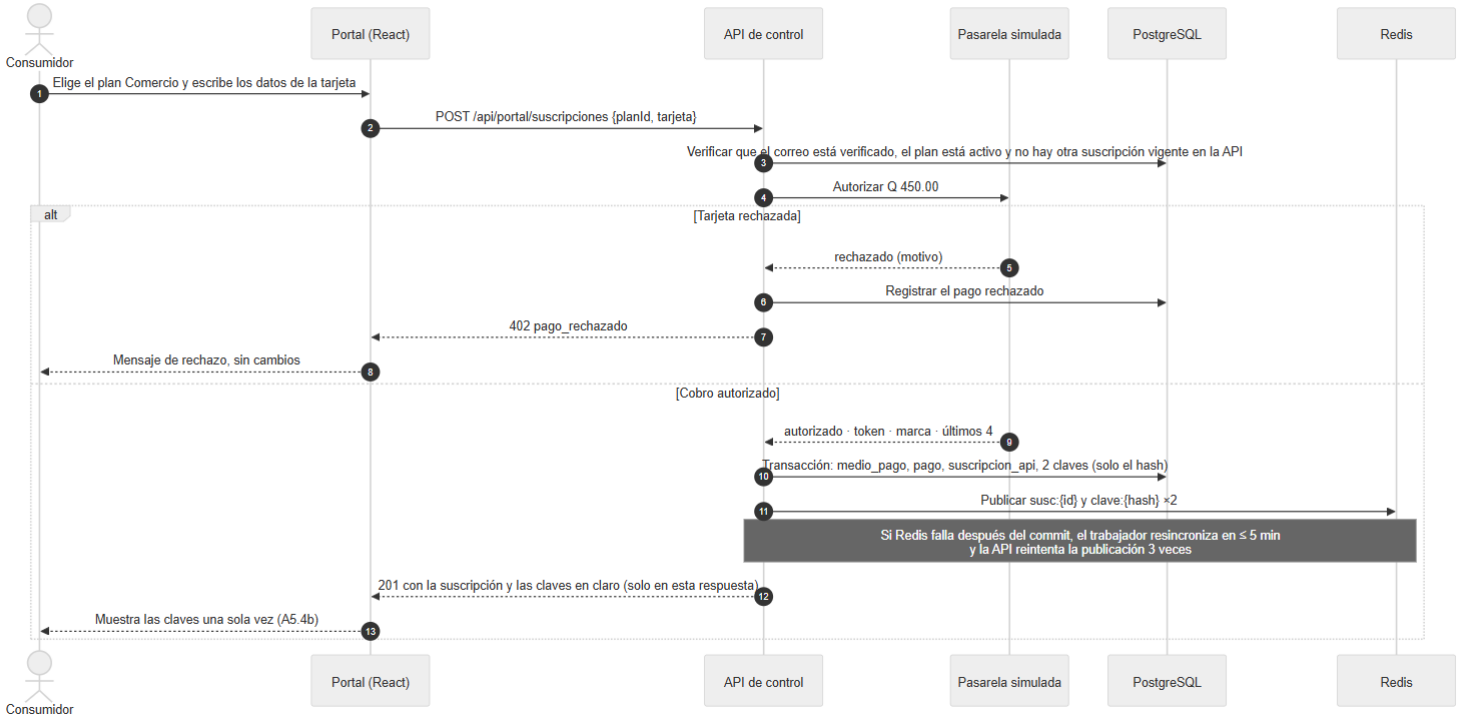


Diagrama 11. Contratación de un plan de API.

Publicación de una API

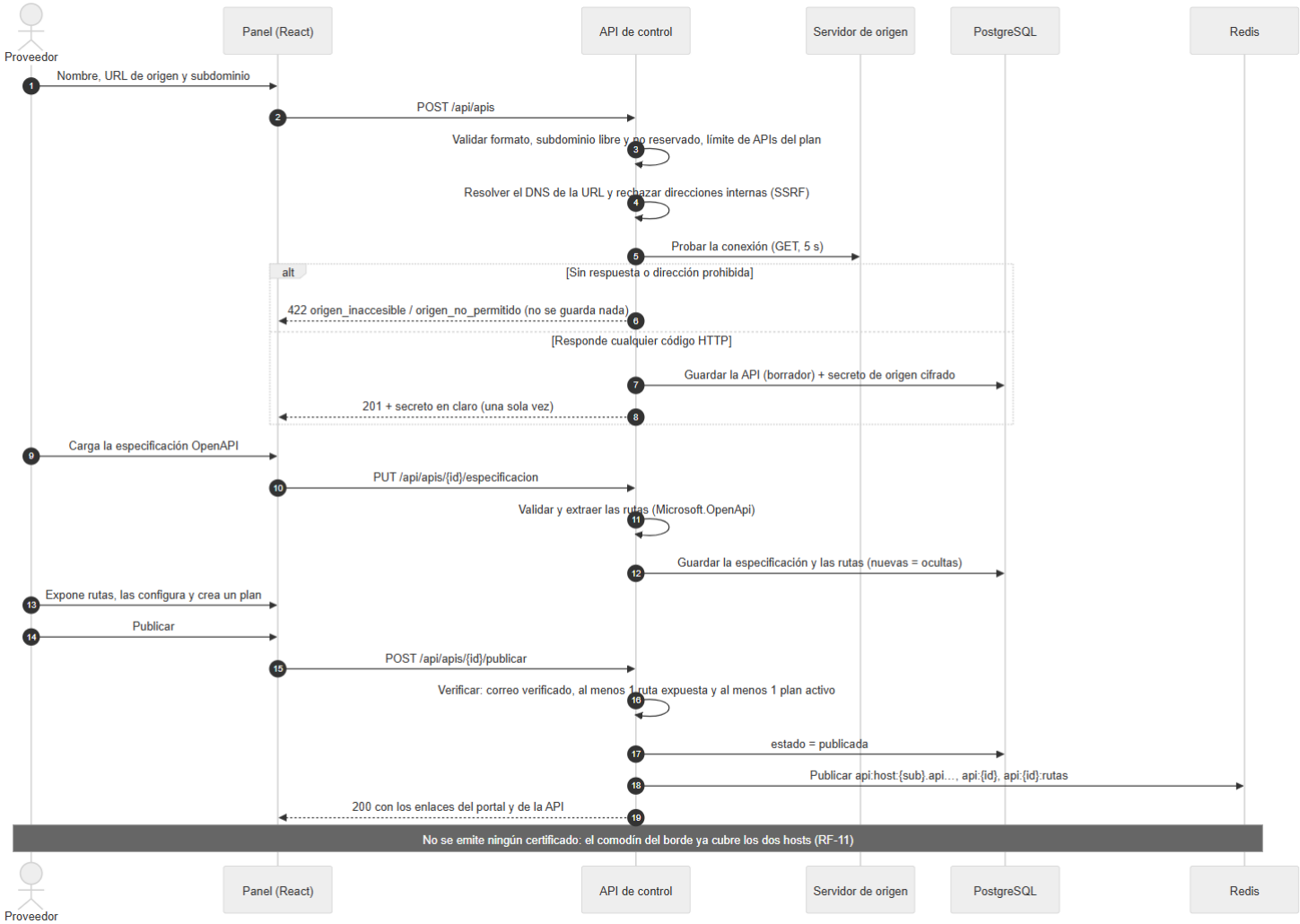


Diagrama 12. Publicación de una API.

Rotación de una clave

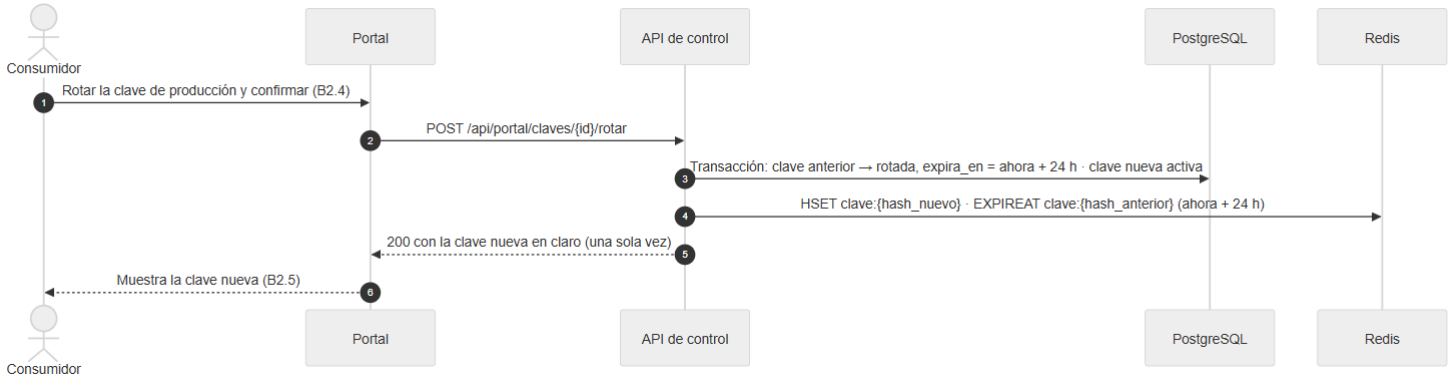


Diagrama 13. Rotación de una clave.

Cierre de ciclo, renovación, gracia y suspensión

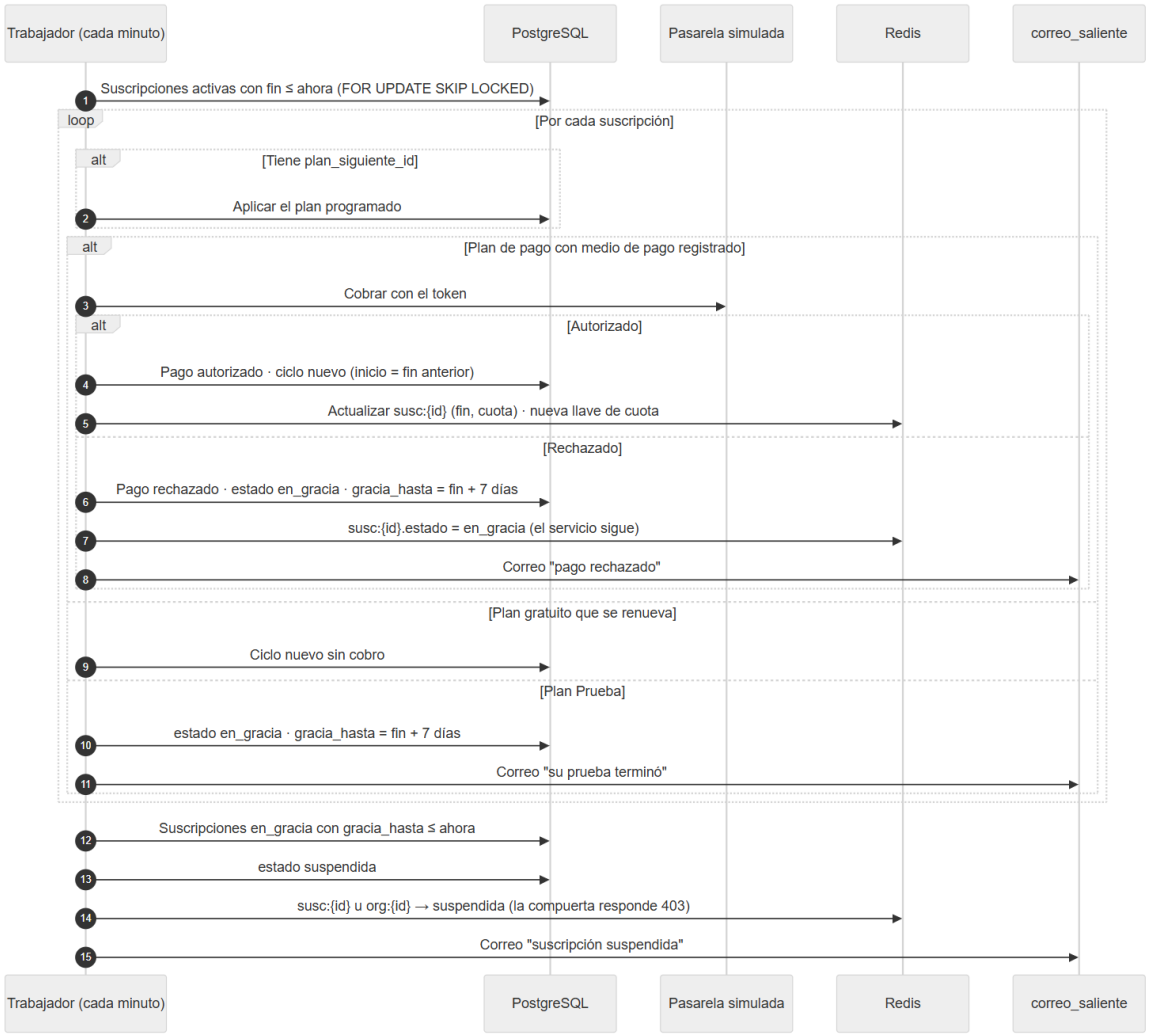


Diagrama 14. Cierre de ciclo, renovación, gracia y suspensión.

Verificación de un dominio propio y emisión del certificado

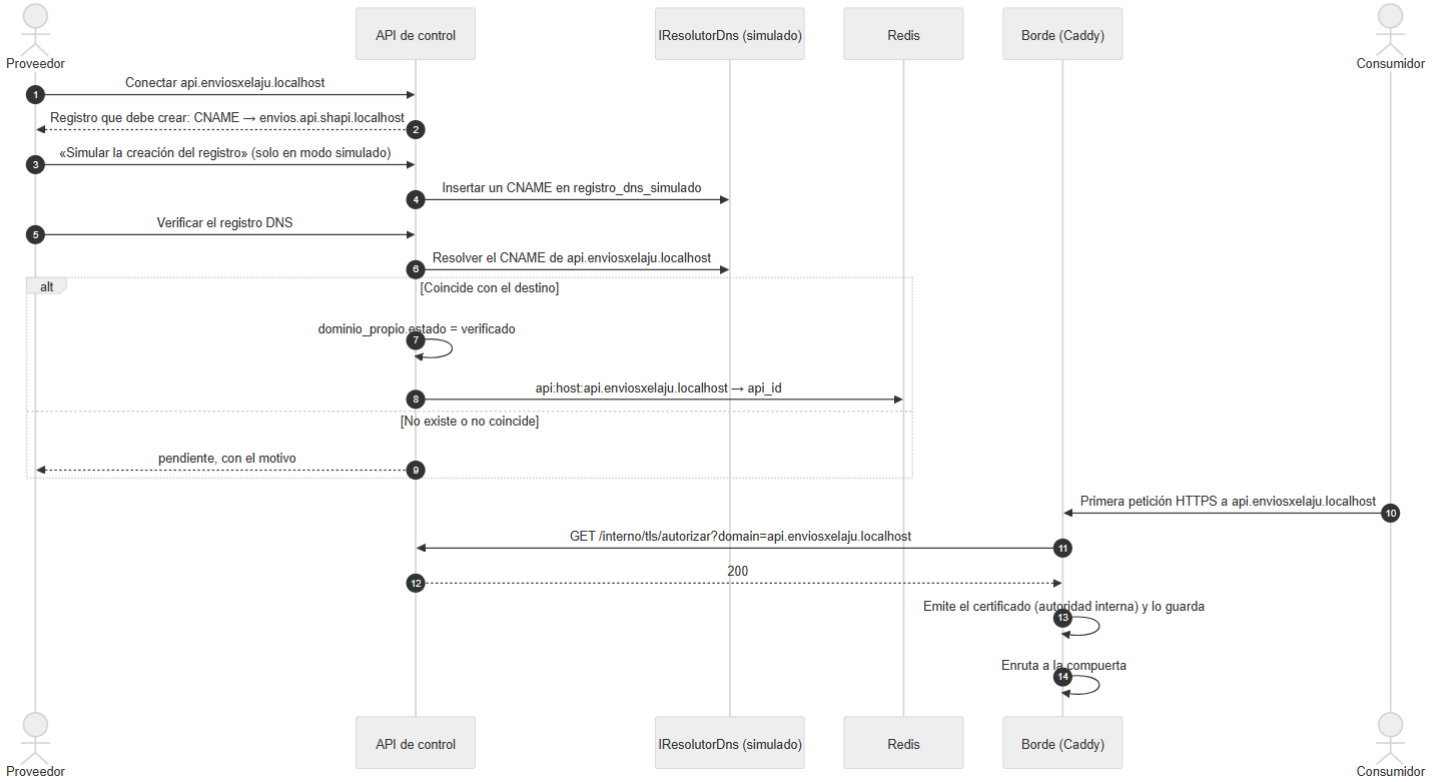


Diagrama 15. Verificación de un dominio propio y emisión del certificado.

Consolidación del consumo

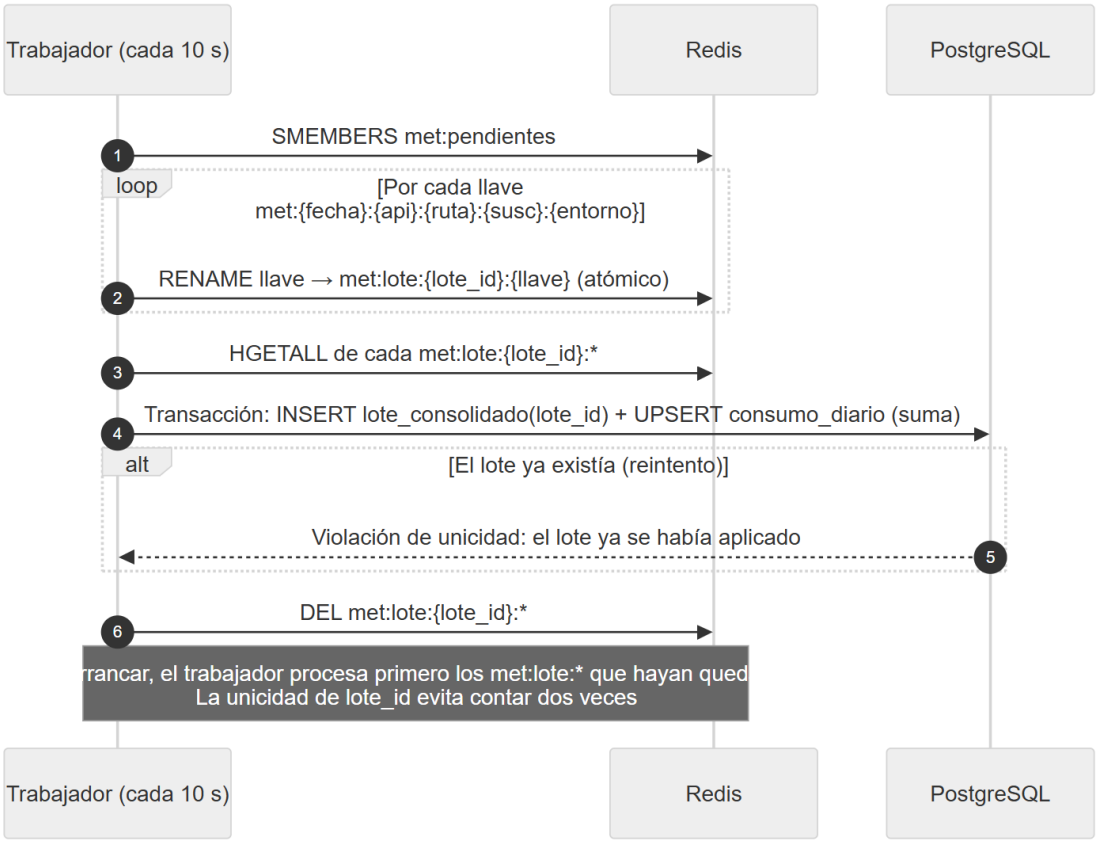


Diagrama 16. Consolidación del consumo.

10.1.4 Diagramas de estados

Suscripciones (de plataforma y de API; las dos usan la misma máquina de estados)

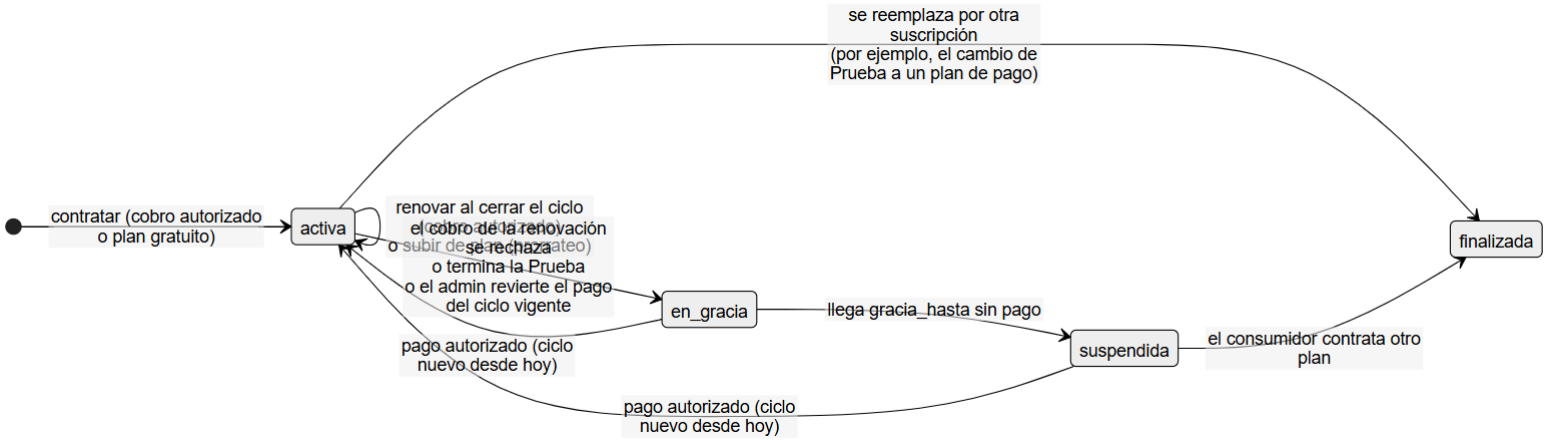


Diagrama 17. Diagramas de estados.

API

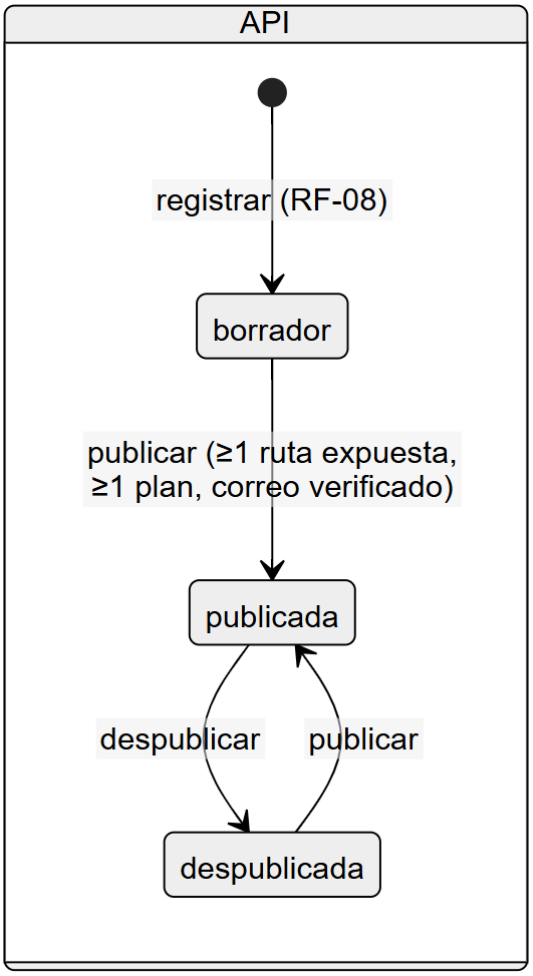


Diagrama 18. API.

Clave

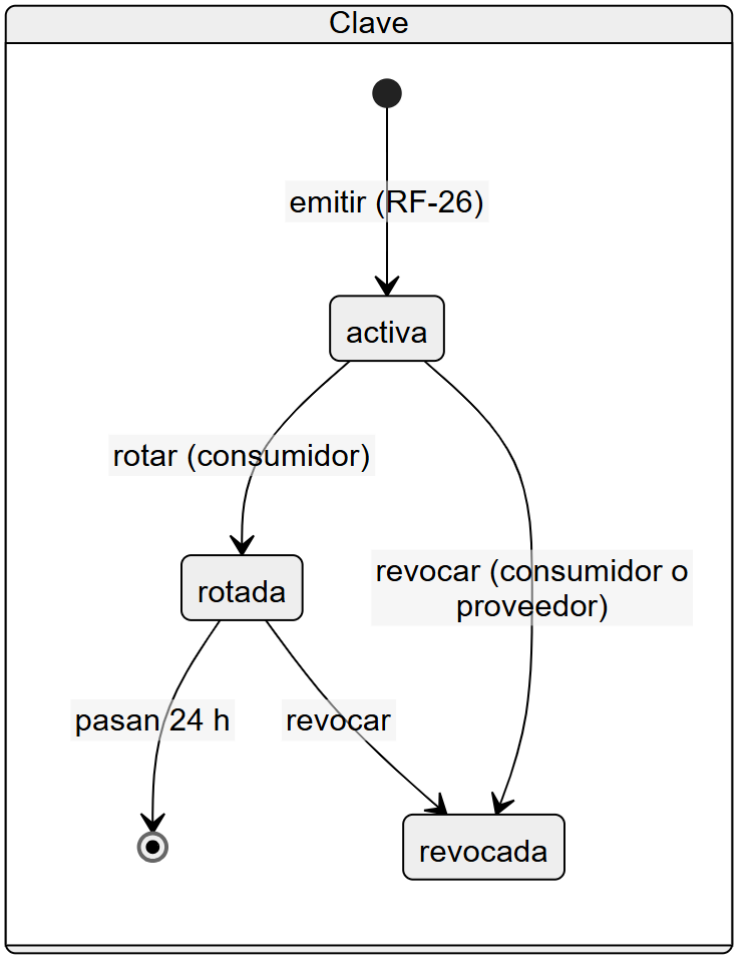


Diagrama 19. Clave.

Dominio propio

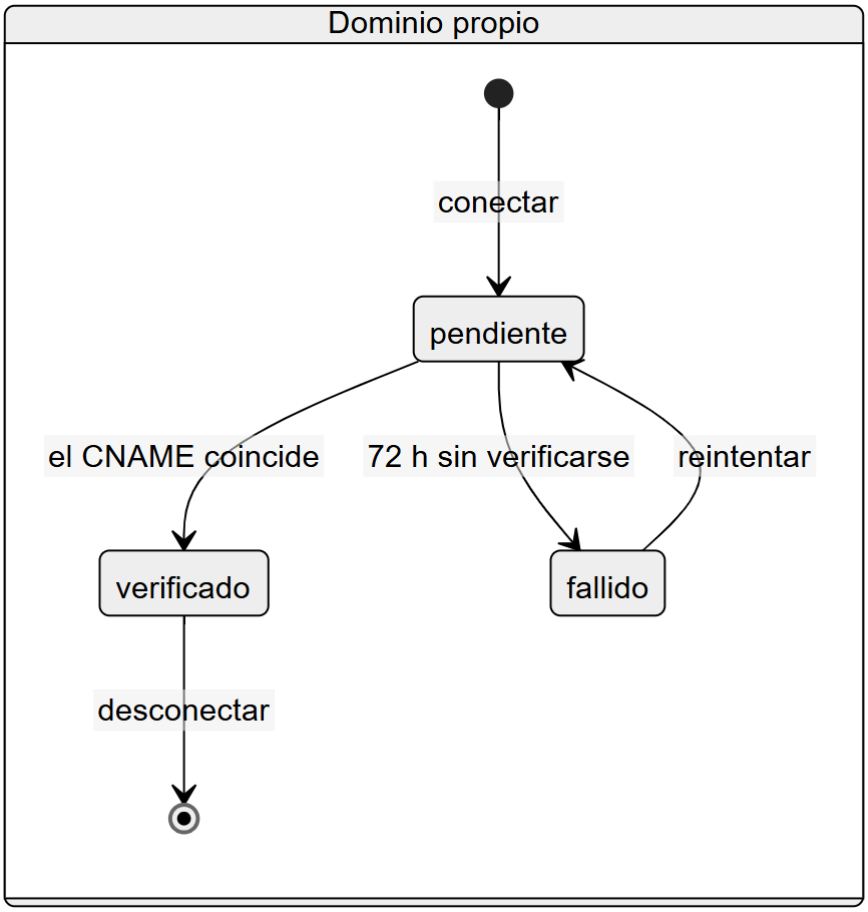


Diagrama 20. Dominio propio.

Caso de soporte

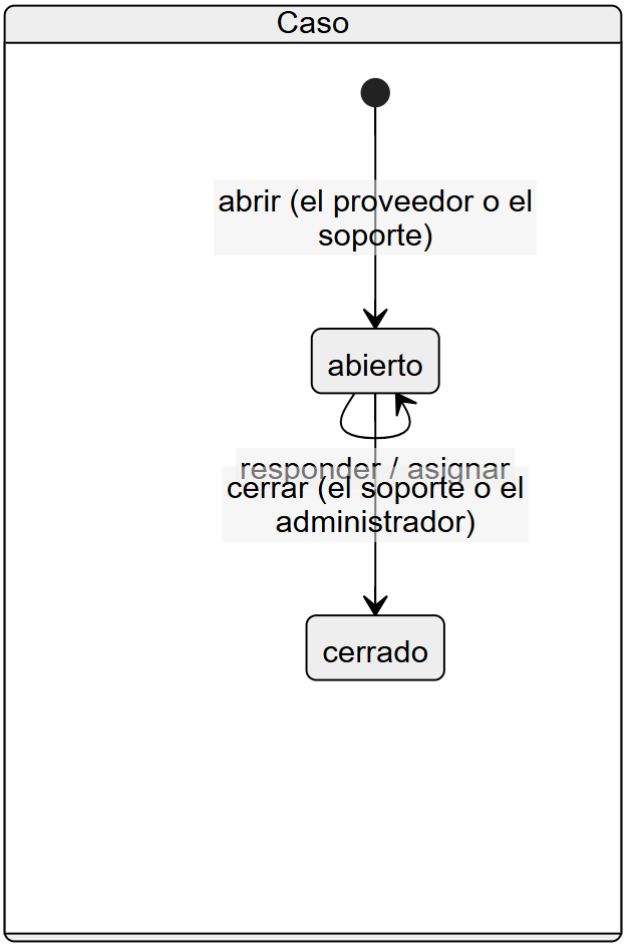


Diagrama 21. Caso de soporte.

10.2 Diagramas de flujo

10.2.1 Tubería de filtros de la compuerta

El recorrido de cada petición por la compuerta, con el error que devuelve cada filtro (capítulo 6.5).

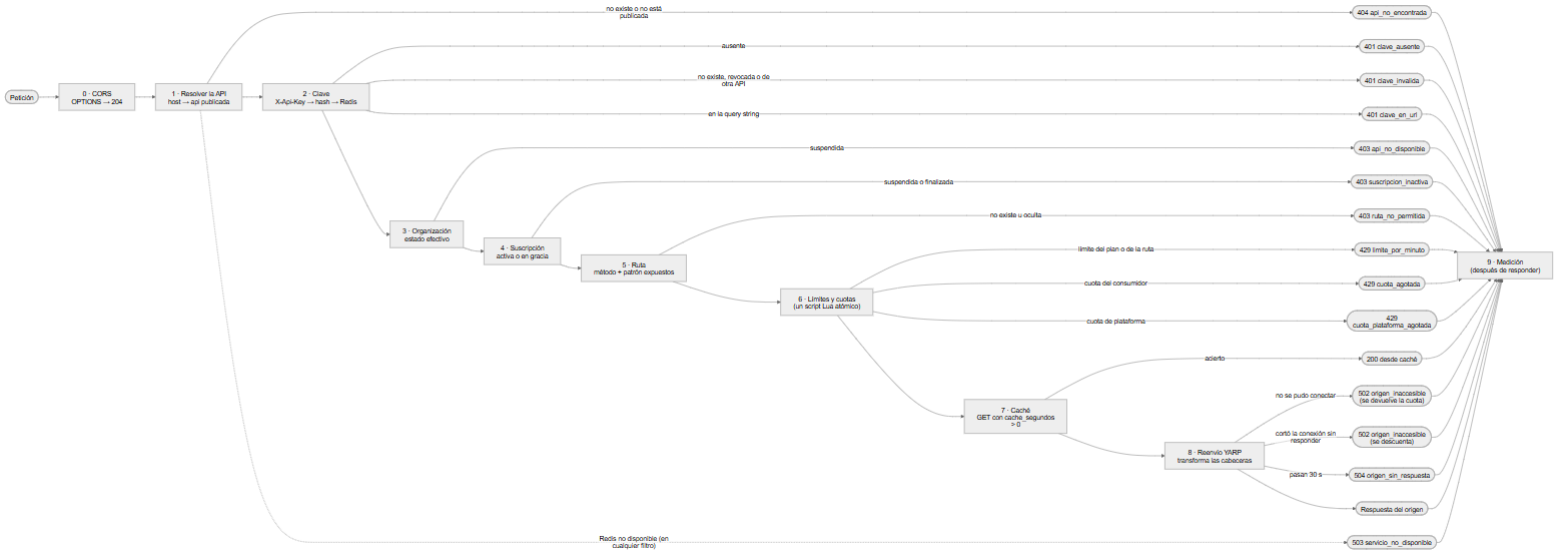


Diagrama 22. Tubería de filtros de la compuerta.

10.2.2 Publicar y cobrar una API (proveedor)

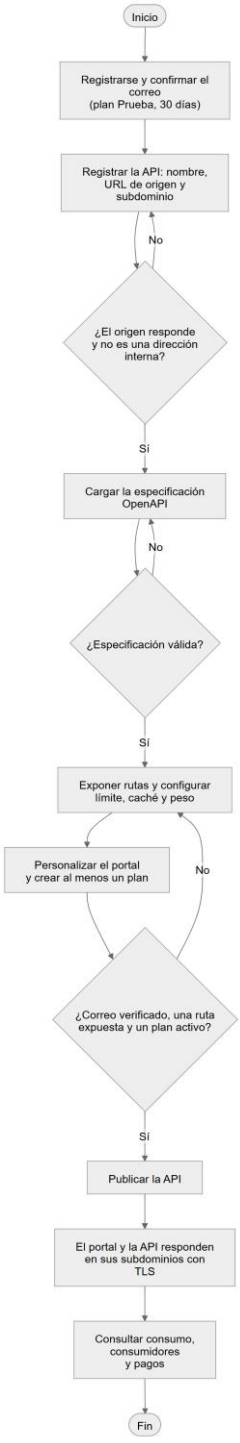


Diagrama 23. Publicar y cobrar una API (proveedor).

10.2.3 Contratar y consumir una API (consumidor)

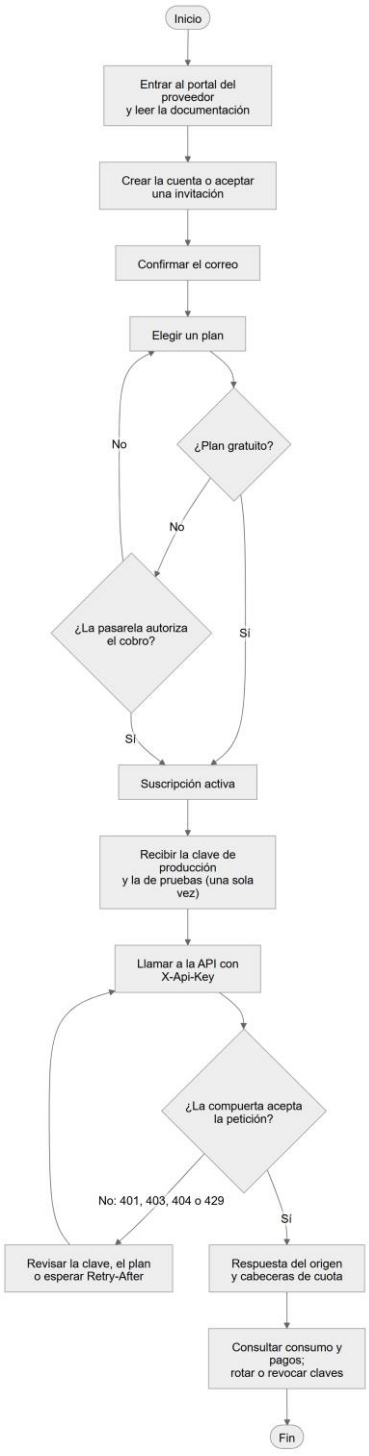


Diagrama 24. Contratar y consumir una API (consumidor).

10.2.4 Cierre de ciclo de una suscripción

Lo que hace el trabajador cada minuto con las suscripciones que llegan al fin de su ciclo (RF-22, RF-23 y RF-44).

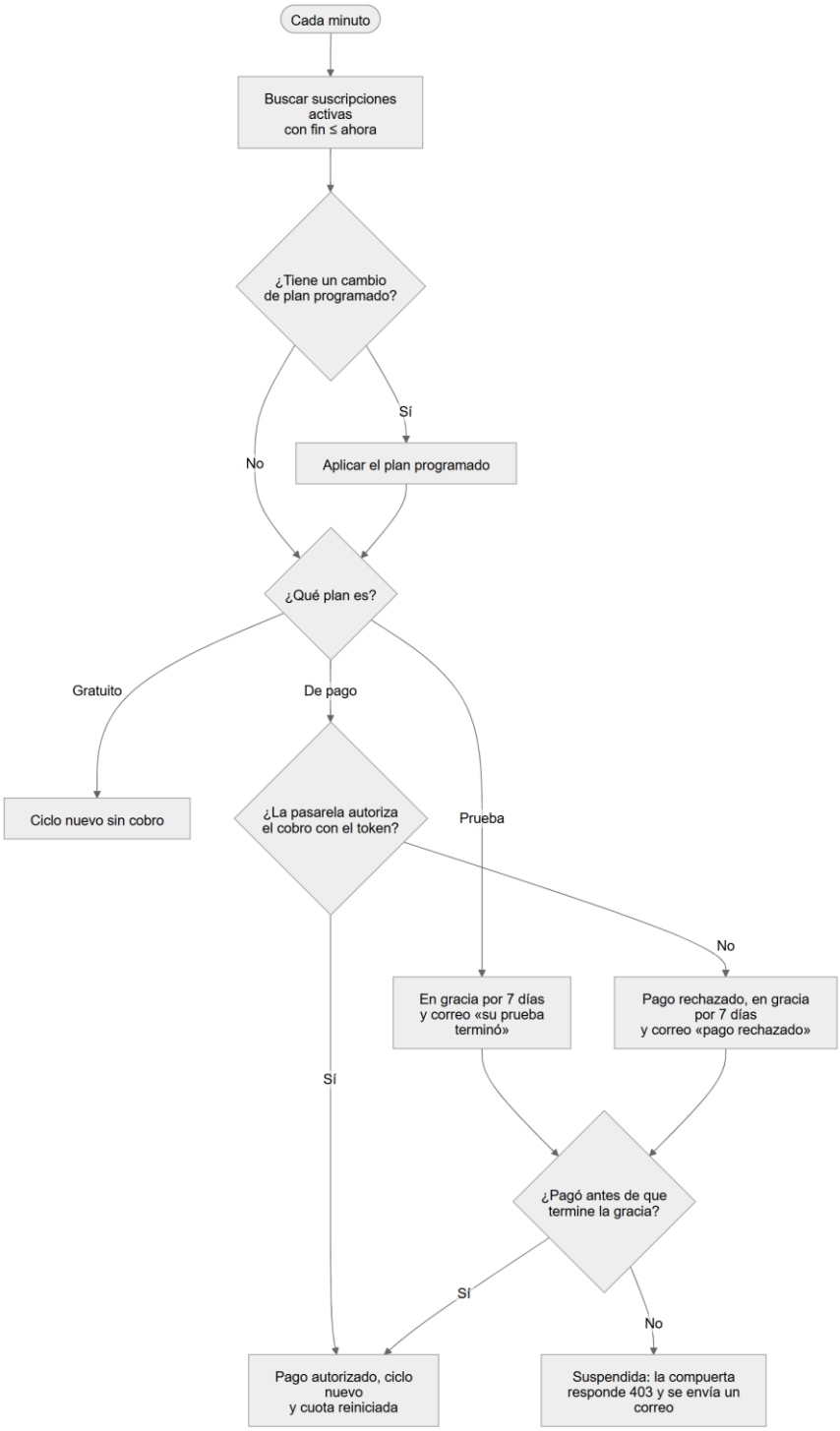


Diagrama 25. Cierre de ciclo de una suscripción.

10.3 Diagramas de bases de datos

La base de datos es PostgreSQL 16 y su esquema lo crean las migraciones de Entity Framework Core (`src/Shapi.Infraestructura/Persistencia/Migraciones`). Las tablas y columnas usan nombres en español y en `snake_case`.

10.3.1 Modelo entidad-relación

Las entidades del negocio y sus relaciones, con la notación de pata de gallo: `||` es «exactamente uno», `o|` «cero o uno», `|{` «uno o más» y `o{` «cero o más». Los atributos de cada entidad están en el modelo relacional (10.3.2) y en `docs/specs/07-modelo-de-datos.md` §2.

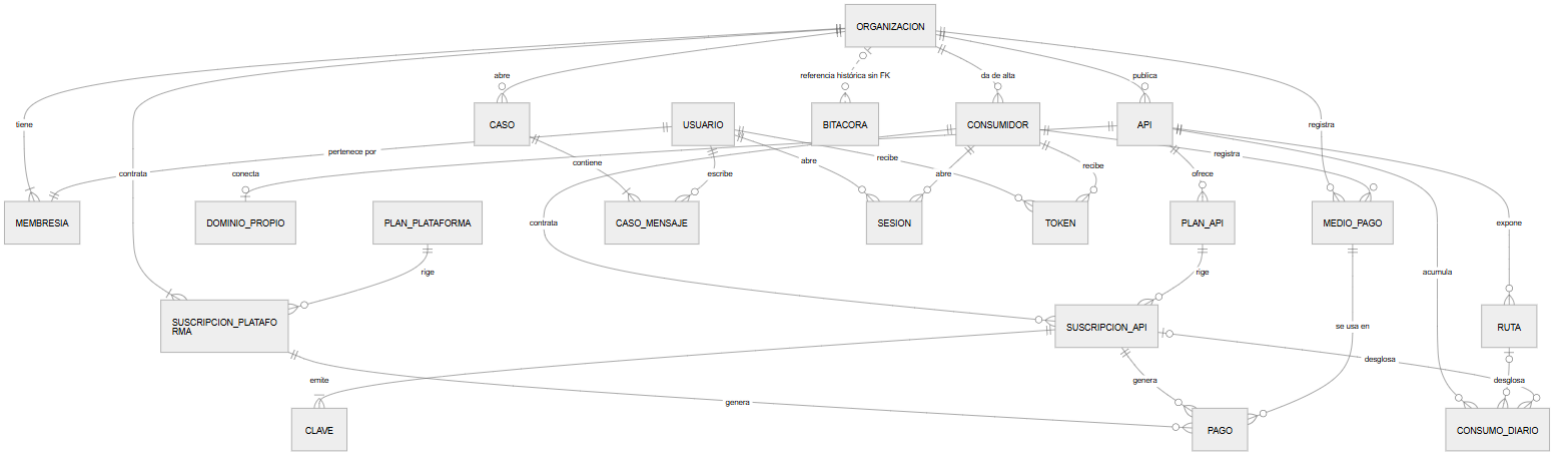


Diagrama 26. Modelo entidad-relación.

10.3.2 Modelo relacional

El modelo relacional tiene **23 tablas**. Cada diagrama muestra un área, con todas las columnas de sus tablas, su tipo en PostgreSQL y sus claves: **PK** clave primaria, **FK** clave foránea y **UK** valor único. Las tablas de otras áreas que se referencian aparecen solo con su clave primaria. La etiqueta de cada relación es la columna que la implementa.

a) Identidad y acceso

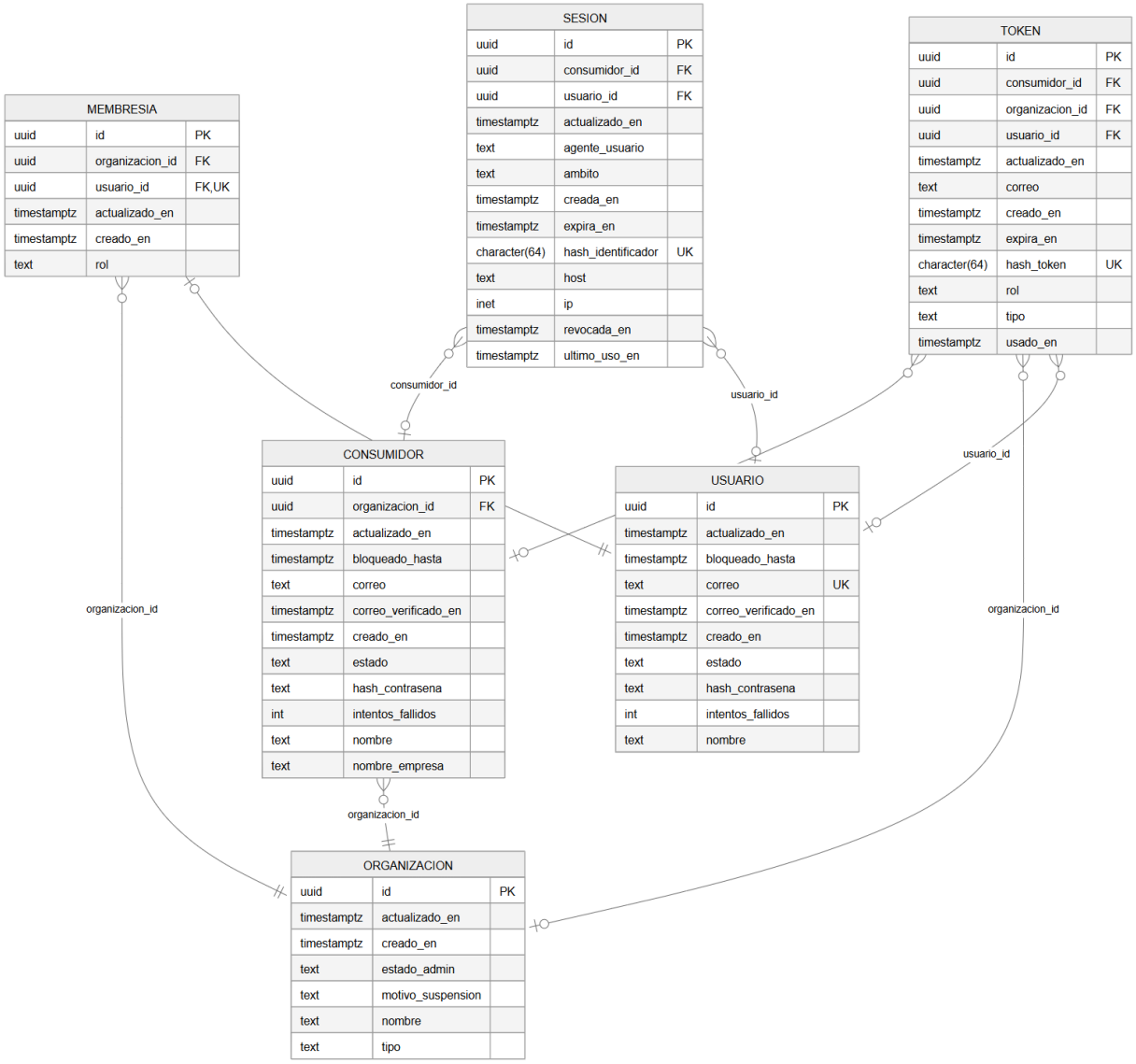


Diagrama 27. a) Identidad y acceso.

b) APIs

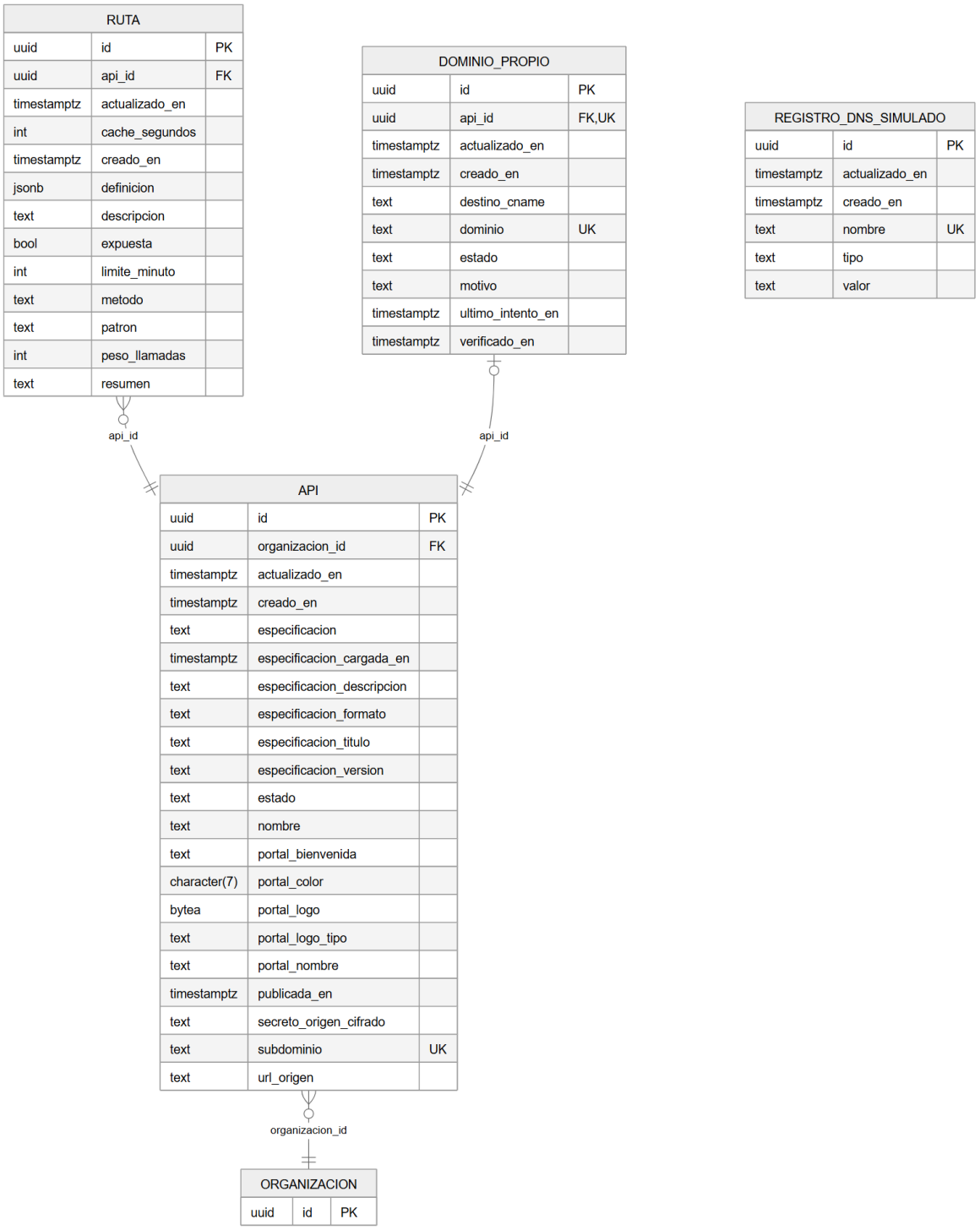


Diagrama 28. b) APIs.

c) Planes, suscripciones y claves

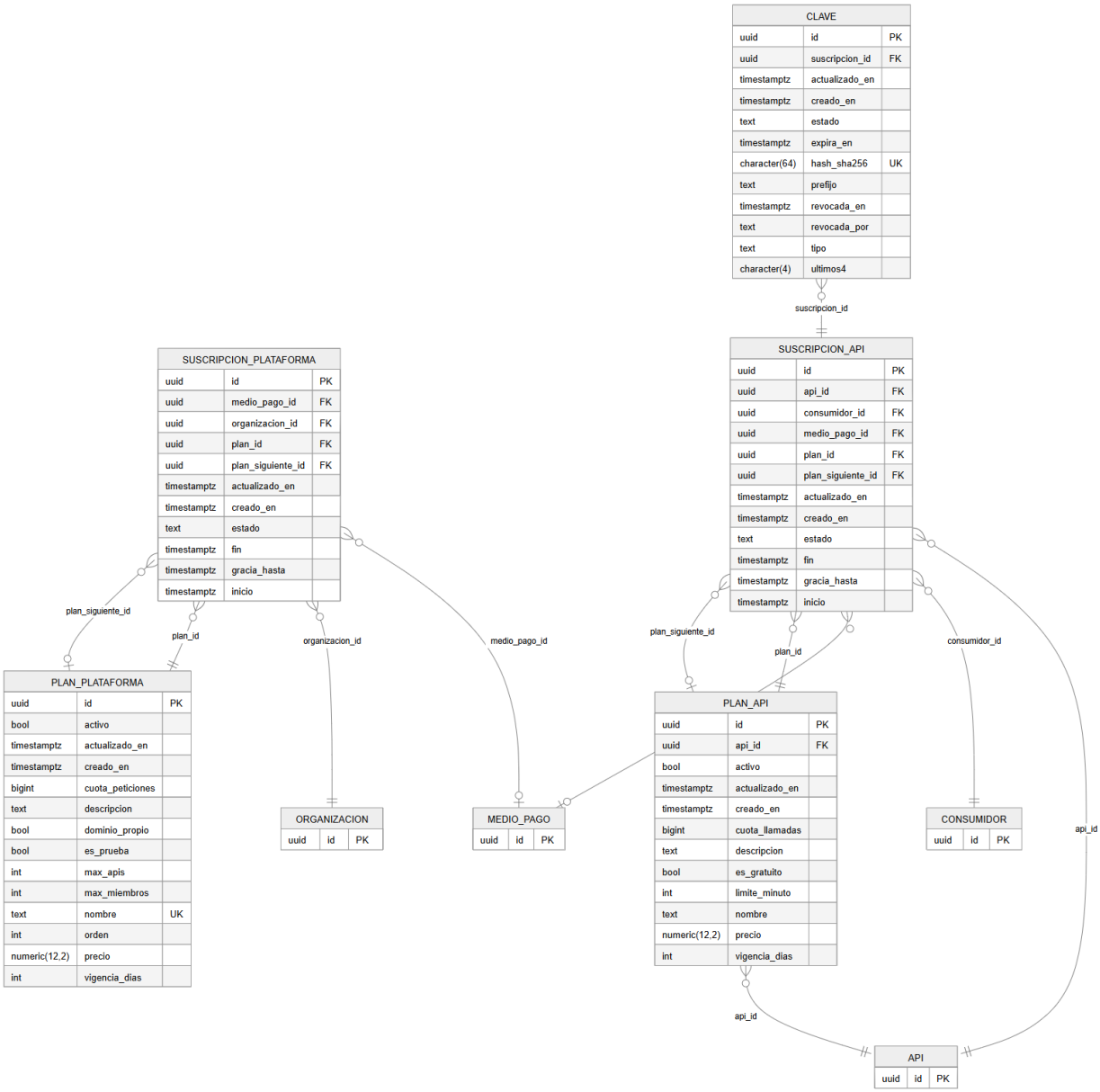


Diagrama 29. c) Planes, suscripciones y claves.

d) Pagos

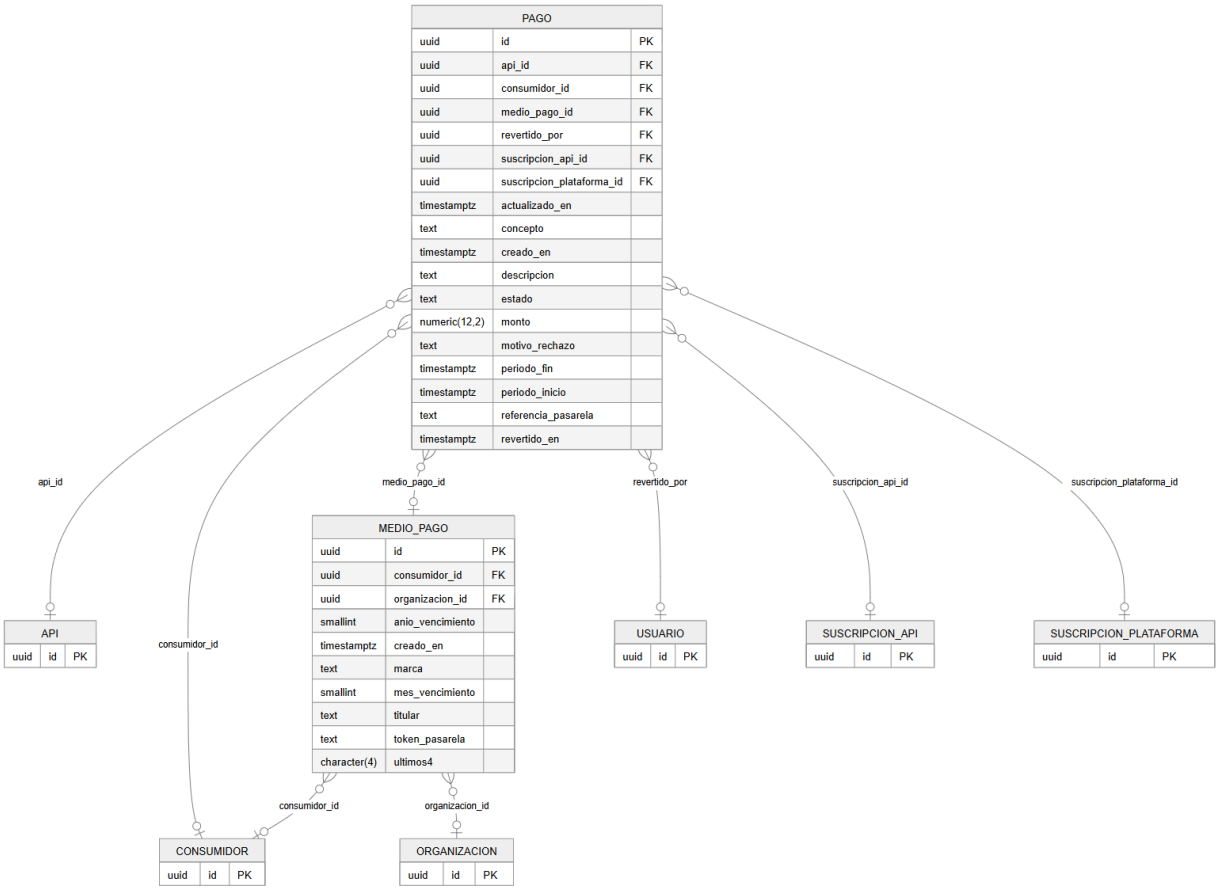


Diagrama 30. d) Pagos.

e) Consumo

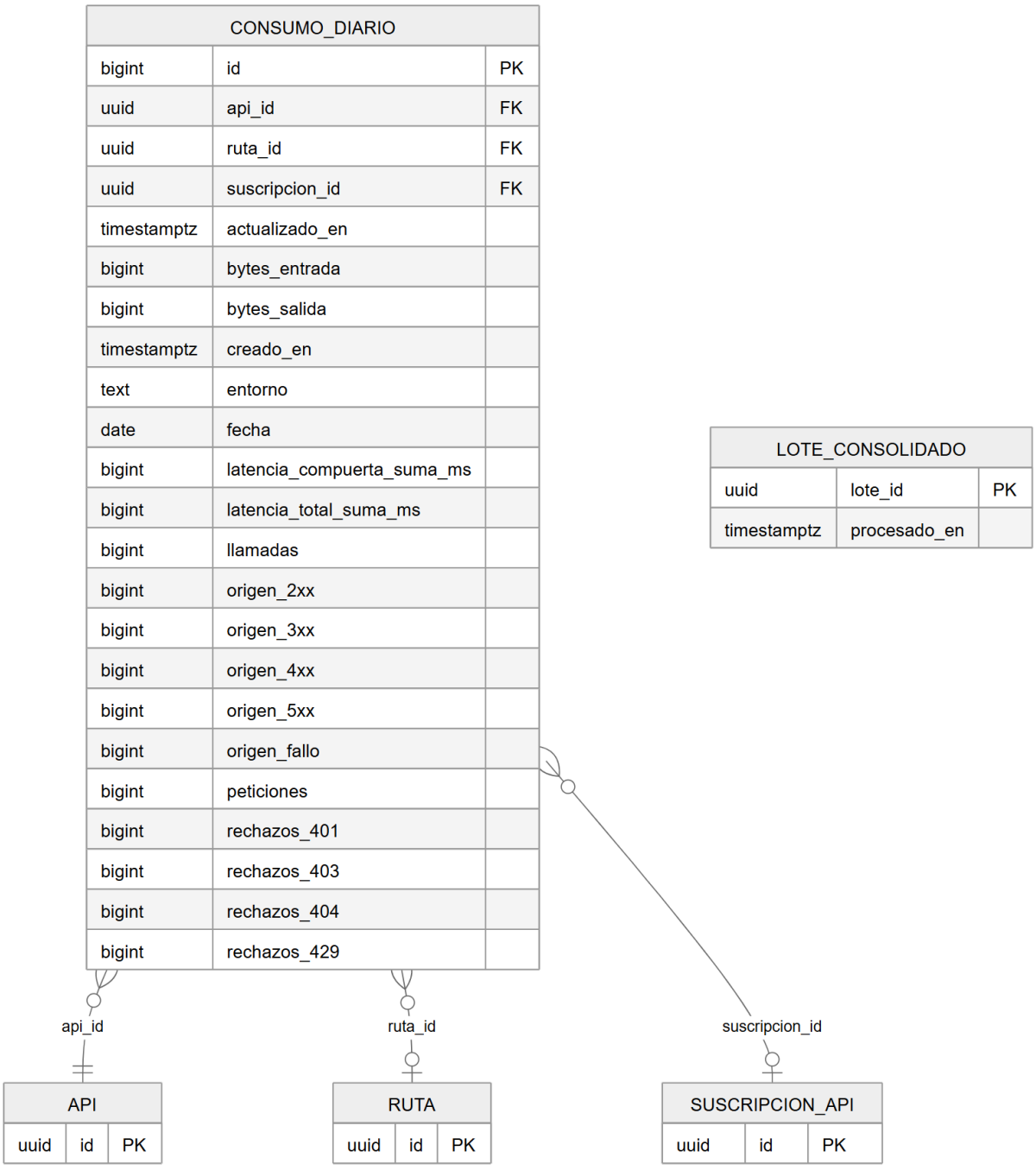


Diagrama 31. e) Consumo.

f) Soporte, auditoría y correo

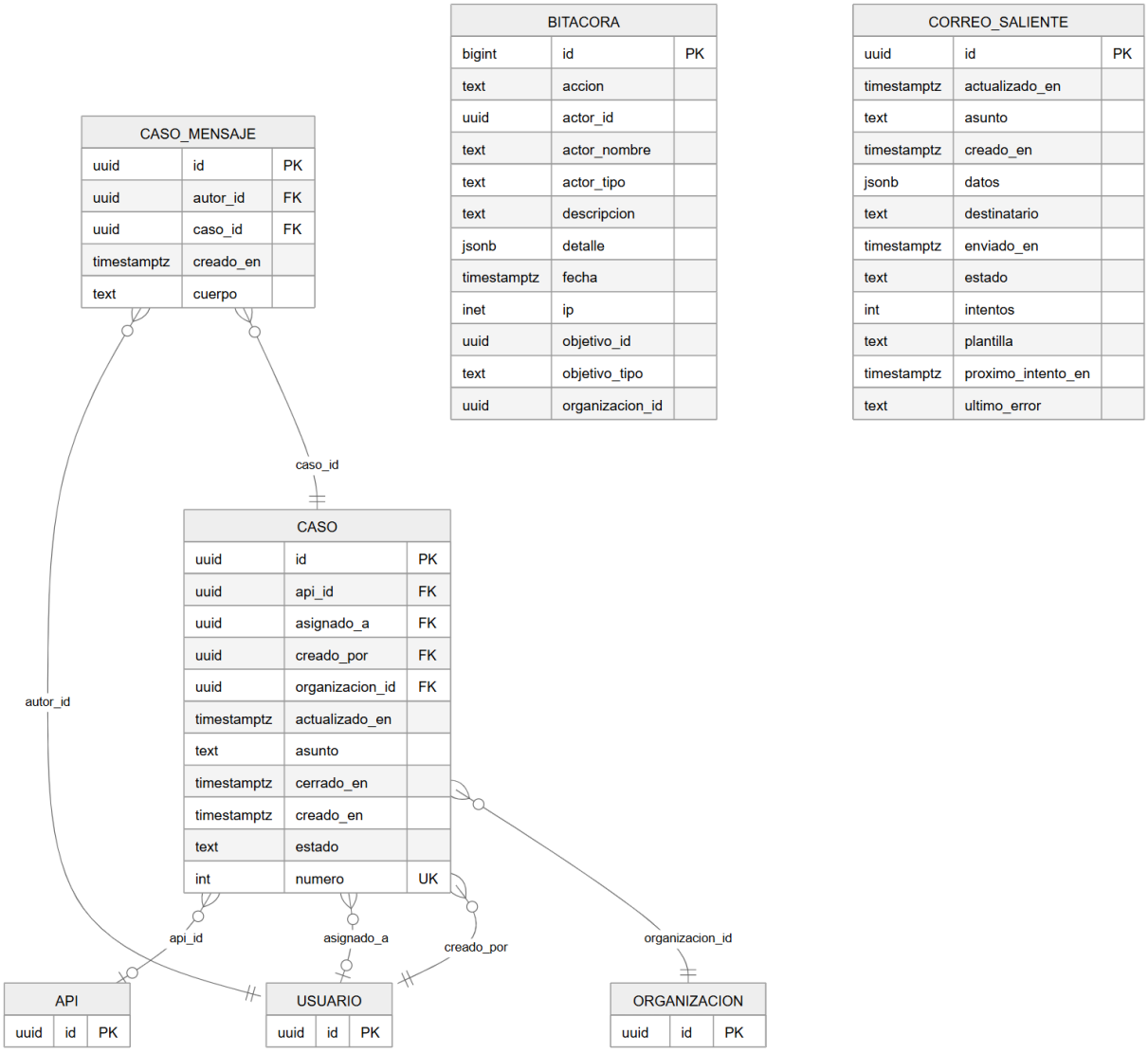


Diagrama 32. f) Soporte, auditoría y correo.

Resumen de tablas y claves

Tabla	Área	Clave primaria	Claves foráneas
`organizacion`	Identidad y acceso	`id`	—
`usuario`	Identidad y acceso	`id`	—
`membresia`	Identidad y acceso	`id`	`organizacion_id` → `organizacion`; `usuario_id` → `usuario`
`consumidor`	Identidad y acceso	`id`	`organizacion_id` → `organizacion`
`token`	Identidad y acceso	`id`	`consumidor_id` → `consumidor`; `organizacion_id` → `organizacion`; `usuario_id` → `usuario`
`sesion`	Identidad y acceso	`id`	`consumidor_id` → `consumidor`; `usuario_id` → `usuario`
`api`	APIs	`id`	`organizacion_id` → `organizacion`
`ruta`	APIs	`id`	`api_id` → `api`
`dominio_propio`	APIs	`id`	`api_id` → `api`
`registro_dns_simulado`	APIs	`id`	—

Tabla	Área	Clave primaria	Claves foráneas
plan_plataforma	Planes, suscripciones y claves	id	—
plan_api	Planes, suscripciones y claves	id	api_id → api
suscripcion_plataforma	Planes, suscripciones y claves	id	medio_pago_id → medio_pago; organizacion_id → organizacion; plan_id → plan_plataforma; plan_siguiente_id → plan_plataforma
suscripcion_api	Planes, suscripciones y claves	id	api_id → api; consumidor_id → consumidor; medio_pago_id → medio_pago; plan_id → plan_api; plan_siguiente_id → plan_api
clave	Planes, suscripciones y claves	id	suscripcion_id → suscripcion_api
medio_pago	Pagos	id	consumidor_id → consumidor; organizacion_id → organizacion
pago	Pagos	id	api_id → api; consumidor_id → consumidor; medio_pago_id → medio_pago; revertido_por → usuario; suscripcion_api_id → suscripcion_api; suscripcion_plataforma_id → suscripcion_plataforma
consumo_diario	Consumo	id	api_id → api; ruta_id → ruta; suscripcion_id → suscripcion_api
lote_consolidado	Consumo	lote_id	—
caso	Soporte, auditoría y correo	id	api_id → api; asignado_a → usuario; creado_por → usuario; organizacion_id → organizacion
caso_mensaje	Soporte, auditoría y correo	id	autor_id → usuario; caso_id → caso
bitacora	Soporte, auditoría y correo	id	—
correo_saliente	Soporte, auditoría y correo	id	—

La tabla bitacora guarda la organización y el actor como referencias históricas **sin clave foránea**, para que el registro se conserve aunque la organización cambie, y dos disparadores de la base de datos rechazan cualquier UPDATE, DELETE o TRUNCATE sobre ella (RF-41).

11. Tecnologías utilizadas

Todas las tecnologías son gratuitas y de código abierto, porque el presupuesto del proyecto es Q 0.00. Las versiones son las que fijan los archivos del repositorio (`Directory.Packages.props`, `frontend/package.json`, los `Dockerfile` e `infra/compose.yml`) al 1 de noviembre de 2026.

11.1 Backend

Tecnología	Versión	Por qué se eligió	Dónde se usa
.NET con ASP.NET Core (Minimal APIs)	10 (SDK 10.0)	Todo el backend queda en un solo lenguaje, C#. Trae de fábrica inyección de dependencias, autenticación, limitación de peticiones y servicios en segundo plano. Es la versión con soporte de largo plazo	API de control, compuerta y trabajador
YARP (`Yarp.ReverseProxy`)	2.3.0	Biblioteca de Microsoft para construir proxies inversos. Resuelve el reenvío al origen, las cabeceras y el cambio de rutas sin reiniciar; sobre ella se monta la tubería de filtros	Compuerta
Entity Framework Core + Npgsql	10.0.12 y 10.0.3	Traduce las clases del dominio a tablas, maneja las migraciones y aplica el filtro global por organización (RNF-08)	Capa de acceso a datos
EFCore.NamingConventions	10.0.1	Nombres de tablas y columnas en `snake_case`, como en el diseño físico	Capa de acceso a datos
StackExchange.Redis	3.3.1	Cliente de Redis con *pipelines* y scripts Lua, para resolver cada petición sin consultar PostgreSQL	Compuerta, API y trabajador
Microsoft.OpenApi (+ YamlReader)	2.12.2	Lee y valida las especificaciones OpenAPI 3.0 y 3.1 en JSON o YAML y extrae sus rutas	Publicación de APIs
Microsoft.Extensions.Identity.Core	10.0.12	El `PasswordHasher` de ASP.NET Core Identity (PBKDF2), sin el resto de Identity	Contraseñas
ASP.NET Data Protection	10	Cifra el secreto de origen; la API y el trabajador comparten el anillo de llaves	Secreto de origen
FluentValidation	12.1.1	Reglas de validación de las solicitudes, separadas de los endpoints	API de control
MailKit	4.18.0	Cliente SMTP con soporte de TLS	Envío de correos
DnsClient.NET	1.8.0	Consulta del DNS público, para el modo `SHAPI_DNS_MODO=real`	Dominios propios

11.2 Frontend

Tecnología	Versión	Por qué se eligió	Dónde se usa
React	19.3	Componentes reutilizables para el panel y el portal	Frontends `panel` y `portal`
TypeScript	5.6	Tipos estáticos; los tipos de la API se generan de los contratos OpenAPI	Todo el frontend
Vite	8.3	Servidor de desarrollo con recarga en caliente y compilación rápida	Desarrollo y compilación
React Router	8.4	Rutas del panel, de la administración y del portal	Navegación
TanStack Query	5.103	Caché y estados de carga y error de las consultas a la API	Llamadas a la API
Tailwind CSS	4.3	Utilidades CSS con los tokens de la variante 4; la marca de cada portal se aplica con variables CSS	Estilos
Recharts	3.10	Gráficas de consumo y latencia	B1.1 y B2.1

Tecnología	Versión	Por qué se eligió	Dónde se usa
openapi-fetch y openapi-typescript	0.17 y 7.13	Cliente HTTP tipado a partir de `contratos/openapi/`	Llamadas a la API
react-markdown y rehype-sanitize	10.1 y 6.0	Muestran las descripciones de la especificación del proveedor sin permitir HTML peligroso	Documentación del portal
pnpm	11.0	Administrador de paquetes con *workspaces* (panel, portal y paquetes compartidos)	Todo el frontend
Node.js	24 LTS	Entorno de ejecución de las herramientas del frontend y de los scripts del plan	Desarrollo, CI y compilación

11.3 Datos, borde e infraestructura

Tecnología	Versión (imagen)	Por qué se eligió	Dónde se usa
PostgreSQL	16 (`postgres:16-alpine`)	Relacional, transaccional y gratuito. Tiene `NULLS NOT DISTINCT`, arreglos para los histogramas y `jsonb`	Fuente de verdad de los datos de negocio
Redis	7.4 (`redis:7.4-alpine`), AOF `everysec`	Datos en memoria con operaciones atómicas y scripts Lua, sin condiciones de carrera; persiste con AOF	Vista de lectura de la compuerta, contadores, métricas y caché
Caddy	2 (`caddy:2` y `caddy:2-alpine`)	HTTPS automático con autoridad certificadora interna y certificados bajo demanda para los dominios propios	Borde
Mailpit	1.27 (`axllent/mailpit:v1.27`)	Servidor SMTP con bandeja web, gratuito; simula el correo sin depender de terceros	Correo simulado
Docker y Docker Compose	Engine 29, Compose v2	Levantan todo el sistema con un solo comando, igual en cualquier equipo (RNF-14)	Desarrollo y ambiente productivo simulado
Imágenes base de .NET	`mcr.microsoft.com/dotnet/sdk:10.0-alpine` y `aspnet:10.0-alpine` (los orígenes de demostración usan las variantes sin `-alpine`)	Compilación en varias etapas: la imagen final solo lleva el ejecutable	Imágenes de la API, la compuerta y el trabajador
GitHub Actions y GHCR	—	Integración continua y registro de imágenes, gratis en repositorios públicos	CI y publicación de imágenes

11.4 Pruebas

Tecnología	Versión	Uso
xUnit	2.9.3	Pruebas del dominio, de la API y de la compuerta
Testcontainers	4.15.0	Levanta PostgreSQL y Redis reales en contenedores para las pruebas de integración
FluentAssertions y NSubstitute	7.2.2 y 6.2.0	Aserciones legibles y dobles de prueba
Microsoft.AspNetCore.Mvc.Testing	10.0.12	Levanta la API completa en memoria (`WebApplicationFactory`)
Vitest, Testing Library y MSW	5.0, 16.3 y 2.15	Pruebas del frontend, con la API simulada
Playwright	1.55	Pruebas de extremo a extremo contra el ambiente productivo simulado y capturas de pantalla
k6	1.6.1 (`grafana/k6`)	Pruebas de carga de RNF-01 y RNF-03

11.5 Herramientas del equipo

Herramienta	Uso
Git y GitHub	Control de versiones, *pull requests*, revisión y protección de `main`
GitHub CLI (`gh`)	Crear e integrar *pull requests* y consultar la CI desde la terminal
Agentes de IA (Claude Code, Codex y otros)	Implementación de las tareas según el protocolo (capítulo 7)
Mermaid	Diagramas de las especificaciones y de este manual, versionados como texto
Visual Studio Code	Editor del equipo

12. Manual de instalación

Shapi se puede instalar de dos maneras:

- **Ambiente productivo simulado** (12.4): todo el sistema corre en contenedores, en configuración `Release`, con un solo comando. Es la forma recomendada para usar o presentar el sistema, y la que se usa en la exposición.
- **Entorno de desarrollo** (12.8): la infraestructura corre en contenedores y los procesos .NET y Vite corren en el equipo, con recarga en caliente. Es la forma de trabajar en el código.

Además, el ambiente productivo simulado se puede publicar en internet en una máquina virtual de Azure (12.11). Así se hizo para la presentación final, y sus comandos se ejecutaron en esa máquina el 9 de noviembre de 2026.

Los comandos de este capítulo y del 14 se ejecutaron en Windows 10 con Docker Desktop el 1 de noviembre de 2026, en un proyecto de Compose aislado para no tocar los datos de otros ambientes. Los exclusivos de Linux (instalación de paquetes y del certificado con `apt`, `update-ca-certificates`, `certutil` y `systemctl`) vienen de la versión anterior del manual y no se repitieron en esa verificación; los de Docker Compose también se ejecutan en Linux en cada corrida de la integración continua.

12.1 Requisitos del equipo

Recurso	Mínimo	Recomendado
Procesador	64 bits, 2 núcleos	4 núcleos o más
Memoria	8 GB (4 GB disponibles para Docker)	16 GB
Disco libre	15 GB (imágenes, volúmenes y dependencias)	30 GB
Sistema operativo	Windows 10 u 11 de 64 bits con WSL 2, Ubuntu o Debian recientes, o macOS	—
Navegador	Versión actual de Chrome, Edge o Firefox de escritorio, desde 1280 px de ancho (RNF-12)	—

Herramienta	Para qué	Windows (PowerShell)	Linux (Ubuntu o Debian)
Git	Obtener el código	`winget install --id Git.Git -e`	`sudo apt install -y git`
Docker con Compose v2	Ambos modos	`wsl --install` y luego `winget install --id Docker.DockerDesktop -e`	Docker Engine con el complemento `compose`, y `sudo usermod -aG docker $USER`
Node.js 24 LTS	Verificadores y frontend	`winget install --id OpenJS.NodeJS.LTS -e`	`nvm install 24`
.NET 10 SDK	Solo desarrollo	`winget install --id Microsoft.DotNet.SDK.10 -e`	`sudo apt install -y dotnet-sdk-10.0`
pnpm	Solo desarrollo	`npm install -g pnpm@latest`	`npm install -g pnpm@latest`
GitHub CLI (opcional)	Clonar y trabajar con PR	`winget install --id GitHub.cli -e`	Repositorio apt oficial de `gh`

Después de instalar, cierre y vuelva a abrir la terminal para que se actualice el `PATH`.

12.2 Obtener el código

```
git clone https://github.com/jordin-garcia/shapi.git
cd shapi
node scripts/verificar-entorno.mjs
```

Con GitHub CLI, el primer comando también puede ser `gh repo clone jordin-garcia/shapi`. El verificador comprueba las herramientas instaladas y dice qué falta; todo debe aparecer con ✅.

12.3 Configuración

Toda la configuración está en variables de entorno, en un archivo `.env` en la raíz del repositorio. Se crea a partir del ejemplo:

```
Copy-Item .env.example .env
```

En Linux o macOS, use `cp .env.example .env`. Los valores del ejemplo son solo para un equipo local. El archivo `.env` nunca se sube al repositorio.

Todos los comandos de Compose pasan `--env-file .env`. Sin esa opción, Compose busca el `.env` en `infra/`, la carpeta de `compose.yml`, e ignora el de la raíz; si falta el archivo, Compose se detiene con `couldn't find env file`.

Antes de levantar el sistema, revise estas variables (la lista completa, con su descripción y su valor por defecto, está en el **Anexo A**):

- `SHAPI_POSTGRES_CONTRASENA` y `SHAPI_ADMIN_CONTRASENA`: cámbielas fuera de un equipo de pruebas.
- `SHAPI_ADMIN_CORREO`, `SHAPI_ADMIN_NOMBRE` y `SHAPI_ADMIN_CONTRASENA`: crean la primera cuenta de administrador al iniciar la API, porque el personal de la plataforma no tiene registro público.
- `SHAPI_POSTGRES_PUERTO`: si otro PostgreSQL ya ocupa el puerto 5432 del equipo, cámbielo (por ejemplo, a `5433`) y ponga el mismo número en `Port=` de `SHAPI_POSTGRES_CADENA`.
- `SHAPI_MODO_DEMO`: `true` para la demostración (habilita la siembra y el reloj de demostración); `false` en un despliegue real.

12.4 Ambiente productivo simulado

Este ambiente ejecuta la API de control, la compuerta, el trabajador, el borde con los dos frontends compilados, PostgreSQL, Redis, Mailpit y los dos orígenes de demostración, todos en contenedores, con `ASPNETCORE_ENVIRONMENT=Production`, reinicio automático y verificaciones de salud. Solo el borde publica puertos (80 y 443), y solo en `127.0.0.1`.

Levantar con los datos de demostración (un solo comando)

Desde la raíz del repositorio, con el `.env` preparado (RNF-14):

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml --profile demo up -d
--build
```

La opción `--build` compila las imágenes en el equipo a partir del código. La primera vez tarda varios minutos; con la caché de Docker, menos de un minuto. El perfil `demo` agrega el servicio `siembra-demo`, de una sola ejecución: espera a que la API esté sana (con las migraciones aplicadas), carga los datos de los mockups y termina.

`up -d` devuelve el control antes de que termine la siembra. Compruebe que terminó con código 0:

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml --profile demo ps -a
siembra-demo
```

Después verifique el ambiente completo:

```
node infra/verificar.mjs --demo
```

El verificador comprueba la configuración, la salud de los contenedores, que solo el borde publique puertos, HTTPS y sus cabeceras de seguridad, el correo, la compuerta con una clave fija de la siembra y el portal de Envíos Xelajú. También calienta la compuerta con 90 cotizaciones de pruebas, que no descuentan cuotas de producción. Tarda unos dos minutos. Repetir el comando de arranque no duplica datos.

Usar las imágenes publicadas en GHCR

La integración continua publica, en cada integración a `main`, las imágenes `ghcr.io/jordin-garcia/shapi-api`, `shapi-compuerta`, `shapi-trabajador` y `shapi-borde`, con la etiqueta `latest`. Para usarlas en lugar de compilar, descárguelas y levante el ambiente sin `--build`:

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml pull api compuerta
trabajador borde
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml --profile demo up -d
```

Las imágenes son públicas: no hace falta iniciar sesión en GHCR. Los dos orígenes de demostración siempre se compilan en el equipo.

Levantar sin datos de demostración

Sin el perfil `demo` no se ejecuta la siembra y los datos existentes se conservan en los volúmenes:

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml up -d --build
node infra/verificar.mjs
```

La verificación sin opciones espera un ambiente sin datos de demostración; para un ambiente ya sembrado use `--demo`. Con `node infra/verificar.mjs --sembrar`, el verificador comprueba primero el ambiente sin siembra, luego lo siembra, lo verifica y repite la siembra para comprobar que no duplica datos; así lo ejecuta la CI. En un despliegue real se usa `SHAPI_MODO_DEMO=false`: la siembra rechaza ese modo aunque se active el perfil por error.

Contenedores y volúmenes

Servicio	Imagen	Publica	Volumen
`borde`	`ghcr.io/jordin-garcia/shapi-borde` (Caddy con los frontends compilados)	80 y 443 en `127.0.0.1`	`caddydata` (autoridad certificadora y certificados)
`api`	`ghcr.io/jordin-garcia/shapi-api`	—	`dpkeys` (llaves de Data Protection)
`compuerta`	`ghcr.io/jordin-garcia/shapi-compuerta`	—	—
`trabajador`	`ghcr.io/jordin-garcia/shapi-trabajador`	—	`dpkeys`
`siembra-demo`	La del trabajador, con el comando `sembrar-demo` (solo con el perfil `demo`)	—	`dpkeys`
`postgres`	`postgres:16-alpine`	—	`pgdata`
`redis`	`redis:7.4-alpine`, con AOF	—	`redisdata`
`mailpit`	`axllent/mailpit:v1.27`	—	—
`origen-envios` y `origen-agro`	Compiladas desde `origenes-demo/`	—	—

La API aplica las migraciones de la base de datos al iniciar. La API y el trabajador comparten el volumen `dpkeys`, que conserva las llaves con las que se cifran los secretos de origen: **sin ese volumen, los secretos guardados no se pueden descifrar** (capítulo 14). El trabajador no expone HTTP y aparece como `running`, sin estado de salud; su latido queda en Redis.

El borde comparte el volumen `caddydata` con el entorno de desarrollo, así que usa la misma autoridad certificadora. Caddy corre sin privilegios, con el usuario `caddy`.

Probar la API

Con el ambiente sembrado, esta llamada usa la clave de pruebas de Mercadito Antigua, que no descuenta cuota. En PowerShell, use `curl.exe` en lugar de `curl`:

```
curl https://envios.api.shapi.localhost/cotizaciones -H "X-Api-Key:
shp_prueba_Jp5sX1cV8nB3yG7tQe2Kv0a19d" -H "Content-Type: application/json" -d "@cotizacion.json"
```

El archivo `cotizacion.json` contiene `{"origen":"Quetzaltenango","destino":"Antigua Guatemala","peso_kg":2}`. La respuesta es `200` con la tarifa y las cabeceras `X-Shapi-Plan`, `X-RateLimit-*` y `X-Cuota-*`; con una clave inválida, `401` y el error `clave_invalida`.

Consultar registros

Los tres procesos .NET escriben registros estructurados en JSON:

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml logs -f
```

Puede agregar al final el nombre de un servicio, por ejemplo `api`, `compuerta` o `trabajador`. `Ctrl+C` deja de seguirlos.

Apagar

Detenga el ambiente sin borrar los datos persistentes:

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml down
```

Para volver a encenderlo, repita el comando de arranque. Los datos siguen en los volúmenes.

12.5 Confiar en la autoridad certificadora local

Caddy emite los certificados de `*.shapi.localhost` con su propia autoridad certificadora, guardada en el volumen `caddydata`. Para que el navegador los acepte, importe la raíz **una sola vez**, después de levantar el sistema por primera vez.

En Windows PowerShell:

```
docker compose --env-file .env -f infra/compose.yml cp
borde:/data/caddy/pki/authorities/local/root.crt .\caddy-root.crt
Import-Certificate -FilePath .\caddy-root.crt -CertStoreLocation Cert:\CurrentUser\Root
Remove-Item .\caddy-root.crt
```

Windows pide confirmar la instalación del certificado. Chrome y Edge usan el almacén de certificados de Windows. En Firefox, active `security.enterprise_roots.enabled` en `about:config`.

En Linux:

```
docker compose --env-file .env -f infra/compose.yml cp
borde:/data/caddy/pki/authorities/local/root.crt ./caddy-root.crt
sudo cp caddy-root.crt /usr/local/share/ca-certificates/caddy-shapi.crt
sudo update-ca-certificates
sudo apt install -y libnss3-tools
certutil -d sql:$HOME/.pki/nssdb -A -t "C,," -n caddy-shapi -i caddy-root.crt
rm caddy-root.crt
```

Firefox permite importar `caddy-root.crt` desde **Ajustes → Privacidad y seguridad → Certificados → Ver certificados → Autoridades**.

Con la raíz instalada, https://correo.shapi.localhost abre Mailpit con un certificado confiable. Si se borra el volumen `caddydata`, Caddy crea otra autoridad y hay que importarla de nuevo.

12.6 Siembra y cuentas de demostración

La siembra de demostración reproduce los datos de los mockups: la organización de la plataforma, cinco organizaciones proveedoras, dos APIs publicadas (Envíos Xelajú y Agro Precios), sus planes, consumidores, suscripciones, claves, pagos, casos y consumo. Exige `SHAPI_MODO_DEMO=true` y es idempotente. Al terminar inicializa las cuotas de Redis con el consumo de los ciclos sembrados y resincroniza la configuración de la compuerta.

Todas las cuentas siguientes usan la contraseña `Shapi2026!demo`:

Área	Nombre	Correo
Administración	Rodrigo Alvarado	rodrigo.alvarado@shapi.localhost
Administración	Lucía Ramírez Pineda	lucia.ramirez@shapi.localhost
Soporte	Sofía Menchú Cojtí	sofia.menchu@shapi.localhost
Soporte, desactivada	Julio Estrada Ixcot	julio.estrada@shapi.localhost
Envíos Xelajú	Ana Lucía Morales	ana.morales@enviosxelaju.com
Envíos Xelajú	Diego Us Pérez	diego.us@enviosxelaju.com
Envíos Xelajú	Karla Batres	karla.batres@enviosxelaju.com
Portal de Envíos Xelajú	María José Quiñónez, Mercadito Antigua	mariajose@mercaditoantigua.com
Portal de Envíos Xelajú	Isabel Herrera, Boutique Cayalá	isabel@boutiquecayala.com
Portal de Envíos Xelajú	Óscar Méndez, Ferretería Zona 11	oscar@ferreteriazona11.com
Portal de Envíos Xelajú	Rosa Choc, Tienda Sololá	rosa@tiendasolola.com
Agro Precios	Carlos Tzul	carlos.tzul@agroprecios.com
Portal de Agro Precios	Andrea Xiloj, Distribuidora San Lucas	andrea@distribuidorasl.com
Cafetalera del Altiplano	Lucía Cotom	lucia.cotom@cafetaleraaltiplano.com
Transportes Petén	Mario Pop	mario.pop@transportespeten.com
Datos Chapines	Gabriela Sac	gabriela.sac@datoschapines.com

Claves fijas para llamadas de demostración:

Portal y consumidor	Entorno	Clave
Envíos Xelajú · Mercadito Antigua	Producción	shp_prod_4fN8qT2xLm6Rv0Zk9Wd3Hs7c2e
Envíos Xelajú · Mercadito Antigua	Pruebas	shp_prueba_Jp5sX1cV8nB3yG7tQe2Kv0a19d
Envíos Xelajú · Boutique Cayalá	Producción	shp_prod_BoutiqueCayalaDemo00004b80
Envíos Xelajú · Boutique Cayalá	Producción, rotada hace 9 horas y vigente 15 horas más	shp_prod_BoutiqueCayalaRotada00e513
Envíos Xelajú · Boutique Cayalá	Pruebas	shp_prueba_BoutiqueCayalaDem0000031c7
Envíos Xelajú · Ferretería Zona 11	Producción	shp_prod_FerreteriaZona11Demo0090fb
Envíos Xelajú · Ferretería Zona 11	Pruebas	shp_prueba_FerreteriaZona11De00005e28
Envíos Xelajú · Tienda Sololá	Producción	shp_prod_TiendaSololaDemo000000b3a4
Envíos Xelajú · Tienda Sololá	Pruebas, revocada	shp_prueba_TiendaSololaDemo000000d6f1
Agro Precios · Distribuidora San Lucas	Producción	shp_prod_AgroPreciosDemo0000000a7f2
Agro Precios · Distribuidora San Lucas	Pruebas	shp_prueba_AgroPreciosDemo00000004c8d

Para volver a los datos iniciales de la demostración, por ejemplo antes de presentar:

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml exec trabajador
dotnet Shapi.Trabajador.dll sembrar-demo --reiniciar
```

El reinicio borra todas las organizaciones ajenas a la siembra y sus datos de ensayos (APIs, consumidores, suscripciones, pagos, casos, sesiones y enlaces), y recrea los de las cinco organizaciones de demostración. Las cuentas del personal y la bitácora se conservan, porque la bitácora solo admite inserciones. El comando rechaza `SHAPI_MODO_DEMO=false`. Si configura `SHAPI_SECRETO_ORIGEN_ENVIOS` o `SHAPI_SECRETO_ORIGEN_AGRO`, la siembra cifra esos mismos secretos para que la compuerta pueda autenticarse ante cada origen.

En el entorno de desarrollo, la siembra también se puede ejecutar directamente:

```
dotnet run --project src/Shapi.Trabajador -- sembrar-demo
dotnet run --project src/Shapi.Trabajador -- sembrar-demo --reiniciar
```

12.7 Modo demostración y reloj

Con `SHAPI_MODO_DEMO=true`, el sistema tiene un **reloj de negocio** que se puede adelantar, para mostrar en vivo la renovación de una suscripción, el periodo de gracia y la suspensión sin esperar 30 días.

1. Entre como administrador (por ejemplo, `rodrigo.alvarado@shapi.localhost`) y abra **Estado de los componentes** (B3.1).
2. Al final de la página está la tarjeta **Modo demostración**, con la fecha y hora de Guatemala del reloj de negocio.
3. Adelante 30 días, 7 días o una cantidad de 1 a 365 días. La API y el trabajador actualizan su reloj en menos de un segundo y el trabajador cierra los ciclos vencidos.

El adelanto no se puede deshacer desde la interfaz, y reiniciar la siembra no lo borra: `sembrar-demo` respeta el desplazamiento guardado. Para volver a la fecha real, borre el desplazamiento en Redis, espere unos 5 segundos (los procesos lo vuelven a leer y, si falta, usan la hora real) y reinicie la demostración (12.6):

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml exec redis redis-cli
DEL demo:reloj:desplazamiento
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml exec trabajador
dotnet Shapi.Trabajador.dll sembrar-demo --reiniciar
```

Las sesiones vencen con la hora real, independiente del reloj de negocio. Con `SHAPI_MODO_DEMO=false` la tarjeta no aparece y el endpoint responde 404.

12.8 Entorno de desarrollo

Levantar y verificar la infraestructura

Desde la raíz del repositorio:

```
docker compose --env-file .env -f infra/compose.yml up -d
node infra/verificar.mjs
```

El primer comando inicia PostgreSQL, Redis, Mailpit, Caddy y los dos orígenes de demostración. El segundo valida la configuración, la salud de los seis contenedores, que solo publiquen puertos en `127.0.0.1`, HTTPS, las cabeceras de seguridad y la bandeja de correo. Ejecute la verificación antes de iniciar .NET o Vite: el verificador ocupa temporalmente los puertos 5080, 5090, 5173 y 5174 y falla con `EADDRINUSE` si alguno ya está en uso.

Cargar las variables y ejecutar los procesos

.NET no lee el archivo `.env`: sin sus variables, la API no encuentra la base de datos (`SHAPI_POSTGRES_CADENA`). Cargue el `.env` en cada terminal antes de `dotnet run`. Los valores no van entre comillas y la cadena de conexión lleva `;`, así que no use `source .env`:

```
# PowerShell
Get-Content .env | ForEach-Object { if ($_ -match '^\s*([A-Z_][A-Z0-9_]*)=(.*)$') { Set-Item
"env:$($Matches[1])" $Matches[2] } }

# bash (Linux, macOS o Git Bash)
while IFS= read -r linea || [[ -n $linea ]]; do [[ $linea =~ ^([A-Z_][A-Z0-9_]*)=(.*)$ ]] &&
export "${BASH_REMATCH[1]}=${BASH_REMATCH[2]%$'\r'}"; done < .env
```

Después, cada proceso en su propia terminal, con las variables cargadas:

```
dotnet run --project src/Shapi.Api
dotnet run --project src/Shapi.Compuerta
dotnet run --project src/Shapi.Trabajador
cd frontend && pnpm install && pnpm dev
```

En desarrollo, la API aplica las migraciones al iniciar. El panel queda en `https://shapi.localhost` y el portal de cada API en `https://{sub}.shapi.localhost`, a través de Caddy.

En Linux con Docker Engine nativo, `host.docker.internal` apunta al puente de Docker y no alcanza procesos que escuchen solo en `localhost`. Inicie los procesos HTTP en todas las interfaces para que Caddy pueda conectarse:

```
dotnet run --project src/Shapi.Api --urls http://0.0.0.0:5080
dotnet run --project src/Shapi.Compuerta --urls http://0.0.0.0:5090
cd frontend && pnpm dev -- --host 0.0.0.0
```

La configuración de Vite usa `server.host: "0.0.0.0"` por la misma razón. Estos puertos son exclusivamente de desarrollo; no exponga este entorno en una red no confiable.

Puertos y direcciones

Servicio	Dirección local	Dirección mediante Caddy
API de control	`http://localhost:5080`	`https://shapi.localhost/api/*`
Compuerta	`http://localhost:5090`	`https://{sub}.api.shapi.localhost`
Panel	`http://localhost:5173`	`https://shapi.localhost`
Portal	`http://localhost:5174`	`https://{sub}.shapi.localhost`
PostgreSQL	`localhost:5432` (o `SHAPI_POSTGRES_PUERTO`)	—
Redis	`localhost:6379`	—
Mailpit SMTP	`localhost:1025`	—
Mailpit web	`http://localhost:8025`	`https://correo.shapi.localhost`
Origen Envíos	`http://localhost:5101`	Mediante la compuerta
Origen Agro	`http://localhost:5102`	Mediante la compuerta

Todos los puertos que publica Compose se abren solo en `127.0.0.1`: el entorno no queda expuesto a la red local. Los dominios `*.shapi.localhost` resuelven siempre a la propia máquina.

Apagar o reiniciar

Detenga los contenedores sin perder los datos:

```
docker compose --env-file .env -f infra/compose.yml down
```

Para reiniciar, ejecute `down` y luego `docker compose --env-file .env -f infra/compose.yml up -d`. Si necesita borrar también PostgreSQL, Redis y los certificados locales, ejecute:

```
docker compose --env-file .env -f infra/compose.yml down -v
```

Cuidado: `down -v` borra los volúmenes sin pedir confirmación. Haga antes un respaldo (capítulo 14) y, después, vuelva a importar la raíz de Caddy (12.5).

12.9 Pruebas

Pruebas	Comando	Requisitos
Backend completo	`dotnet test Shapi.slnx`	Docker encendido (Testcontainers levanta PostgreSQL y Redis). En Windows, si falla por errores de Docker, agregue `-m:1`
Backend de un módulo	`dotnet test tests/Shapi.Api.Tests --filter "FullyQualifiedName~Shapi.Api.Tests.Planes"`	Igual que el anterior

Pruebas	Comando	Requisitos
Formato del backend	`dotnet format Shapi.slnx --verify-no-changes`	—
Frontend	En `frontend/`: `pnpm lint`, `pnpm typecheck`, `pnpm test` y `pnpm build`	`pnpm install`
Extremo a extremo	En `tests/e2e/`: `pnpm install`, `pnpm exec playwright install chromium` (la primera vez) y `pnpm test`	El ambiente productivo simulado levantado, en `https://shapi.localhost`
Captura de una pantalla	En `tests/e2e/`: `pnpm captura <url> <archivo.png>`	—
Carga (RNF-01 y RNF-03)	Ver `tests/carga/README.md`	El ambiente productivo simulado y Docker para k6

Las pruebas de carga del 18 de octubre de 2026 cumplieron los dos requisitos. Con 200 peticiones por segundo durante 5 minutos, la compuerta completó 60 001 peticiones con **0 % de errores** (RNF-03). Su p95 adicional sobre el origen directo fue de **8.583 ms**, menor que el máximo de 15 ms (RNF-01). El detalle está en `docs/pruebas-de-carga.md`.

12.10 Integración continua y publicación

Cada *pull request* ejecuta en GitHub Actions los checks `plan`, `titulo`, `backend`, `frontend`, `revision-claude` y `ambiente-productivo` (las pruebas E2E sobre el ambiente productivo simulado). Un PR solo se integra a `main` con todos en verde. Cada integración a `main` vuelve a ejecutar todo y publica las cuatro imágenes en GHCR con la etiqueta `latest`, que son las que usa la sección «Usar las imágenes publicadas en GHCR».

12.11 Despliegue en Azure

Para la presentación final, el mismo ambiente productivo simulado se publicó en internet en `https://shapi.tech` (ADR-36). Corre en una máquina virtual de **Azure for Students** con las imágenes de GHCR, `infra/compose.yml`, `infra/compose.prod.yml` y dos archivos propios del servidor:

- `infra/caddy/Caddyfile.servidor`: certificados de Let's Encrypt y la bandeja de Mailpit protegida con contraseña.
- `infra/compose.servidor.yml`: el borde en los puertos 80 y 443 de la máquina, y el reenvío de Mailpit a Resend.

Ambos tienen el dominio `shapi.tech` escrito. Para otro dominio, cambie `shapi.tech` en los dos archivos y en el `.env`. Los comandos de esta sección se ejecutan en Git Bash o en Linux, salvo donde se indica.

Requisitos

- Una suscripción de Azure. En Azure for Students solo se pueden crear recursos en las regiones que permite la política «Allowed resource deployment regions» de la suscripción (en la del equipo: North Central US, Canada Central, Brazil South, France Central y Norway East).
- La CLI de Azure. En Windows se instala con `winget install -e --id Microsoft.AzureCLI`, y desde Git Bash se agrega al `PATH` la carpeta `C:\Program Files\Microsoft SDKs\Azure\CLI2\wbin`.
- Un dominio cuyo DNS pueda administrar.

1. Crear la máquina virtual

Inicie sesión, elija la suscripción y cree una llave SSH solo para el servidor:

```
az login
az account set --subscription "Azure for Students"
ssh-keygen -t ed25519 -f ~/.ssh/shapi_azure -N "" -C "shapi-azure"
```

Si la suscripción es nueva y Azure dice que no hay tamaños disponibles, registre primero el proveedor de máquinas virtuales:

```
az provider register -n Microsoft.Compute --wait
```

Cree el grupo de recursos y la máquina, que tiene 2 vCPU y 8 GB, con un disco de 64 GB y una IP fija:

```
az group create -n shapi-rg -l northcentralus --tags proyecto=shapi
az vm create -g shapi-rg -n shapi-vm --image Ubuntu2404 --size Standard_B2as_v2 \
  --admin-username shapi --ssh-key-values "$(cat ~/.ssh/shapi_azure.pub)" \
  --public-ip-sku Standard --public-ip-address shapi-ip --nsg shapi-nsg --nsg-rule NONE \
  --os-disk-size-gb 64 --storage-sku StandardSSD_LRS --tags proyecto=shapi
```

Abra el firewall: el 80 y el 443 para todos y el 22 solo para su IP. Cambie `<su-ip>` por la que muestra `curl https://api.ipify.org`:

```
az network nsg rule create -g shapi-rg --nsg-name shapi-nsg -n permitir-ssh-admin --priority 100 \
  --access Allow --protocol Tcp --source-address-prefixes <su-ip>/32 --destination-port-ranges 22
az network nsg rule create -g shapi-rg --nsg-name shapi-nsg -n permitir-http --priority 110 \
  --access Allow --protocol Tcp --source-address-prefixes Internet --destination-port-ranges 80
az network nsg rule create -g shapi-rg --nsg-name shapi-nsg -n permitir-https --priority 120 \
  --access Allow --protocol Tcp --source-address-prefixes Internet --destination-port-ranges 443
az network public-ip show -g shapi-rg -n shapi-ip --query ipAddress -o tsv
```

El último comando muestra la IP pública de la máquina.

2. Apuntar el dominio

En el DNS del dominio, borre los registros de estacionamiento que haya creado el registrador y agregue tres registros A hacia la IP de la máquina, con TTL de 300:

Tipo	Nombre	Valor
A	`shapi.tech` (la raíz)	IP de la máquina
A	`*.shapi.tech`	IP de la máquina
A	`*.api.shapi.tech`	IP de la máquina

Antes de seguir con el paso 5, compruebe que `nslookup envios.api.shapi.tech 8.8.8.8` responda con esa IP.

3. Preparar el servidor

```
ssh -i ~/.ssh/shapi_azure shapi@<ip-de-la-maquina>
curl -fsSL https://get.docker.com -o /tmp/get-docker.sh && sudo sh /tmp/get-docker.sh
sudo usermod -aG docker shapi
exit
```

Vuelva a entrar con el mismo `ssh` para que tome el grupo `docker`, y clone el repositorio:

```
git clone https://github.com/jordin-garcia/shapi.git ~/shapi
```

4. Configurar el `.env` del servidor

En `~/shapi`, cree el `.env` con contraseñas nuevas. El bloque genera valores aleatorios, guarda el `.env` con permisos `600` y deja las contraseñas que se usan a mano en `~/credenciales-shapi.txt`:

```
cd ~/shapi
gen() { openssl rand -base64 48 | tr -dc "A-Za-z0-9" | head -c 24; }
PG=$(gen); ADMIN=$(gen); MAIL=$(gen); SE=$(gen); SA=$(gen)
HASH=$(docker run --rm caddy:2-alpine caddy hash-password --plaintext "$MAIL")
umask 077
cat > .env <<EOF
SHAPI_DOMINIO_BASE=shapi.tech
SHAPI_MODO_DEMO=true
SHAPI_APLICAR_MIGRACIONES=false
SHAPI_POSTGRES_USUARIO=shapi
SHAPI_POSTGRES_CONTRASENA=$PG
SHAPI_POSTGRES_BASE_DATOS=shapi
SHAPI_DNS_MODO=simulado
SHAPI_REDES_BORDE=
SHAPI_ADMIN_CORREO=admin@shapi.tech
SHAPI_ADMIN_NOMBRE=Administrador Shapi
SHAPI_ADMIN_CONTRASENA=$ADMIN
SHAPI_PASARELA_FALLA=false
SHAPI_SECRETO_ORIGEN_ENVIOS=$SE
SHAPI_SECRETO_ORIGEN_AGRO=$SA
SHAPI_MAILPIT_HASH='$HASH'
SHAPI_RESEND_CLAVE=
EOF
printf "Administrador: admin@shapi.tech / %s\nMailpit: equipo / %s\n" "$ADMIN" "$MAIL" >
~/credenciales-shapi.txt
chmod 600 .env ~/credenciales-shapi.txt
```

El hash de Mailpit va entre comillas simples porque contiene `$`, que Docker Compose interpretaría como una variable. La clave de Resend se completa en el paso 6. Como `compose.servidor.yml` la exige, mientras esté vacía Compose no levanta nada: haga los puntos 1 a 4 del paso 6 antes del paso 5, o quite el bloque `mailpit` de ese archivo si no quiere correo real.

5. Levantar y sembrar

Defina un atajo para los tres archivos de Compose (repítalo en cada sesión de SSH):

```
cd ~/shapi
C="docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml -f
infra/compose.servidor.yml"
```

Levante primero todo **menos el borde**, compile los orígenes de demostración y siembre:

```
$C pull api compuerta trabajador borde postgres redis mailpit
$C build origen-envios origen-agro
$C up -d postgres redis mailpit origen-envios origen-agro api compuerta trabajador
$C --profile demo up siembra-demo
```

Cuando el DNS ya resuelva (paso 2), levante el borde. Caddy pide los certificados de `shapi.tech` al arrancar, y los de cada subdominio con su primera visita:

```
$C up -d borde
$C logs borde | grep "certificate obtained"
```

El borde se levanta al final porque, si el DNS todavía no apunta a la máquina, Let's Encrypt rechaza los intentos y, tras varios fallos, bloquea el dominio por una hora.

Compruebe desde su equipo que respondan `https://shapi.tech`, `https://envios.shapi.tech`, `https://correo.shapi.tech` (pide el usuario `equipo` y la contraseña de `~/credenciales-shapi.txt`) y la compuerta:

```
curl -X POST https://envios.api.shapi.tech/cotizaciones -H "Content-Type: application/json" \
  -H "X-Api-Key: shp_prueba_Jp5sX1cV8nB3yG7tQe2Kv0a19d" \
  -d '{"origen":"Quetzaltenango","destino":"Antigua Guatemala","peso_kg":2}'
```

6. Correo real con Resend

Shapi envía todos los correos a Mailpit. Con `compose.servidor.yml`, Mailpit guarda una copia y **reenvía a Resend** los que tienen un destinatario real. No reenvía los dominios de la siembra, del guion y de los

manuales (`enviosxelaju.com`, `mercaditoantigua.com`, `cafedelvolcan.com` y los demás), ni `ejemplo.com`, `example.com`, `shapi.tech` y `*.localhost`. Así, las cuentas de demostración nunca envían correos a dominios ajenos.

1. Cree una cuenta en resend.com (el plan gratuito permite 100 correos al día).
2. En **Domains → Add Domain**, agregue el dominio y copie al DNS los registros que muestra: el TXT `resend._domainkey` (DKIM), los CNAME `send` y `rsend` y el TXT `_dmarc` (`v=DMARC1; p=none;`). Deje **Enable receiving** apagado. Espere a que el dominio quede **Verified**.
3. En **API Keys**, cree una clave con permiso **Sending access** para el dominio. Se muestra una sola vez.
4. Guárdela en el `.env` del servidor editando el archivo, sin pegarla en una terminal ni en un chat:

   ```
   ssh -t -i ~/.ssh/shapi_azure shapi@<ip-de-la-maquina> "nano ~/shapi/.env"
   ```

   Deje la línea así, con la clave entre comillas simples y sin espacios: `SHAPI_RESEND_CLAVE='re_…'`. Guarde con **Ctrl+O** y salga con **Ctrl+X**.
5. Recree Mailpit para que la tome (se pierden los mensajes guardados, porque Mailpit no tiene volumen):

   ```
   $C up -d --force-recreate mailpit
   $C logs mailpit | grep relay
   ```

Si Resend rechaza la clave, el registro de Mailpit muestra `535 Authentication credentials invalid`: vuelva al punto 4. Para desplegar sin correo real, quite el bloque `mailpit` de `compose.servidor.yml`.

7. Antes de presentar

Si en un ensayo se adelantó el reloj de demostración, regréselo **primero**, porque la siembra usa la fecha de ese reloj (12.7). Bórrelo de Redis y reinicie los procesos que lo usan:

```
$C exec redis redis-cli DEL demo:reloj:desplazamiento
$C restart api compuerta trabajador
```

Después deje los datos como recién sembrados. Esto borra las organizaciones, las cuentas y las APIs que no son de la siembra (por ejemplo, las que se registraron por su cuenta en el sitio público), y restablece las cuentas de demostración:

```
$C exec trabajador dotnet Shapi.Trabajador.dll sembrar-demo --reiniciar
```

Por último, vacíe la bandeja en `https://correo.shapi.tech` con **Delete all**.

En el ambiente publicado no se demuestra el dominio propio de una API: con el DNS simulado, ese dominio termina en `.localhost` y no recibe tráfico ni certificado (ADR-36).

No use `down -v` en el servidor: borra también el volumen `caddydata`, con los certificados, y pedirlos de nuevo muchas veces activa los límites de Let's Encrypt.

Para actualizar a las imágenes más recientes de GHCR: `git pull`, `$C pull` y `$C up -d`. Si solo cambió el Caddyfile: `$C exec borde caddy reload --config /etc/caddy/Caddyfile --adapter caddyfile`.

8. Apagar y borrar

La máquina cobra cómputo mientras está asignada. Después de presentar, **desasígnela**: deja de cobrar el cómputo y las contraseñas públicas de la siembra dejan de servir. El disco y la IP fija siguen cobrándose (unos US$ 9 al mes) y conservan los datos. Encendida todo el mes, la máquina cuesta unos US$ 55 de cómputo más el disco y la IP, unos US$ 63 en total, que salen del crédito de estudiante:

```
az vm deallocate -g shapi-rg -n shapi-vm
az vm start -g shapi-rg -n shapi-vm
```

Los contenedores vuelven solos al encender, porque tienen reinicio automático. Para borrar todo, máquina, disco, IP y firewall incluidos:

```
az group delete -n shapi-rg --yes
```

Si su IP cambia y el SSH deja de conectar, actualice la regla del firewall:

```
az network nsg rule update -g shapi-rg --nsg-name shapi-nsg -n permitir-ssh-admin --source-
address-prefixes <su-ip-nueva>/32
```

13. Problemas que se pueden presentar

Problema	Causa probable	Solución
El navegador dice «La conexión no es privada» en `*.shapi.localhost`	La raíz de Caddy no está instalada, o el volumen `caddydata` se borró y Caddy creó otra autoridad	Importe la raíz como dice 12.5. En Firefox, active `security.enterprise_roots.enabled` o importe el certificado en **Autoridades**
`couldn't find env file` al usar Compose	Falta el archivo `.env` en la raíz	Créelo con `Copy-Item .env.example .env` (o `cp` en Linux) y ejecute Compose desde la raíz del repositorio
PostgreSQL no arranca o la API no se conecta: `port is already allocated` en el 5432	Otro PostgreSQL instalado en el equipo ocupa el puerto	Cambie `SHAPI_POSTGRES_PUERTO` (por ejemplo, a `5433`) y el `Port=` de `SHAPI_POSTGRES_CADENA` en el `.env`
El borde no arranca: puertos 80 o 443 ocupados	Otro servidor web (IIS, otro proxy u otro ambiente de Shapi) usa esos puertos	Detenga el otro servicio. Con `docker ps` vea si hay otro ambiente de Shapi levantado y apáguelo con su `down`
`node infra/verificar.mjs` falla con `EADDRINUSE`	La API, la compuerta o Vite ya están corriendo en 5080, 5090, 5173 o 5174	Ejecute la verificación antes de iniciar los procesos, o deténgalos primero
`docker` no responde o «Cannot connect to the Docker daemon»	Docker Desktop no está abierto o WSL 2 no está instalado	Abra Docker Desktop y espere a que termine de iniciar. En Windows, `wsl --install` y reinicie. En Linux, `sudo systemctl start docker` y verifique que su usuario esté en el grupo `docker`
La API se detiene al iniciar con un error de conexión a la base de datos	En desarrollo no se cargaron las variables del `.env` en esa terminal	Cargue el `.env` como dice 12.8 en cada terminal antes de `dotnet run`
`dotnet test Shapi.slnx` falla con errores de Docker o tiempos agotados en Windows	Varios proyectos de prueba levantan contenedores a la vez	Repita con `dotnet test Shapi.slnx -m:1`, que corre un proyecto a la vez
En Linux, Caddy responde 502 en desarrollo	Los procesos escuchan solo en `localhost` y Caddy no los alcanza desde Docker	Inicie los procesos con `--urls http://0.0.0.0:<puerto>` y Vite con `--host 0.0.0.0` (12.8)
En desarrollo, la página se recarga sola cada pocos segundos y borra los formularios	Un antivirus que analiza las conexiones cifradas (por ejemplo, Kaspersky) corta el WebSocket de recarga en caliente de Vite	Excluya `*.shapi.localhost` del análisis de conexiones cifradas, o use el ambiente productivo simulado, que sirve el frontend compilado
`siembra-demo` termina con un código distinto de 0	`SHAPI_MODO_DEMO` no es `true`, o la API todavía no estaba sana	Revise sus registros con `logs siembra-demo`. Ponga `SHAPI_MODO_DEMO=true` y repita el comando de arranque; la siembra es idempotente
La compuerta responde 503 `servicio_no_disponible`	Redis no está disponible	Revise `docker compose ... ps` y los registros de `redis`. Al volver Redis, el trabajador reconstruye la configuración desde PostgreSQL en 5 minutos como máximo, o al reiniciarlo
La compuerta responde 401 `clave_invalida` con una clave correcta	La clave se revocó o rotó, o se usa con la API equivocada	Revise la clave en B2.3 (consumidor) o A4.3 (proveedor). Las claves de producción y de pruebas son distintas
La compuerta responde 403 `api_no_disponible` o `suscripcion_inactiva`	La organización o la suscripción están suspendidas	Pague con otra tarjeta en B1.4 o B2.3, o reactive la organización en A6.2

Problema	Causa probable	Solución
El trabajador registra que no puede descifrar el secreto de una API	La API y el trabajador no usan las mismas llaves de Data Protection: en desarrollo, porque el secreto se cifró con llaves de otra carpeta (por ejemplo, las que cada proyecto creaba en `src/<proyecto>/.shapi/dpkeys` antes de la versión 1.0.0); en el ambiente productivo, porque se restauró la base sin su volumen `dpkeys`	En desarrollo, copie los archivos `key-*.xml` de `src/Shapi.Api/.shapi/dpkeys` a la carpeta de `SHAPI_DPKEYS_DIR` (con el valor del ejemplo, `.shapi/dpkeys` en la raíz del repositorio) o regenere el secreto de origen en A3.6. En el ambiente productivo, restaure el volumen `dpkeys` del mismo respaldo (14.4) o regenere el secreto
B3.1 muestra la compuerta como «degradada»	El p95 de la última hora pasa de 15 ms, por ejemplo en un equipo lento o justo después de arrancar	Caliente la compuerta con `node infra/verificar.mjs --demo` y espere la actualización de 30 segundos
Los correos no llegan	El trabajador no está corriendo, o Mailpit no está sano	Revise la bandeja en `https://correo.shapi.localhost` y los registros del `trabajador`. Los correos esperan en `correo_saliente` y se reintentan hasta 5 veces
`curl` o PowerShell no resuelven `*.shapi.localhost`	Windows PowerShell 5.1 (`Invoke-WebRequest`) no resuelve `*.localhost`	Use `curl.exe`, que sí lo resuelve, o agregue la entrada al archivo `hosts`
En Git Bash, un comando con una ruta como `/tmp/archivo` falla dentro de un contenedor	Git Bash convierte las rutas que empiezan con `/` en rutas de Windows	Anteponga `MSYS_NO_PATHCONV=1` al comando o use PowerShell

14. Respaldo y recuperación de datos

14.1 Qué se respalda

Dato	Dónde está	Importancia	Cómo se respalda
Base de datos de negocio	PostgreSQL, volumen `pgdata`	**Crítica**: cuentas, APIs, planes, suscripciones, claves (su hash), pagos, consumo, casos y bitácora	`pg_dump` en formato personalizado (14.2)
Llaves de Data Protection	Volumen `dpkeys`	**Crítica**: cifran los secretos de origen. Un respaldo de la base sin sus llaves deja los secretos sin poder descifrarse	Copia del volumen (14.3), siempre junto con la base
Redis	Volumen `redisdata` (AOF)	Media: la configuración de la compuerta se reconstruye desde PostgreSQL, pero los contadores de cuota del ciclo en curso y las métricas aún no consolidadas solo están en Redis	Copia del volumen (14.3)
Autoridad certificadora	Volumen `caddydata`	Baja: si se pierde, Caddy crea otra y hay que volver a importar la raíz	Copia del volumen, opcional
Configuración	Archivo `.env`	Alta: contraseñas y parámetros del ambiente	Copia del archivo en un lugar seguro, nunca en el repositorio

El código no se respalda aparte: está en GitHub y las imágenes, en GHCR.

Los comandos de este capítulo usan el ambiente productivo simulado. Con el proyecto por defecto, los volúmenes se llaman `shapi_pgdata`, `shapi_dpkeys`, `shapi_redisdata` y `shapi_caddydata`; `docker volume ls` los muestra. En el entorno de desarrollo, use los mismos comandos con solo `-f infra/compose.yml`: allí las llaves de Data Protection no están en un volumen, sino en la carpeta de `SHAPI_DPKEYS_DIR`, que se respalda copiándola. Con el valor del ejemplo, `.shapi/dpkeys`, la carpeta está en la raíz del repositorio y la comparten la API y el trabajador. Si cambió `SHAPI_POSTGRES_USUARIO` o `SHAPI_POSTGRES_BASE_DATOS`, reemplace `shapi` en `-U` y `-d`.

14.2 Respaldar la base de datos

Cree una carpeta para los respaldos, fuera del repositorio, y genere el respaldo dentro del contenedor y cópielo al equipo:

```
mkdir ../respaldos-shapi
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml exec postgres
pg_dump -U shapi -d shapi -Fc -f /tmp/shapi.dump
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml cp
postgres:/tmp/shapi.dump ../respaldos-shapi/shapi.dump
```

El formato personalizado (`-Fc`) va comprimido y permite restaurar con `pg_restore`. El respaldo se hace en caliente: el sistema puede seguir funcionando mientras tanto, y `pg_dump` obtiene una copia consistente. Se genera dentro del contenedor y se copia con `cp` para que el archivo binario no pase por la consola, que en Windows PowerShell 5.1 lo corrompería al redirigirlo con `>`.

Para comprobar que el respaldo se puede leer:

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml exec postgres
pg_restore --list /tmp/shapi.dump
```

La salida empieza con la fecha del respaldo, la base `shapi` y el número de entradas.

14.3 Respaldar los volúmenes

Las llaves de Data Protection y Redis se respaldan copiando el volumen a un archivo comprimido con un contenedor temporal de Alpine:

```
docker run --rm -v shapi_dpkeys:/datos -v "${PWD}/../respaldos-shapi:/respaldo" alpine tar czf
/respaldo/dpkeys.tar.gz -C /datos .
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml exec redis redis-cli
BGREWRITEAOF
docker run --rm -v shapi_redisdata:/datos -v "${PWD}/../respaldos-shapi:/respaldo" alpine tar czf
/respaldo/redisdata.tar.gz -C /datos .
```

`${PWD}` funciona igual en PowerShell y en bash. `BGREWRITEAOF` compacta el archivo de Redis antes de copiarlo. Respalde la base y las llaves en el mismo momento y guárdelos juntos, junto con una copia del `.env`.

14.4 Recuperar después de una pérdida total

Por ejemplo, en un equipo nuevo o después de `down -v`. El orden importa: primero las llaves, después la base y al final los procesos.

1. Cree los contenedores y los volúmenes vacíos, sin iniciarlos:

   ```
   docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml create
   ```

2. Restaure las llaves de Data Protection (y, si lo respaldó, Redis):

   ```
   docker run --rm -v shapi_dpkeys:/datos -v "${PWD}/../respaldos-shapi:/respaldo" alpine tar xzf
   /respaldo/dpkeys.tar.gz -C /datos
   docker run --rm -v shapi_redisdata:/datos -v "${PWD}/../respaldos-shapi:/respaldo" alpine tar
   xzf /respaldo/redisdata.tar.gz -C /datos
   ```

3. Inicie solo PostgreSQL, copie el respaldo y restáurelo:

   ```
   docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml up -d postgres
   docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml cp ../respaldos-
   shapi/shapi.dump postgres:/tmp/shapi.dump
   docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml exec postgres
   pg_restore -U shapi -d shapi --clean --if-exists /tmp/shapi.dump
   ```

4. Levante el resto del ambiente y verifíquelo:

   ```
   docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml up -d
   node infra/verificar.mjs --demo
   ```

Al arrancar, el trabajador reconstruye en Redis la configuración de la compuerta (APIs, claves, suscripciones y organizaciones) desde la base restaurada. Si el ambiente no tiene datos de demostración, use `node infra/verificar.mjs` sin opciones. Si se perdió `caddydata`, vuelva a importar la raíz de Caddy (12.5).

14.5 Volver a un punto anterior

Por ejemplo, para deshacer un cambio hecho por error. Detenga los procesos que escriben en la base, restaure y vuelva a levantarlos:

```
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml stop api compuerta
trabajador
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml cp ../respaldos-
shapi/shapi.dump postgres:/tmp/shapi.dump
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml exec postgres
pg_restore -U shapi -d shapi --clean --if-exists /tmp/shapi.dump
docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml up -d
```

`--clean --if-exists` borra los objetos actuales antes de recrearlos, así que **todo lo que se hizo después del respaldo se pierde**. Los disparadores de la bitácora impiden modificar o borrar sus filas, pero no impiden restaurar la base completa: proteja también los archivos de respaldo.

14.6 Recomendaciones

- **Frecuencia:** un respaldo diario de la base y de `dpkeys` en un despliegue real, y uno antes de cada cambio importante (actualizar las imágenes, aplicar migraciones o reiniciar la demostración). Se puede programar con el Programador de tareas de Windows o con `cron` en Linux.
- **Retención:** conserve al menos los últimos 7 respaldos diarios y uno por semana del último mes, con la fecha en el nombre del archivo.
- **Copia externa:** guarde una copia fuera del equipo, porque el ambiente productivo simulado corre en un solo equipo.
- **Prueba de restauración:** restaure periódicamente un respaldo en un proyecto de Compose aparte (por ejemplo, con la variable `COMPOSE_PROJECT_NAME=shapi-prueba`), para comprobar que sirve sin tocar los datos reales. Antes, apague el ambiente principal con `down`: los dos usan la red `shapi` y los puertos 80 y 443.
- **Confidencialidad:** el respaldo contiene correos, hashes de contraseñas y claves, y datos de pagos (sin números de tarjeta). Guárdelo con el mismo cuidado que la base de datos.

15. Anexos

Anexo A · Variables de entorno

El archivo `.env.example` tiene todas las variables `SHAPI_*` con sus valores de desarrollo; después de la tabla están tres ajustes más que no aparecen en él. La columna «Por defecto» indica el valor del ejemplo. En el ambiente productivo simulado, `infra/compose.prod.yml` fija algunas variables y no las toma del `.env`, porque los servicios se encuentran por su nombre en la red de Docker.

Variable	Descripción	Por defecto	En el ambiente productivo
`SHAPI_DOMINIO_BASE`	Dominio base de los hosts del panel, los portales y las APIs	`shapi.localhost`	Del `.env`
`SHAPI_MODO_DEMO`	Habilita la siembra de demostración, el reloj de demostración y los orígenes de demostración en la red de Docker. `false` en un despliegue real	`true`	Del `.env` (por defecto, `true`)
`SHAPI_APLICAR_MIGRACIONES`	Aplica las migraciones al iniciar la API. En desarrollo se aplican siempre	`false`	Fija: `true`
`SHAPI_POSTGRES_USUARIO`	Usuario de PostgreSQL que crea el contenedor	`shapi`	Del `.env`
`SHAPI_POSTGRES_CONTRASENA`	Contraseña de ese usuario. Cámbiela fuera de un equipo local	`shapi_desarrollo`	Del `.env`
`SHAPI_POSTGRES_BASE_DATOS`	Nombre de la base de datos	`shapi`	Del `.env`
`SHAPI_POSTGRES_PUERTO`	Puerto de PostgreSQL publicado en el equipo (solo desarrollo)	`5432`	No se publica
`SHAPI_POSTGRES_CADENA`	Cadena de conexión de la API y el trabajador	`Host=localhost;Port=5432;Database=shapi;Username=shapi;Password=shapi_desarrollo`	Fija: `Host=postgres`, con el usuario, la contraseña y la base del `.env`
`SHAPI_REDIS`	Dirección de Redis	`localhost:6379`	Fija: `redis:6379`
`SHAPI_SMTP_HOST`	Servidor SMTP	`localhost`	Fija: `mailpit`
`SHAPI_SMTP_PUERTO`	Puerto SMTP	`1025`	Fija: `1025`
`SHAPI_SMTP_USUARIO`	Usuario SMTP; vacío si el servidor no pide autenticación	(vacío)	Del `.env`
`SHAPI_SMTP_CONTRASENA`	Contraseña SMTP	(vacío)	Del `.env`
`SHAPI_SMTP_TLS`	`true`: cifrado obligatorio (SMTPS en el puerto 465, STARTTLS en los demás). `false`: sin cifrar, como Mailpit	`false`	Fija: `false`
`SHAPI_DNS_MODO`	Verificación de dominios propios: `simulado` (tabla `registro_dns_simulado`) o `real` (DNS público)	`simulado`	Del `.env` en la API; el trabajador, que verifica los dominios pendientes, usa siempre `simulado`
`SHAPI_REDES_BORDE`	Redes (CIDR, separadas por comas) desde las que el borde llega a la API; solo de ellas se aceptan `X-Forwarded-For` y `X-Forwarded-Proto`. Vacío: la máquina y la red `shapi` (172.30.0.0/24)	(vacío)	Del `.env`
`SHAPI_ORIGENES_PERMITIDOS`	Hosts que la protección contra SSRF permite aunque sean internos (los orígenes de demostración)	`localhost:5101,localhost:5102,origen-envios:8080,origen-agro:8080`	Fija: `origen-envios:8080,origen-agro:8080`

Variable	Descripción	Por defecto	En el ambiente productivo
SHAPI_DPKEYS_DIR	Carpeta de las llaves de Data Protection, que la API y el trabajador comparten. Una ruta relativa se resuelve desde la raíz del repositorio (la carpeta de Shapi.slnx), así que en desarrollo los dos usan la misma carpeta aunque cada uno arranque en la de su proyecto	.shapi/dpkeys	Fija: /var/lib/shapi/dpkeys (volumen dpkeys)
SHAPI_ADMIN_CORREO	Correo de la primera cuenta de administrador, que se crea al iniciar la API	admin@shapi.localhost	Del .env
SHAPI_ADMIN_NOMBRE	Nombre de esa cuenta	Administrador Shapi	Del .env
SHAPI_ADMIN_CONTRASENA	Contraseña de esa cuenta. Cámbiela fuera de un equipo local	ShapiLocal2026!	Del .env
SHAPI_PASARELA_FALLA	Simula la pasarela de pagos fuera de servicio, para demostraciones	false	Del .env en la API; el trabajador, que cobra las renovaciones, usa siempre false
SHAPI_URL_ORIGEN_ENVIOS	URL del origen de demostración de Envíos Xelajú, que usa la siembra	http://localhost:5101	Fija: http://origen-envios:8080
SHAPI_URL_ORIGEN_AGRO	URL del origen de demostración de Agro Precios	http://localhost:5102	Fija: http://origen-agro:8080
SHAPI_SECRETO_ORIGEN_ENVIOS	Secreto que exige el origen de Envíos en X-Shapi-Secreto; vacío: no lo exige	(vacío)	Del .env
SHAPI_SECRETO_ORIGEN_AGRO	Secreto que exige el origen de Agro	(vacío)	Del .env
SHAPI_DOCKER_BIN	Ejecutable de Docker que usa infra/verificar.mjs (no lo usa la aplicación)	El de Docker Desktop en Windows, si existe; si no, docker	—

Ajustes opcionales. El código lee además tres valores de configuración de .NET que no están en .env.example ni los pasa infra/compose.prod.yml. Como variables de entorno se escriben con doble guion bajo; para cambiarlos en el ambiente productivo hay que agregarlos al environment del servicio:

Variable	Descripción	Por defecto	Proceso
Sesion__InactividadHoras	Horas de inactividad tras las que vence una sesión del personal o de un consumidor (RF-04); el máximo absoluto sigue siendo 7 días	8	API
Autenticacion__LimitePorMinuto	Peticiones por minuto y por IP permitidas en los endpoints de acceso	10	API
Pagos__DemoraMs	Demora, en milisegundos, de cada operación de la pasarela simulada	Al azar entre 300 y 800	API y trabajador

Variables del servidor de Azure. infra/compose.servidor.yml (12.11) lee dos variables más del .env del servidor. No están en .env.example, porque no se usan en un equipo local:

Variable	Descripción
SHAPI_MAILPIT_HASH	Hash bcrypt de la contraseña de la bandeja de Mailpit (usuario equipo), generado con caddy hash-password. Va entre comillas simples
SHAPI_RESEND_CLAVE	Clave de API de Resend con permiso de envío, con la que Mailpit reenvía los correos con destinatario real. Va entre comillas simples

Además, `infra/compose.prod.yml` fija `ASPNETCORE_ENVIRONMENT=Production` y `ASPNETCORE_URLS=http://+:8080` en la API y la compuerta, y `DOTNET_ENVIRONMENT=Production` en el trabajador. `COMPOSE_PROJECT_NAME` (de Docker Compose) permite levantar un ambiente aparte, con sus propios volúmenes.

Anexo B · Estructura del repositorio

```
shapi/
├─ docs/
│   ├─ specs/            ← especificaciones (fuente de verdad)
│   ├─ plan/             ← protocolo, convenciones, calendario, tareas y bitácoras
│   ├─ pdf/              ← documentos de requisitos y de diseño en PDF
│   ├─ manual-tecnico.md ← este manual (y Manual_Tecnico.docx)
│   └─ lineamientos.md   ← reglas del curso
├─ mockups/              ← pantallas aprobadas
├─ contratos/openapi/    ← contratos HTTP de la API de control
├─ src/
│   ├─ Shapi.Dominio/          ← entidades, reglas y máquinas de estado
│   ├─ Shapi.Aplicacion/       ← casos de uso e interfaces
│   ├─ Shapi.Infraestructura/  ← EF Core, migraciones, Redis, SMTP, DNS y pasarela simulada
│   ├─ Shapi.Api/              ← API REST
│   ├─ Shapi.Contratos/        ← llaves de Redis y códigos de error compartidos
│   ├─ Shapi.Compuerta/        ← YARP y la tubería de filtros
│   └─ Shapi.Trabajador/       ← procesos en segundo plano y siembra
├─ tests/                    ← dominio, API, compuerta, E2E (Playwright) y carga (k6)
├─ frontend/                 ← panel, portal y paquetes compartidos (pnpm)
├─ origenes-demo/            ← APIs de Envíos Xelajú y Agro Precios
├─ infra/                    ← compose.yml, compose.prod.yml, compose.servidor.yml (Azure),
Caddy, borde y verificador
├─ scripts/                  ← tareas, CI, verificación del equipo y PDF
└─ .github/workflows/        ← integración continua y publicación
```

Anexo C · Comandos frecuentes

Qué	Comando
Levantar todo con la demostración	`docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml --profile demo up -d --build`
Verificar el ambiente sembrado	`node infra/verificar.mjs --demo`
Ver el estado de los contenedores	`docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml ps`
Ver los registros	`docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml logs -f`
Reiniciar la demostración	`docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml exec trabajador dotnet Shapi.Trabajador.dll sembrar-demo --reiniciar`
Respaldar la base	`docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml exec postgres pg_dump -U shapi -d shapi -Fc -f /tmp/shapi.dump`
Apagar sin borrar datos	`docker compose --env-file .env -f infra/compose.yml -f infra/compose.prod.yml down`
Estado del plan de tareas	`node scripts/tareas.mjs`

Anexo D · Referencias

- Repositorio: github.com/jordin-garcia/shapi
- Especificaciones: `docs/specs/` (visión, glosario, requisitos, roles, casos de uso, arquitectura, modelo de datos, compuerta, cobros, seguridad, interfaz y decisiones)
- Documento de Requisitos y Documento de Diseño: `docs/pdf/`
- Plan de desarrollo: `docs/plan/` (protocolo, convenciones, calendario, tareas y bitácoras)

- Pruebas de carga: `docs/pruebas-de-carga.md` y `tests/carga/README.md`
- Lineamientos del curso: `docs/lineamientos.md`
- Documentación de las tecnologías: .NET, YARP, PostgreSQL 16, Redis, Caddy, React y Docker Compose